

Make the Noise Wik(i) : Proactive Information Retrieval using Wikipedia Concepts

Candidate Number : SKRS6[1]

MSc Data Science and Machine Learning,
Department of Computer Science

Supervisors
Prof Emine Yilmaz, University College London
emine.yilmaz@ucl.ac.uk
Research Assistant Sahan Bulathwela, University College London
m.bulathwela@ucl.ac.uk

Submission date: September 2022

[1]**Disclaimer:** This report is submitted as part requirement for the MY DEGREE at UCL. It is substantially the result of my own work except where explicitly indicated in the text. *Either:* The report may be freely copied and distributed provided the source is explicitly acknowledged
Or:
The report will be distributed to the internal and external examiners, but thereafter may not be copied or distrbuted except with permission from the author.

Abstract

Open Educational Resources hold great promise of democratizing education globally yet have a problem of equal accessibility which can be traced back to the majority of users being self-motivated learners with an idea of what they are looking for. A larger problem in this domain is search-engine working on noisy textual data emanating through means of scans from optical character recognition or automatic speech transcription of audio/video lectures. An effective, scalable solution for Noisy Information Retrieval (IR) thus can act as a bridge to fill the gap between the users with a pre-existing knowledge of things as compared to those without. The study aims to explore a solution for Noisy IR by learning from the advancements made in spoken document retrieval in conferences such as TREC (Text Retrieval Conference), CLEF (Conference and Labs of the Evaluation Forum, NTCIR, and MediaEval, the most recent one being end-to-end neural architectures being less effective than sophisticated probabilistic models. The study explores answers to four major questions (1) What kind of architectures provide the best baseline performance? (2) Does a combination of effective approaches lead to even better results? (3) Can a simple, scalable Natural Language Processing (NLP) technique enhance the performance of the architectures providing appreciable results? (4) Can Proactive IR using Wikipedia Concepts further the performance of state-of-the-art approaches? To answer these questions the study picks the challenging task of segment retrieval on a diverse dataset encompassing varying degrees of formality without being domain-specific as in the case of news-broadcast data and audio-video lectures-Spotify Podcast Dataset released for TREC 2020. The study provides an exhaustive comparison between sophisticated probabilistic approaches such as BM25, PL2, DPH, and Dirichlet LM, with state-of-the-art Learning to Rank approaches such XGBRanker, Coordinate Ascent along with a comparatively new neural re-ranking approach using the Expand Mono-Duo. Demonstrating merit in the learning of sophisticated probabilistic models being better, the study expands on these approaches by using linear combinations, Query Expansion, NLP, reproducing state-of-the-art results, and proposing a Proactive IR approach using Wikipedia concepts which provides appreciable gains in early precision. The study also annotates and creates two down-sampled versions of the Spotify Podcast Dataset- *Podcast Small* and *Podcast SE* for future use by other researchers.

Accompanying code can be found at https://github.com/tab-mus-33/Noisy-IR

Contents

1 Introduction **2**

1.1 Background . . . 3
1.1.1 The road to OER: 'Open' before OER . . . 3
1.1.2 Why OER? . . . 3
1.2 Motivation . . . 3
1.2.1 Not more OER but a better "Bridge" . . . 3
1.2.2 What is the 'gap'? : A brief overview of user statistics: . . . 4
1.3 The Research Hypothesis . . . 5
1.4 The Dataset . . . 5
1.4.1 Why Podcast Data? . . . 5
1.5 Objectives . . . 5
1.6 Structure of the Thesis . . . 6

2 Literature Review **8**

2.1 The Dataset: Spotify Podcast Dataset . . . 8
2.1.1 Transcription and format . . . 9
2.1.2 Data Characteristics . . . 9
2.2 A brief history of Information Retrieval . . . 10
2.2.1 From 'Memex' to Univac : Giant baby steps to IR . . . 10
2.2.2 Cranfield Experiments to Ranked Retrieval . . . 11
2.2.3 70s - 90s: The birth of TF-IDF, BM25 . . . 11
2.2.4 Revisiting Test Collections: The birth of TREC and modern retrieal conferences . . . 12
2.2.5 Retrieval Conferences : What makes them "relevant"? . . . 12
2.3 Spoken Document Retrieval : History and Related Work . . . 13
2.3.1 TREC Spoken Document Retrieval Track 1997-2000 . . . 13
2.3.2 Cross-Language Speech Retrieval 2005-2007 . . . 14
2.3.3 NTCIR Content Retrieval 2010-2016 . . . 14
2.3.4 Mediaeval 2011-2015 . . . 15
2.4 Important IR Concepts : Building a basic intuition . . . 15
2.4.1 Vector Space Model: A basic intuition . . . 15
2.4.2 Probabilistic Retrieval, Language Modelling and Divergence From Randomness . . . 16

2.4.3 Machine Learned Ranking . . . 17
2.4.4 Neural Retrieval . . . 19
2.4.5 Proactive Information Retrieval . . . 19

3 Methodology **21**
3.1 The Task: Segment Retrieval . . . 21
3.2 The Dataset : Challenges . . . 21
3.2.1 Segmentation Task . . . 22
3.2.2 Topics: Training . . . 23
3.2.3 Topics: Testing . . . 25
3.2.4 Topics : Description . . . 27
3.2.5 Topics : Types . . . 27
3.3 Models . . . 27
3.3.1 Baseline 1: BM25 . . . 27
3.3.2 Baseline 2: DPH . . . 28
3.3.3 Baseline 3: PL2 . . . 28
3.3.4 Baseline 4: Dirichlet LM . . . 29
3.3.5 Baseline 5: Learning to Rank XGBRanker . . . 30
3.3.6 Baseline 6: Learning to Rank Coordinate Ascent (FastRank) . . . 31
3.3.7 Baseline 7: Nueral IR Expando Mono Duo-T5 Re-ranker . . . 31
3.4 Evaluation Metrics . . . 32
3.4.1 The IR train-test split . . . 32
3.4.2 Precision and Recall . . . 33
3.4.3 Evaluating Ranked Retrieval : Normalized Discounted Cumulative Gain . . . 34
3.5 Framework : Python Terrier . . . 35
3.6 Hardware . . . 35
3.7 Experiments . . . 36

4 Implementations, Results and Discussion **38**
4.1 Indexing . . . 38
4.2 Experiment 1: “Conventional” Baseline Runs . . . 39
4.2.1 Simple Runs . . . 39
4.2.2 Results . . . 39
4.2.3 Discussion of Results . . . 39
4.3 Experiment 2: Combining Scores and Query Expansion . . . 40
4.3.1 Combining Scores . . . 40
4.3.2 Results . . . 40
4.3.3 Discussion . . . 41
4.4 Experiment 3: Query Expansion Experiments . . . 41
4.4.1 Increasing Terms and test results . . . 42
4.4.2 Increasing Docs and Test Results . . . 42
4.4.3 Results: Train Set and Final Query Expansion Models . . . 42
4.4.4 Discussion . . . 43
4.5 Experiment 4: Learning to Rank - XGB Ranker . . . 43

4.5.1 Approach and Results . . . 43
4.5.2 Ablation Study . . . 44
4.5.3 Discussion . . . 44
4.6 Experiment 5: Learning to Rank-FastRank (Coordinate Ascent) . . . 44
4.6.1 Approach . . . 45
4.6.2 Results : Ablation Study on Test Set . . . 45
4.6.3 Discussion . . . 45
4.7 Experiment 6: Neural Re-ranking . . . 45
4.7.1 Approach . . . 46
4.7.2 Train results . . . 46
4.7.3 Test results . . . 46
4.7.4 Discussion . . . 46
4.8 Experiment 7: Named Entity Recognition and Part-of-Speech Tagging . . . 47
4.8.1 Approach . . . 47
4.8.2 Nouns: Results . . . 47
4.8.3 Entities: Results . . . 48
4.8.4 Nouns + Entities: Results . . . 49
4.8.5 Discussion . . . 50
4.9 Experiment 8 : Proactive Information Retrieval . . . 50
4.9.1 Approach: Pre-processing the train Set . . . 51
4.9.2 Approach: Structuring output files . . . 51
4.9.3 Approach: Hypothesis Testing . . . 52
4.9.4 Results: Hypothesis Testing . . . 53
4.9.5 Approach: Information Retrieval on *Podcast Small* . . . 54
4.9.6 Results: Information Retrieval on *Podcast Small* . . . 54
4.9.7 Discussion: IR results on *Podcast Small* . . . 55
4.9.8 From *Podcast Small* to *Podcast Small Extended (SE)* . . . 56
4.9.9 Results : IR Results on Podcast SE . . . 57
4.9.10 Discussion : Results *Podcast SE* . . . 58

5 Analysis **59**
5.0.1 Effectiveness of NLP approaches on Topics . . . 61

6 Conclusion **62**
6.1 Final Proposed Model . . . 63

A Submission to PASIR 2022 **73**

B Other Charts *Podcast Small* **86**
B.1 Topic-wise Relevant vs Non-relevant Based on Cosine Similarity Score cut off at rank 1,2,3,5,10 and 20 . . . 86
B.2 Topic-wise Relevant vs Non-relevant Based on Page-rank Score cut off at rank 1,2,3,5,10 and 20 . . . 89

C Code **93**

Chapter 1

Introduction

Open-Educational Resources (OERs) verily hold the potential to act as a catalyst in democratizing education globally with a possibility of snowballing into a pedagogical renaissance in terms of how subjects can be taught, aided with supplementary resources, and ease of accessibility[17]. However, such a scenario assumes a lot of ideal prerequisites. The major one is the reliance on a highly-accurate search engine that can retrieve information requirements accurately. This major pre-requisite, in turn, relies on the content of OER, that they contain or capture the context of what the user is looking for, in turn depending on that the user knows what to look for.

In the real-world the 'ideal' scenario is far from true. Content can exist in a non-digital format which has to be read digitally through solutions like Optical Character Recognition (OCR) or in the case of a video/audio has to be transcribed using automatic speech recognition (ASR) which could lead to noise in such content eventually leading to distortion of 'context' not suitable for information retrieval. Furthermore, a large disparity can exist in how users search for the information, the disparity can in turn rely on the knowledge of the user and especially their sociological advantages stemming from multiple factors such as demography, ethnicity, no learning disability, and financial status. Over some time this disparity can get amplified in just a certain group that was able to access knowledge as compared to another which wasnt.

The project explores Noisy Information Retrieval in detail by looking at different Information Retrieval (IR) architectures, from basic and sophisticated probabilistic IR models based on Bayesian/Frequentist approaches, and machine learning solutions, to the current state-of-the-art approaches using information extraction/enrichment using Natural Language Processing (NLP) culminating into a proposed state-of-the-art approach of using Proactive Information Retrieval based on concept-based enrichment using Wikipedia concepts providing improvements of up to 30% in early precision for a noisy search engine over the current state-of-the-art non-neural approaches in training tasks, while also creating two annotated datasets derived from the Spotify Podcast Dataset[30] can be used for future research called *Podcast Small* and *Podcast Small Extended (SE)*.

1.1 Background

1.1.1 The road to OER: 'Open' before OER

The implications of the term 'Open' are rather literal here and imply open accessibility in the vein of sharing educational resources free on the internet for the social well-being of society. However, the notion of what constitutes 'Open' and the conditions subjected to sharing varies according to the context of what kind of content is being shared. The history of emphasizing "Openness" can be traced back to the Open-Source Initiative (OSI) founded in February 1998 founded by Eric Raymond and Bruce Perens. The aim was to promote open-source software, with the source code being publicly available for copying, modification, or redistribution without inviting any liability to pay royalties or fees and was guaranteed by Open-Source Licenses[118].

The OSI initiative and its success inspired David Wiley to start the 'Open Content Project' in 1998 itself, which later became the Open Content Initiative and was intended for the creation and re-usability of learning-based objectives and content guaranteed by a license analogous to the OSI. The popularity of the prior two 'Open' initiatives inspired open access initiatives for scholarly work starting post a meeting of the Open Society in 2001 at Budapest as the Budapest Open Access Initiative (BOAI) prompting similar initiatives on scholarly work such as Bethesda Statement from April 2003 and Berlin Declaration from October 2003.

The aforementioned history is by no means comprehensive but is only to highlight that idea of sharing knowledge of one kind or another didn't have a glacial rise and it was rather rapid because the internet was technically publicly accessible through a graphical browser only from 1993 [48]

1.1.2 Why OER?

The term OER was coined in 2002 at a UNESCO forum centered around the same idea of 'Openness' [56]. The idea was to provide educational resources which are easily accessible through the internet for "consultation, use, and adaptation" for non-commercial purposes. The core mission of OER can be lucidly understood in the words of Walker in aiming to be "convenient, effective, affordable, and sustainable" while being easily accessible by learners and teachers worldwide[36].

'Affordable' here implies having no cost but doesn't imply without restrictions. There are conflicting opinions on what classifies as 'restriction', 'cost', 'resource', and a detailed discussion can be found in[36].

1.2 Motivation

1.2.1 Not more OER but a better "Bridge"

If we imagine the world, as an infinite library, with an infinite collection and doors in every country, with different librarians sitting on each door with different knowledge sets, a logical question to ask would be, *is the knowledge of all the librarians the same?* Because thats the most important aspect which would determine who would be able to find the information most accurately.

The OER library on the internet is seemingly 'infinite' library we are talking about here, and the search engines are the 'librarians'. This means the barrier to bridging the gap between how

users could access knowledge from the developed and the developing world, will be 'bridged' by a search engine.

In the current scenario, we need a better 'bridge' and not more content. Currently around 2,000 courses are available from various universities around the world with more OER coming up from universities in Canada, Brazil, Hungary, Iran, Russia, India, Vietnam, Ireland, South Africa, Spain, the United Kingdom, Portugal, the Netherlands, and the United States[56]. It must be noted that the term 'infinite' is purely hyperbolic here to provide a visual metaphor along with another visual metaphor of a 'library'. A library because it describes the real imagined intent behind 'OER' in the words of Atkins Et Al. further described in the vein of Nobel Laureate, Amartya Sens spirit - "to expand people's substantive freedoms through the removal of unfreedoms", that is poverty, limited economic opportunity, inadequate access to education and knowledge, deficient healthcare, and oppression[8][102]. OER is designed to increase human capital and remove barriers from knowledge, and our motivation thus is to 'strengthen' the bridge, so it is equally accessible to anyone with the desire of 'traveling' to the realm of knowledge.

1.2.2 What is the 'gap'? : A brief overview of user statistics:

While a comprehensive study around OER users is not available, we would narrow down our discussion to look at one of the most popular OER platforms, *MIT OpenCourseware*. In as early as 2005, it was noted by Carson that the website traffic was predominantly from North America, specifically between November 2003 to October 2004 in which 36% visitors were from North America alone. Western Europe and East Asia both contributed to 16% of user=share, while Eastern Europe and Latin America both contributed to around 11%. Close to 9% each of user-share came from Africa, Eastern Europe, the Middle East, the Caribbean and Central Asia combined, and the Pacific[24]. In an evaluation report published by MIT in 2011, around 90 million users had accessed the website with 127 million hits as of October 2011. The traffic was more 'global' as 56% of users were non-US. 18% percent of users came from East Asia, with 12% from Western Europe, 9% from South Asia, 4% from Latin America, and 13% from other regions[88].

It is also important to note that as per the 2011 evaluation report 41% of the traffic came via search engines, 28% was direct, 24% from other references, and 4% from MIT's website. Another pivotal statistic is that close to 50% of the users are self-learners, with less than half of them being teachers and students from other educational institutions. Therefore, if we were to guess the user profile, we are looking at a self-motivated individual with already good access to knowledge, by good access we mean that the user knows sufficiently what to search and can perhaps keep trying to look for what they want with multiple queries.

No official evaluation reports are available beyond 2011 although the last available monthly statistics report published by MIT OCW cites around 267 million visits up till September 2018.

1.3 The Research Hypothesis

Informed from the previous discussion on OER, we have already established the motivation behind our project with one additional point being that we want our solution to be in the vein of AIEd[40]. Further informing our choices from the results published in [61], we hypothesize the success of sophisticated IR systems, can be furthered by looking at computationally less expensive means such as Proactive Information Retrieval using the same sophisticated probabilistic models such as the BM25, DPH, PL2, and Query Expansion Techniques on these architectures. We explain Proactive Information Retrieval in detail in the next chapter and explain our methodologies in detail in Chapter 3 along with a description of the models used.

To explore our hypothesis we not only look at the solutions we mentioned but also look at state-of-the-art Machine learning or Deep Learning approaches to inquisitively explore if Machine Learning solutions or computationally expensive Deep-learning approaches or (Neural IR) can provide better solutions than sophisticated probabilistic models on noisy IR.

1.4 The Dataset

1.4.1 Why Podcast Data?

To test our hypothesis of proactive IR on noisy text, we choose the Spotify Podcast Dataset [30] which is the largest publicly available spoken document retrieval dataset. The dataset has been generated using state-of-the-art automatic speech recognition (ASR) on podcast audio of around 2 terabytes.

A dataset made of podcast transcripts provides a compelling choice for noisy IR primarily because of the diversity in the genre of podcasts, they can range from being completely formal (fiction, non-fiction) in various degrees of formality to being completely informal to the degree of being completely colloquial. This provides i) a variety in style suited to more generalizability because of not fitting one 'type' of content when juxtaposed to previously spoken retrieval datasets in spoken retrieval conferences (TREC and CLEF) like broadcast news and video lecture transcripts. And ii) The transcription has inherent noise due to transcription of the audio of various degrees of formality, different accents, and diversity in the structure of content. For example, an ASR algorithm can make common mistakes like human audio transcribers based on the accent and emphasis on a particular sound by a speaker. Such as incorrectly perceiving 'their' as 'there' or 'this' or 'these'. Small errors like these can occur on multiple other scenarios and lead to a transcript that can have a considerable deviation in meaning from what was originally meant. We provide details of the dataset in the next chapter.

Furthermore, the dataset also provides a compelling challenge, because of its enormous size, noise, and scale which is closer to a real-world search-engine scenario on OER websites.

1.5 Objectives

While our overarching aim is to effectively suggest a solution for noisy information retrieval in the vein of AIEd, we have an exhaustive list of objectives we wish to accomplish through this

discussion :

1. We aim to extend upon the work done in Spoken Document Retrieval in conferences such as TREC, and CLEF, by not only reproducing state-of-the-art approaches but by also proposing work which can provide further improvements in noisy IR on spoken documents
2. Exploring the effectiveness of different architectures and approaches on noisy IR. We compare Probabilistic retrieval, query expansion approaches, Machine Learning (ML) based approaches such as Learning to Rank, Neural Information Retrieval, and Natural Language Processing for information extraction.
3. Providing key observations, and approaches, for future discussion, along with a dataset which could be used to answer or ask even questions on proactive IR or noisy IR.

To accomplish our objectives we list our modelling approaches here

1. BM-25
2. DPH
3. PL2
4. Dirichlet Language Modelling
5. Linear combination of scores of Probabilistic models
6. Query Expansion using Probabilistic Modelling
7. Learning to Rank: XGBRanker and Coordinate Ascent
8. Neural Re-ranking using Expando Mono-Duo
9. Wikipedia Enrichment of Data + Information Needs and using probabilistic approaches

For reproduction of state-of-the-art results, we look at approaches by the Dublin City University [81] published in [61]. We not only aim to reproduce the results but suggest considerable improvements for better early precision.

We hypothesize, implement, evaluate, and juxtapose all our approaches on the Spotify Podcast dataset, using the training and testing topics provided along with their provided relevance judgements using the Python Terrier Framework [74].

1.6 Structure of the Thesis

We have already provided a brief background of OER along with the motivation behind our discussion. In the second chapter, we do a literature review of the dataset, information retrieval, and spoken document retrieval conferences, and provide an overview of basic IR concepts to set an intuition for our third chapter.

In the third chapter, we discuss our methodology, explaining the challenges owing to the large size of the dataset, moving further to explain our different models and approaches, followed by our

evaluation metrics, the framework used to conduct experiments, the hardware used, and listing our experiments.

In the fourth chapter, we provide the implementations of our experiments, followed by the results and discussion of each of the implementations. In the fifth chapter, we analyze the overarching insights followed by the final chapter in which we conclude our discussion.

Chapter 2

Literature Review

In this chapter, we provide a literature review of the Dataset, followed by a brief history of IR, and Spoken document retrieval. We then move to set up a basic intuition behind our methodology in the next chapter.

2.1 The Dataset: Spotify Podcast Dataset

Our dataset here contains over 100,000 such transcripts from around 60,000 hours of audio data resulting in the largest corpus of transcribed speech data. The data is also provided with labels for the task of segment retrieval as well as the task of summarization along with metadata which contains creator-provided descriptions of the over 100,000 episodes[30].

The dataset was randomly sampled from 105,360 English podcast episodes published between January 1, 2019, to March 1, 2020, on Spotify with 10% of the podcasts being sampled from professional creators with high production values and the other 90% coming from amateurs creating the podcast using a mobile application on their smartphone. The language determination didn't only rely on the creator-provided label of the language used but was also identified using automated language detection applied to the description provided by the creator. However, despite the two-step process, the creators found that the dataset wasn't strictly mono-lingual and contains few non-English podcasts. This happened primarily because several non-English podcasts had their descriptions written in English, defying, according to the creators the notion of mono-lingual cultures. However, though purely speculative, it can also be argued that in some countries this could mostly be due to the nature of users on the platform who can juggle between different languages, one of them being English. There could be various reasons for what demographic of users access the platform but it is beyond the scope of our discussion. Other scenarios for which non-English podcasts ended up in the dataset were instances of language learning podcasts where English was the language of instruction or podcasts where religious texts were being read in the original language but were being analyzed in the English language.

As indicated in chapter 1 the podcast dataset is considerably diverse in terms of genres and covers a wide range of topics from culture and lifestyle, sports and recreation, news, storytelling, health, commentary, and documentary. What adds to the diversity, as mentioned before, is the variety of formats in terms of how the podcast is structured. Which could be several speakers, the

overall structure in general, degree of formality, improvisational or scripted, narrative oriented or conversational.

2.1.1 Transcription and format

The transcripts of the dataset were generated using Google's Cloud Speech-to-Text API [2] with word-level time alignments for each word, speaker diarization, punctuation, and casing. From a noise perspective, the transcriptions had an overall word error rate of 18.1% over the diverse dataset with a named-entity-recognition accuracy of 81.8%. The error rate is higher when juxtaposed to other highly optimized corpora like Switchboard with a word error rate of less than 5%. As also cited by the authors of the Spotify podcast dataset, the high error rate can arise from a domain mismatch between the training data and podcast data. This is a problem that arises from how well or poorly a particular model generalizes on unseen data or data that represents a not-so-domain-specific scenario. Whereas, the other related datasets like Switchboard achieve a fairly low error rate only through optimization with the ASR algorithm.


```
{"results": [{"alternatives": [{}]}, {"alternatives": [{}]},
{"alternatives": [{"transcript": "Hey, hey. Hey. Hey. Hey,
this is your girl Jenny and Kita. This is the keys to the new
Gene. Yes the keys to the new generation. We're giving it to
you straight. No chase the truth unfiltered. We ain't on
nobody's Airways. But I also we going to say what we want to
say when we want to say it. Basically, we don't give a [redacted]
we gonna talk about [redacted] what now, but we go keep it ratchet.
", "confidence": 0.7978801727294922, "words": [{"startTime":
"23.900s", "endTime": "24.100s", "word": "Hey,"},
{"startTime": "24.100s", "endTime": "24.600s", "word": "hey.
"}, {"startTime": "24.600s", "endTime": "24.700s", "word":
"Hey."}, {"startTime": "24.700s", "endTime": "24.900s",
"word": "Hey."}, {"startTime": "24.900s", "endTime": "25.
400s", "word": "Hey,"}, {"startTime": "25.400s", "endTime":
"25.600s", "word": "this"}, {"startTime": "25.600s",
"endTime": "25.700s", "word": "is"}, {"startTime": "25.700s",
"endTime": "25.900s", "word": "your"}, {"startTime": "25.
900s", "endTime": "26.400s", "word": "girl"}, {"startTime":
"26.400s", "endTime": "27.100s", "word": "Jenny"},
{"startTime": "27.100s", "endTime": "28.100s", "word":
"and"}, {"startTime": "28.500s", "endTime": "29.700s",
```


Figure 2.1: Snippet from a .JSON transcript

A sample from the transcript of the dataset with word-level time stamps and tags has been shown in Figure 2.1.

2.1.2 Data Characteristics

The total episode count of the podcast is around 100,000 all available as discrete JSON files with a time-level stamp as shown in Figure 1 above. All the episodes come from 18,376 shows with 52% of the shows having more than one episode in the dataset. The average podcast episode

duration is 33.8 minutes with 5,700 transcribed words with a large variance. Descriptions provided averaged 85 words per podcast episode. The most common categories (based on what creators provided in the RSS feed) weighted by the total length of the episode are (in increasing order): Fiction, Government and Organization, News Politics, Science, Education, Society Culture, Health Fitness, Sports, and Comedy.

A very small set of creators provided the geographical location of the episodes and consequently, it's available only for 2,223 episodes with 67% of them coming from the US. Other prominent regions include Great Britain, Canada, Australia, and India (in decreasing order).

The speaker diarization using Google's Cloud Speech-to-Text API revealed a median speaker duration of 110 seconds but the inference could be erroneous as the diarization was found to be noisy by the authors after they manually corroborated the results from a random sample of 20 episodes with 11 wrongly determining the number of speakers and 4 having errors around speaker boundaries.

For comparison with other corpora, the Spotify Podcast Dataset has considerable linguistic differences from written corpora as noted by the authors in differences with the Brown corpus. The Brown corpus for example has much less emphasis on first-person pronouns and amplifiers as compared to the Spotify podcast dataset. As the largest such corpus available especially as a transcription of diverse spoken content, juxtaposition with other corpora and studying features of the dataset can be of interest from the point of view of research in linguistics.

2.2 A brief history of Information Retrieval

2.2.1 From 'Memex' to Univac : Giant baby steps to IR

Our visualization of an Infinite library finds an eerie similarity in what became the precursor to the field of IR, the 'Memex' by Vannevar Bush in "As We May Think"(1945) [16]. Though memex was entirely hypothetical, it was based on Bush's work from 1938-1940 on improving Emanuel Goldberg's photoelectric microfilm selector developed in the 1920s. Goldberg a pioneer in photography and photoelectric retrieval, had developed the first very high-resolution microfilm[47]. Microfilm was a storage medium to store a scaled-down version of documents known as the microform.

Photoelectric retrieval of documents however didn't function in the manner modern information retrieval works, but a good analogy to how Bush conceptualized memex would be to look at any modern-day search engine. Bush essentially conceptualized a template to solve the problem of IR, an ideological template that would iteratively grow and refine itself as an idea. Coupled with the transistor revolution which birthed digital computing, the template has eventually manifested itself as the modern-day search engine. For example, Bushs memex could find any number of documents with related information or what Bush called 'specific coding'[23]. Any modern-day user of the internet would perhaps exclaim that memex sounds like any modern-day search engine and that perhaps is a testament to how ahead of its time Bush's hypothetical machine was.

However, what must be noted is that Bush didn't conceptualize indexing the documents and rather had a low opinion of indexes and classification schemes[100][112]. It wasn't until 1948 that Holmstrom talked about 'Univac', a machine capable of searching text references associated with a subject code with both the code and text being stored on a magnetic steel tape[53][15].

Throughout the 1950's multiple computer-based IR projects were being run with one notable system by General Electric that searched through 30,000 document abstracts[82].

2.2.2 Cranfield Experiments to Ranked Retrieval

The most fundamental advancement in bolstering early attempts in IR came in the early 1960s through the Cranfield Experiments. In the early 1960s Cyril Cleverdon and his colleagues at the College of Aeronautics investigated how to index document content for further effective retrieval[113][29]. Until this point, a classical approach was to use numerical codes on collection items for indexing. Dewey's Decimal system was a common way to assign such numerical codes. Another method proposed by Taube et al.[108] was the Uniterm system to index documents using keywords.

Cleverdon's group conducted experiments that concluded that using 'Uniterms' or keywords for indexing proved to be as effective or better than using classical techniques for retrieval effectiveness[28]. The findings though intuitive in retrospection were scrutinized heavily at that time but later due to being correct paved the way for how retrieval effectiveness was evaluated and researched.

The most notable feature of how Cleverdon's group conducted experiments was to measure the retrieval effectiveness of different indexing methods on the same document set. Cleverdon's approach solved variability in the comparison of different indexes by using what we now know as a 'test collection' and began to be known as the Cranfield Paradigm. The Cranfield experiments in essence demonstrated an effective way of using document collections to hypothesize, verify, produce, and compare insights about retrieval effectiveness primordially in an experiential way by providing an insight which at the time seemed counter-intuitive.

The late 1950s and the early 1960s had another important idea take shape, ranked retrieval. The emphasis of Information retrieval before Luhn's proposal[69] and its testing by Maron et al.[75] was on Boolean retrieval. Ranked retrieval was a different approach to this where a relevance score for each document was calculated for a given query and then all the documents were sorted according to the score. The highest-ranking documents according to the score were then returned to the user as top documents. The testing by Maron et al. further prompted Luhn to suggest the idea of *term frequency* at his time known as 'word significance'. Luhn suggested that the frequency of a word occurring in a document can be a useful measure of significance[70].

Another significant development during the 1960s was led by Gerard Salton. The idea to view both the queries and documents as vectors belonging to the N-dimensional space, where N is the number of unique terms in the corpus, was a pivotal standpoint in ranked retrieval first proposed by Switzer and later refined by Salton in what would become Cosine Similarity[100].

2.2.3 70s - 90s: The birth of TF-IDF, BM25

The 1970s saw many advancements with the most notable being Jones's work on complementing Luhn's proposal of term frequency (tf). Jone's introduced the idea of inverse document frequency (IDF) stating that the frequency of the word occurring in a document collection is inversely proportional to how significant it is in retrieval. Through idf Jone's argued that less common words referred to more specific concepts and hence were more important in retrieval. Combining tf and idf for a $tf \times idf$ based approach for retrieval was widely adopted thereafter. Formalizing

retrieval processes continued rigorously and Salton along with his group proposed the vector space model, spiraling into existence multiple retrieval processes in the next two decades, that is, the 80s and 90s.

Marton et al's idea of using probability theory for retrieval was also explored extensively during the 1970s. Probabilistic retrieval had its beginnings through the probability ranking principle proposed by Robertson[95]. The idea was to rank documents based on probabilistic measures by defined evaluation measures. The approach was later extended by both Jones and Robertson[1] and also by Rijsbergens derivation of a probabilistic model.

During the 80s many variations of tf-IDF weighting schemes were developed [99] along with more formalization of retrieval models. TF weighting schemes were applied to probabilistic models with the work eventually leading to the BM25 ranking function which remains a highly effective retrieval function despite the new-age advancements in retrieval including machine-learning and neural retrieval. Latent Semantic Indexing (LSI) was another very important idea that was born during this phase from attempts in improving the basic vector space model[34].

2.2.4 Revisiting Test Collections: The birth of TREC and modern retrieal conferences

The idea of using a test collection proposed by Cleverdon and his group didnt evolve with rapidly growing IR research in terms of size, especially when juxtaposed to real-world scenarios, and eventually seemed unrealistic to even some Cranfield practitioners by the beginning of the 1990s [96]. In 1992 the National Institute of Standards and Technology (NIST) founded TREC, particularly through the efforts of Vorhees and Harman to build test collections that were comparable to commercial collection sizes[114].

TREC's success paved way for many other retrieval conferences such as Cross-Language Evaluation Forum (CLEF), NII Testbeds and Community for Information access Research (NTCIR), and Forum for Information Retrieval Evaluation (FIRE) all of which have resulted in a plethora of commercial-scale test collections for a multitude of tasks to look at specific domains and spoken document retrieval is one such task.

2.2.5 Retrieval Conferences : What makes them "relevant"?

We must understand what makes retrieval conferences so pivotal. In the words of Vorhees, we can say that they provide an "abstraction" for a real user search task, something Jones calls "competency of search"[60]. The salient features of Pre-Cranfield TREC give us a fair idea of understanding such an "abstraction".

- **Test Collection**: Intuitively enough, a test collection consists of a set of documents and queries. The queries are referred to as "topics" in the context of the collection. Other than a set of documents and information need statements, that is, "topics" there is also a relevance judgment mapping available for each topic listing which document should be returned for them.

- **"Runs"**: A ranked list of documents retrieved in response to each topic using a retrieval system is called a "run". An IR researcher can make many such "runs" on a test collection.

- **Ranking**: The ranking of documents is reflective of which one is the most relevant concerning a certain topic.
- **Evaluation**: The relevance judgments provided for each "topic" are used to compute the decided evaluation metric and the metric score is indicative of the average of the metric over the entire set of topics.
- **IR Systems**: The "runs" on a certain collection can be produced using different IR systems and for different "runs" the same evaluation metrics can be computed and compared. The IR system returning the best scores on the choice of evaluation metric is considered to be more effective.

With major assumptions being:

- Relevance of a certain document is a measure of how similar it is to a certain information need, that is, topic. This means all documents are equally desirable and independent of any other document's relevance to a certain topic. It is also assumed that the topic/information need is static.
- The relevance judgments provided for a given topic are representative of actual user populations.
- The relevant files for each topic are known.

The assumptions are centered around constituting an 'abstraction', that is, a building block for what a putative search system should be like. It must also be noted that the original Cranfield tests focussed on precision and recall of a test-set rather than a ranked list approach[62].

2.3 Spoken Document Retrieval : History and Related Work

Our research hypothesis aims to solve noisy information retrieval emanating from ASR transcripts generated through the Google speech-to-text API. We discussed the details of transcription and the word-error rate in 2.1.1. While our dataset is pivotal from the perspective of spoken word retrieval, especially in terms of size, diversity as a genre, and the challenge of noise, spoken document retrieval has seen many advances in the last two decades.

2.3.1 TREC Spoken Document Retrieval Track 1997-2000

TREC Spoken Word Document Retrieval Track which ran from 1997-2000 is the best-known work of spoken document retrieval[61]. The test collection focused on the retrieval of broadcast news from radio and television by increasing the size and complexity of each edition. The task was split into two parts to study spoken document retrieval i) through baseline transcripts generated through then state-of-the-art ASR and ii) through near-accurate manual transcripts of the broadcast. Initially, the documents consisted of manually created news segments but later automated start points in unsegmented news broadcasts were also explored. The idea was to compare retrieval effectiveness on both the error-free and error-ridden documents to understand how Information

retrieval would work on a noisy dataset and what approaches would lead to retrieval effectiveness similar to the non-noisy dataset. It was found that the same level of performance could be achieved with ASR transcripts if external data sources like contemporary news text archives were used[45].

2.3.2 Cross-Language Speech Retrieval 2005-2007

Cross-Language Evaluation Forum (CLEF) was initiated to research on Cross-Language Information Retrieval (CLIR) and Multilingual Information Retrieval (MIR) with a primary focus on European languages[59]. Rapidly growing multimedia technologies however invited a lot of interest and initiatives in spoken and visual content. During the years 2005-2007 CLEF ran a spoken retrieval task focussing on Cross-Language-Speech Retrieval (CL-SR) focussing on spontaneous conversations[89]. This was done through a large collection of oral archives in the form of personal testimonies. ASR transcripts along with meta-data such as:

- Manually and automatically assigned descriptors for concepts present in each testimony were provided.
- The descriptors also included things like dates, locations, and person names associated with the testimony.
- Expert-written summaries of segments from the testimonies which included all the key events were also provided and the "topics" were very carefully designed.

The aim of the task was to correctly identify starting points for cohesive stories in each of the testimonies. The cohesive starting points or boundaries were manually identified by experts. Overall, the task concluded that it was challenging to find such starting points and that conversational data of the type which was used in the track often didn't capture important entities within dialogues to aid in IR. As a consequence, search queries didn't often didn't match with the relevant segments which didn't have any associated entities in the dialogues. This is in complete contrast to data such as news broadcast data which would have prominent entities in dialogues. However, effective usage of the provided metadata greatly improved the retrieval effectiveness but the improved performance was only reflective of an unrealistic scenario for such detailed meta-data generated manually by domain experts would not be available for a large, real-world dataset.

2.3.3 NTCIR Content Retrieval 2010-2016

The NTCIR content retrieval from 2010 to 2016 focussed on content retrieval from lectures and technical presentations in Japanese. The task had two phases – i) the first only focussing on spoken content retrieval and ii) focussing on spoken queries which added more complexity to the task. The biggest challenge of the task was the transcription of content, primarily because of how specialized the vocabulary of the domain-specific content was[4][3]. Topics were provided to the participants to locate relevant content in the transcripts. The fine granularity of how the relevant content was labelled with respect to each query was a unique feature of the dataset and enabled a very detailed analysis of retrieval methods used on the dataset along with studying the relationship between how spoken word queries affected the retrieval performance and how the overlap between relevant documents and queries changed with spoken word retrieval.

2.3.4 Mediaeval 2011-2015

The Mediaeval conference that ran from 2011 to 2015 focussed on datasets that can perhaps be thought to be somewhat similar in the range of diversity as our dataset. The conference focussed on Rich Speech Retrieval, Search, and Hyperlinking tasks[39][38][65]. The datasets were different in different years but the main task was to find "jump-in" points in the spoken content related to the provided "topics". The task started with the Blip10000 dataset which consisted of content from blip.tv platform consisting of SPUG or semi-professional user-generated content. Later the conference used the broadcast content provided by the BBC[37]. All the datasets were accompanied by state-of-the-art ASR transcripts of the content along with search queries. Other tasks included ad-hoc search and known item search and the relevance assessment was done using crowdsourcing. A strong emphasis was laid on evaluating the submissions comparatively and identifying the trade-off between the ranking of retrieved content and how accurately were the "jump-in" points identified in the retrieved content.

2.4 Important IR Concepts : Building a basic intuition

In this section, we will look at some important IR concepts albeit not very granularly. The aim is to establish an intuitive understanding to understand the experiments and models used in the next few chapters.

2.4.1 Vector Space Model: A basic intuition

In a vector space model, queries and documents are represented as vectors in an N-dimensional space where N is the number of unique terms in the set of documents. The query vector can be represented as $\vec{V}(q)$ and the document vector can be represented as $\vec{V}(d)$.

An effective way to look at the similarity between a document and a query would be to calculate the Cosine Similarity between the two in the vector space. Mathematically Cosine similarity between two vectors $\vec{V}(q), \vec{V}(d)$ can be calculated as:

$$Cos(\vec{V}(q), \vec{V}(d)) = \frac{\vec{V}(q).\vec{V}(d)}{\|\tilde{\mathbf{V}}(\mathbf{q})\| \times \|\tilde{\mathbf{V}}(\mathbf{d})\|} \tag{2.1}$$

A visualization of Cosine similarity can be seen in Figure 2.2.

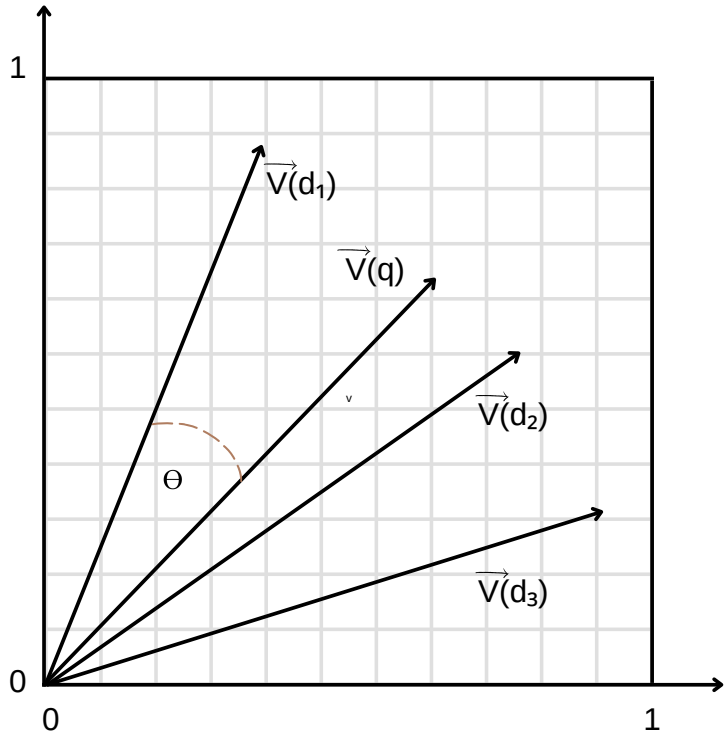


Figure 2.2: Cosine Similarity between document and a query

2.4.2 Probabilistic Retrieval, Language Modelling and Divergence From Randomness

Language modelling (LM) uses statistical inference for IR and has arguably produced some of the most efficient ranking functions which are capable of outdoing the modern approaches which use state-of-the-art neural networks in certain scenarios. The basic assumption is that the document is a sample of the population and LM entails computing the probability that a query can be generated by the document in question. The approach can be quite complex and may require compounding/mixing two probability distributions with the first distribution to model the document and the second to model the *collection*. Unlike the Vector Space Model in the last section which we covered intuitively; these models contain a parameter determined by the topics in the training data along with *relevance* generally assigned by human assessors. The *parameter* can vary with size, type of data, and query length and can often vary for different collections.

The 80s[10][58], 90s[14][12][91], and much later even the 2000s[64] have been witness to such ranking functions with later work focussing more on frequentist approaches rather than Bayesian deploying Divergence From Randomness (DFR) framework to build models like the DLH[5], a variation of which (DPH) we will be using as a Baseline in the next chapter.

A deeper mathematical explanation of probabilistic, LM and DFR frameworks in IR is beyond the scope of our discussion hence we will restrict ourselves only to developments and ideas. The basic intuition behind the Probability Ranking Principle which assumes that the relevance of a

particular document is binary and given a certain *information need* can be summarized through an intuitive notation such as[94]:

$$P(rel|d,q) = \sum_{q,tf_i} w_i \tag{2.2}$$

Equation 2.2 simply implies that the Probability of relevance given a document and query is equal to the summation of weights of query terms. While the prodigal "BM25" is much different from LM and DFR approaches and owes its roots to the PRP, it doesnt take a simple approach like equation 2.2 and is rather elegant as we will see in the next chapter when we define our baselines.

2.4.3 Machine Learned Ranking

Ranking can also be done using Machine Learning (ML) techniques and the domain which uses ML to solve the problem of ranking is called *Machine Learned Ranking* with the problem being referred to as *Learning to Rank*. Unlike "conventional" methods which we have discussed so far, ML solutions do not require manual tuning (when juxtaposed to parameter-oriented models) and can combine multiple features whilst avoiding over-fitting[68]. ML models can take up training in two ways the first being i) featured-based training and ii) discriminative training. In feature-based training, the document is represented by a set of feature vectors, this can also be a pre-existing ranking function such as a BM25, TF-IDF Weight Scheme, Cosine Similarity, and so on. In discriminative training, the model trains on training data and can be defined easily through "four pillars of discriminative training" which use i) the input space ii) the output space iii) the hypothesis space, and iv) the loss space.

The first official introduction of MLR as a framework for ranking documents in IR is credited to Norbert Fuhr in 1992 through a paper titled "Probabilistic Models in Information Retrieval" which discussed parameter estimation and what Fuhr called "advanced models for document representation"[42]. The first unofficial introduction, however, happened earlier through Fuhr's other work in 1989 where he defined a Least Squares Polynomial Retrieval Function[41]. MLR since has since been rife with ranking functions that can be categorized into three algorithmic categories i) point-wise ii) pair-wise, and iii) list-wise.

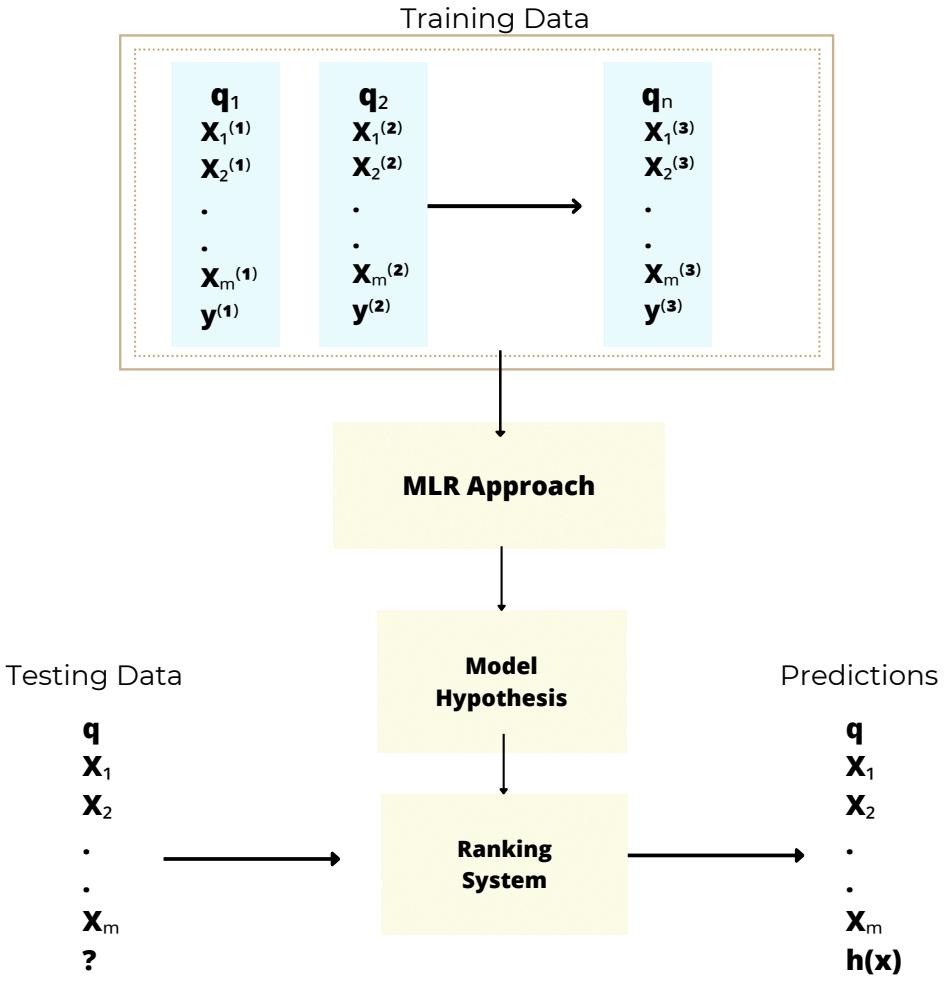


Figure 2.3: A simple Learning to Rank Framework[68]

The point-wise approach is rather more direct in the sense of how it looks at data. The training set of data consists of (q_i, d_j) document, query pairs where i belongs to the set of queries and j belongs to the set of documents. Each such point is trained through supervised algorithms with as the relevance values using ML algorithms such as logistic regression, polynomial regression, ordinal regression, and so on.

In the pair-wise approach for each query , where i belongs to the set of queries, we train it with a pair of documents (d_{j1}, d_{j2}) as a binary classification problem whilst minimizing a loss function. Popular examples of this approach are RankNet[20], LambdaSmart[21], XGBoost[26] Ranker.

In contrast to the above two approaches, the list-wise approach optimizes evaluation measures such as Normalized Discounted Cumulative Gain (NDCG) or mAP (mean average precision) on the set of queries present in the training data. Notable examples include LambdaMart[21], SVM-MAP[119], among others.

A comprehensive history of MLR along with a review and analysis of the conceptual framework can be found in [68]. A visualization of a simple MLR framework inspired by [68] can be seen in figure 2.3.

2.4.4 Neural Retrieval

Neural Retrieval finds its eponymous roots in IR applications that use a Neural Network (NN). The network could either be deep or shallow but for something to qualify as Neural IR it must specifically use a NN to rank and not just use NLP applications like ASR or named entity linking to classify as one[80]. For example, our dataset includes text transcribed by a NN through the Google Speech to Text API, a simple model developed such as a BM25 will not qualify our model as an example of Neural IR.

Neural IR however became of significant interest to researchers only with the "third-wave" of NN approaches with state-of-the-art results in many tasks ranging from Computer Vision, Natural Language Processing (NLP), and Automatic Speech recognition (ASR)[87]. The "third-wave" has inspired applications not only in textual retrieval but also in Wan et al.'s "Content-based image retrieval" [115] and other multimodal approaches which take an amalgamation of multiple types of content such as sound with text or images and text[73][72] however we are only going to restrict our discussion to textual retrieval under Neural IR.

The single most important juncture in the adoption of Neural approaches to NLP, that is, translating the "third-wave" of success of Neural approaches from Computer Vision applications to language process tasks happened after the introduction of *Word2Vec* in 2013 [77]. The use of word embeddings along with Neural Networks paved way for early Neural IR. The first approach however was four years before Mikolov et al's seminal paper introduced *word2vec* and used an *auto-encoder* for semantic modelling to search for documents[]. Deep Structure Semantic Model was proposed by [55] in 2013 for ad-hoc retrieval and led to a lot of further work ranging from Deep Semantic Similarity Model [44],Learning semantic representations using Convolutional Networks (CNNs) [103], representing queries with distributed representations [78], query auto-completion [79],and a recommender system proposed by [104] for modelling temporal behaviour.

However, using embeddings as representations can be limiting and is difficult for commercial web-search applications because of the size of data involved resulting in layer size becoming "unmanageable" for both training and inference [122] while 'word-hashing' to reduce the dimensions by representing word as 'trigrams' has been proposed as a solution, we see in a large dataset this can fast become a limitation. The challenge of adapting to varying levels of granularity concerning architecture and achieving the same success as conventional IR approaches with full-text indexing has been studied by [31].

For an overall understanding of architectures, *Deep Learning* concepts and terms, [66] presents an exhaustive introduction while a comprehensive account of Neural IR can be studied through [87],[122], with a more recent introduction and tutorial by [80].

2.4.5 Proactive Information Retrieval

Is more "complex" always better? From the results published after the first edition of TREC Podcast Track, it is evident that "conventional" models with rather creative approaches from

looking at Named-Entity Recognition and coupling it with a combination of "conventional" models fare much better than the more sophisticated approaches that look at neural retrieval[61].

Other effective approaches such as [43] look at not only word-embeddings and neural re-ranking but do this with the caveat of combining the topic with the entire description. The task is computationally expensive yet still uses a non-neural architecture as the first ranker, that is, the sequential dependence model. Given the noise in the dataset, a Transformer based embedding approach would rather lead to a "loss" of meaning. It can be understood from a simple example such as "I am talking about dogs in the backyard", could have a completely different meaning from "I am talking about docs in the backyard", especially with regards to a Neural IR approach which relies on an end-to-end "learned" meaning.

Proactive Information Retrieval as a field looks at the approaches that "recover" the lost context by appending queries/documents with additional information without onerous computation through a putative, "deeply-connected" NN. The key approach is to harvest user intent with little effort by building systems that can add relevant signals to the documents and queries to increase the overlap. One approach has been rather explicit in terms of how it is approached in terms of the user experience, involving asking the user further questions about what they are looking for[111]. Other has been using demographic data of the user to recover the lost context[117].

It can be argued however asking questions to the user doesn't provide for the most seamless of experiences and is rather an impediment. Using click-through data and impressions from previous search results of the user can be a suitable mitigation in this regard[117][18]. Such concept-based modelling is commonplace in the social-media domain which builds on data of user interaction for a better idea about user preferences. Queries can then be appended with the extra information or more formally "expanded"[83][9], alternatively "zero-effort" queries[121] can be created or search results can be personalized[106].

Social media concept-based modelling involves building a user profile through the history of a user's social media posts[90]. The approach results in a rather "fine-grained" feature space which can be too large from the perspective of increasing recall. Topic detection using *Linear Discriminant Analysis* is a popular approach to reduce such a huge feature space and compensate for increasing the recall[120]. Annotating data through the use of experts is another approach that has been successful in the domain of Education[32] but is rather cumbersome, expensive, and has very little potential for scalability.

Our final proposed solution in this paper will look at a rather novel approach to concept-based modelling by arguing for the use of Wikipedia-linked entities[13] to not only improve performance but also a greater overlap. We would investigate this incisively by first building experiments and then testing the significance statistically before proceeding to implement it in our search engine.

.

Chapter 3

Methodology

In the previous two chapters, we have built the motivation, context, history, and conceptual framework for our discussion. In this chapter we will define the methodology we will use to conduct our experiments to solve noisy information retrieval. First, we will start by extending our discussion about the dataset from 2.1 and its subsections by looking at the segmentation task in our noisy IR task followed by defining the models we will use for our experiments. Third, we talk about the evaluation we use to assess the success of our IR systems in the discussion, and towards the end, we discuss our choice of the framework along with the hardware used and conclude by listing our experiments.

3.1 The Task: Segment Retrieval

Our task is to build a segment retrieval search engine on a noisy dataset. Although we have already talked about our aim multiple times, the overall aim is to find "jump-in" points in a podcast for a particular information need. The segments are of 2 minutes with a minute of overlap in between. For example, if a listener wants to know about a podcast that talks about "Dragon Ball Z Final fight", our task would be to build an IR system efficient enough to retrieve the most relevant 2-minute segment for the user, that is, a "jump-in" point in the podcast. Lets say the podcast show is called "Top 10 Anime Battles" and it has a mention about the "Dragon Ball Z final fight" somewhere between 2 minutes to 2 minutes 40 seconds, then the ideal retrieved podcast jump-in point would be "Top 10 Anime Battles" 2 minutes to 4 minutes. We will discuss more segments and how they were made in the next section.

3.2 The Dataset : Challenges

The biggest challenge as reported by [61] was the overwhelming size of the dataset totalling around 90 Gigabytes. As can be seen from table 3.1 with statistics taken from [61], out of 285 email sign-ups which had an appreciable conversion of 213 registrations, only 7 participants eventually participated in the search task. A questionnaire was sent to the participants to understand the challenges, and while only 10 responses were received, the challenge stated was the "overwhelming" size of the dataset.

Since the data is not readily usable and requires some significant pre-processing to extract the segments could be another impediment in starting up the search task.

Activity	Count
Email list sign-ups	285
In TREC slack channel #podcasts 2020	194
TREC podcasts registrations	213
Signed data sharing agreement	77
Downloaded transcripts	64
Downloaded audio	18
Participated in Search	7
Participated in Summarization	8
Participated in Both	2

Table 3.1: Participation Statistics TREC Podcast TRACK 2020[61]

Beyond pre-processing, that is, coming up with segments, which are 2-minute segments taken from each podcast episode, the challenge is of choosing the hardware to build an IR system. Having such a large dataset proved difficult to process even while using a high-performance cluster (HPC) and a Google Colab Pro + subscription, especially with the NN-based architectures. The indexing as well as any attempt to pre-process the segments obtained also proved to be difficult being plagued with high-ram usage, crashes after 25-30 hours of computational time with state-of-the-art GPUs like the Nvidia Tesla P100 and Nvidia Tesla V100 with close to 30 Gigabytes of ram. In the next subsection, we will discuss more the segmentation task.

3.2.1 Segmentation Task

The primary requirement of the segmentation task at the TREC 2020 Podcast Track was to build to 2-minute segments with a one-minute overlap which would eventually culminate into close 3.4 million segments in all with a word count of 340 ±70 words.

In figure 2.1 we looked at a snippet from one of the JSON files in the dataset. The 'transcript' field of the JSON file can be seen in the figure along with time-level stamps below. The data structure of the text can also be observed from the snippet, which shows the transcript of the text is in a dictionary nested inside a list data structure. The first key "transcript" gives the transcript of the text, another key "words" provides time-level stamps of words in the transcript with keys "start-time", "end-time", and "word". The structure of time-level stamps can be seen in figure 3.1


```
"words": [{"startTime": "23.900s", "endTime": "24.100s", "word": "Hey,"}, {"startTime": "24.
100s", "endTime": "24.600s", "word": "hey."}, {"startTime": "24.600s", "endTime": "24.
700s", "word": "Hey."}, {"startTime": "24.700s", "endTime": "24.900s", "word": "Hey."},
{"startTime": "24.900s", "endTime": "25.400s", "word": "Hey,"}, {"startTime": "25.400s",
"endTime": "25.600s", "word": "this"}, {"startTime": "25.600s", "endTime": "25.700s",
"word": "is"}, {"startTime": "25.700s", "endTime": "25.900s", "word": "your"},
```


Figure 3.1: "Words" field from a JSON file

For the segmentation task, we created 2-minute segments using the supplied time-stamps with

a 1-minute overlap in-between. More exhaustively explained, the first segment ran from time-stamps 0 seconds to 119.9 seconds, the second segment ran from 60 seconds to 179.9 seconds, and so on.

After our segmentation task, we found the total number of 2-minute segments to be greater than 3.4 million segments, that is, 3,593,512. The number was closer to 3.6 million segments in all. To keep these documents manageable and optimize the ram, a script was written to convert these documents with a one-to-one correspondence to the JSON files. More lucidly, an XML containing segments from the JSON corresponding was created and saved on the disk, the process was repeated for all the 105,362 JSON files.

Example of an extracted 2-minute segment is shown below in figure 3.2.

```
<DOC>
<DOCNO>00CWg03og7RjcCbKxXdSUN_0-120</DOCNO>
Before I continue one of the ways, we keep all of our content for you.The listener free of charge is our amazing sponsors,
and today anchor is one of those sponsors. If you haven't heard about anchor it's the easiest way to make a podcast.
Let me explain it's free. There's creation tools that allow you to record and edit your podcast right from your phone or
computer anchors going to distributor podcast for you. So it can be heard on Spotify Apple podcasts and everywhere podcasts are
listened to and you can even make money from your podcast with no minimum listenership. It's everything you need to make a podcast
in one place download the free anchor app or go to Anchor Dot f m-- to get started tonight. We discuss how Fools Rush In And
full Killers rush to kill.Have you ever wanted to see Loki working the door at Studio 54 you will plus there was a dark
Phoenix movie this weekend. We're going to talk a little bit about that and a lot about the dark Phoenix comics and oh,
yeah, you know, we love the X-Men animated series Marvel TV weekly starts. Now you are tuned in to AfterBuzz TV the ESPN.
Of tv top now let them go welcome to Marvel TV weekly also known as Marvel movie news TV addition slash Saturday edition.
Yeah. All right. I'm Christy black joined as always by the lovely and talented my very own the Marvel TV Weekly's very own Kitty Pryde.
Is he Anderson? I am saving all of these intros because the last one I saved was so nice this one so nice God Christian after the show.
It's not going to be like this. But for now, this is great. I'm xia Anderson. Hell, let's see if I wanted to be a jerk. I would say
sitting next to his Hank McCoy, but no see that's not cool. I love Hank McCoy, but I'm calling you a beastly is that these
the thing is it's like it's like you would want to mean it to be an insult, but it's really not he's awesome because he's
 amazing and like a genius so and he messes up stuff all the time, but she's
</DOC>
```

Figure 3.2: A 2 minute segment extracted into trec xml format from JSON files

It must be noted that the document tags and the formatting were done as per the *trecxml* format to be used by our choice of framework for this project.

A comprehensive analysis along with visualizations can be found in [30].

3.2.2 Topics: Training

For training purposes, a training set of topics was provided.

Topic ID	Topic	Bad (0)	Fair(1)	Good(2)	Excellent(3)	Perfect (4)	Total
1	coronavirus spread	9	16	52	2	0	79
2	greta thunberg cross atlantic	34	12	17	7	0	70
3	black hole image	40	9	13	16	0	78
4	story about riding a bird	68	8	4	2	1	83
5	daniel ek interview	69	7	4	2	1	83
6	michelle obama becoming	25	7	5	2	0	39
7	anna delvey	78	7	7	2	0	94
8	facebook stock prediction	69	10	2	2	0	83

Table 3.2: Training topics and distribution of their segments from the qrels file

The 8 training topics along with the counts of their graded relevance judgments from the qrels file have been show in table 3.2. As can be seen only two topics, 4 and 5, have one perfectly relevant segment available.

The relevance of the topics was judged by NIST assessors and their relevance was graded on a scale of 0 to 4. With 0 being classified as "Bad", 1 as "Fair", 2 as "Good", 3 as "Excellent" , and 4 as "Perfect". All the assessments are made concerning jump points for the segmentation task which essentially entails finding segments that are most relevant to a human listener[61]. The distribution of only the relevant segments (No Bad) concerning each training topic can be found in figure 3.3.

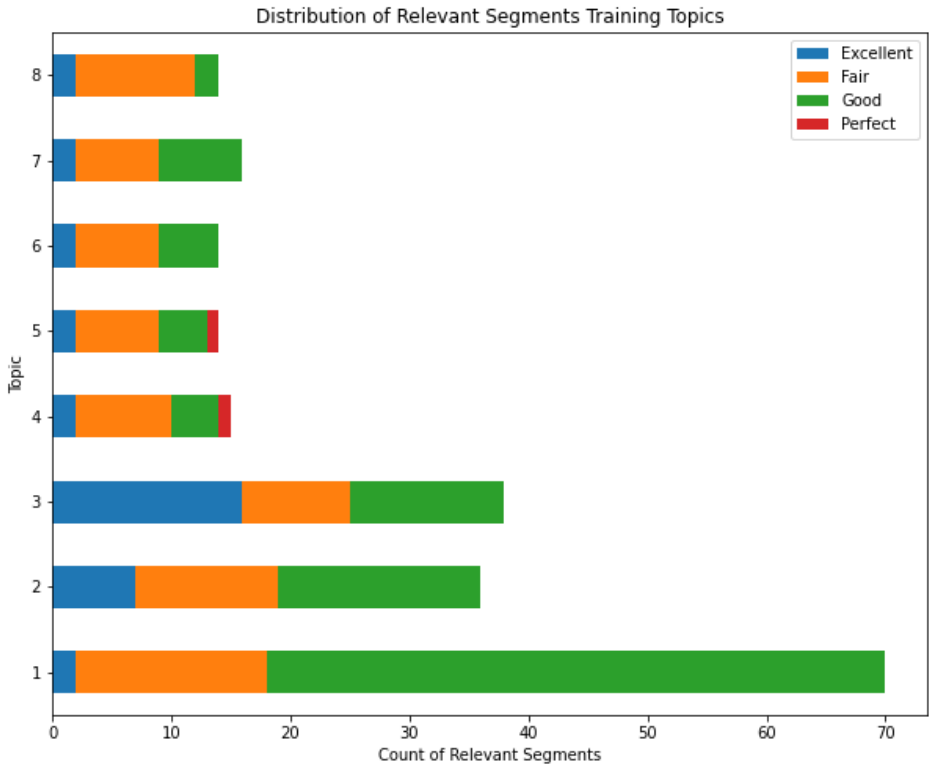


Figure 3.3: Distribution of Relevant Segments Per Training Topic

3.2.3 Topics: Testing

Topic ID	Topic	Bad(0)	Fair(1)	Good(2)	Excellent(3)	Perfect(4)	Total
9	trump call ukrainian president	99	18	11	0	0	128
10	boeing 737 crash causes	102	6	8	15	15	146
11	how to cook turkey	152	40	10	7	0	209
12	imran khan career	159	7	0	0	0	166
13	drug addiction recovery	190	7	0	0	0	197
14	near death experiences	84	17	34	17	0	152
15	podcast about podcasts	186	30	45	43	2	306
16	causes and prevention of wildfires	91	25	32	10	26	184
17	time between meetings	230	18	14	0	0	262
18	women in stem	141	38	2	0	0	181
19	ai in healthcare	136	24	13	9	0	182
20	cost of childcare	162	28	23	1	0	214
21	juneteenth	131	13	6	2	0	152
22	chernobyl hbo	150	26	12	2	1	191
23	notre dame fire	197	9	5	0	0	211
24	france yellow vest protests	146	7	4	0	0	157
25	black lives matter	154	20	0	0	0	174
26	bob woodward	94	30	26	12	0	162
27	civil rights protest stories	104	10	15	27	30	186
28	yo yo dieting	94	5	47	14	6	166
29	racism in canada	92	42	30	5	0	169
30	motherhood	183	24	1	0	0	208
31	horoscope reading cancer	232	7	0	0	0	239
32	giants game december 22	252	4	1	0	0	257
33	hvac industry environmentalism	172	37	11	12	4	236
34	halloween stories and chat	253	2	8	5	5	273
35	living debt free	106	49	25	15	0	195
36	cryptocurrency risks	155	55	18	4	0	232
37	slow travel	199	13	3	0	0	215
38	workplace diversity	76	13	35	21	0	145
39	social media marketing	91	25	43	22	11	192
40	bees dying	133	5	9	26	27	200
41	gmo food labeling	113	54	9	0	0	176
42	fyre festival	206	16	3	0	0	225
43	hong kong protests	156	3	0	0	0	159
44	thanksgiving comedy special	211	0	1	2	1	215
45	drafting tight ends	103	40	33	16	0	192
46	missouri quilt mom	127	4	1	2	0	134
47	sci fi author interview mars	0	0	0	0	0	0
48	spike lee movie score	183	2	2	0	0	187
49	thrift store smell	175	1	1	1	1	179
50	coast guard coxswain	0	0	0	0	0	0
51	queer eye veteran	164	14	11	1	4	194
52	fauci interview	273	1	0	6	8	288
53	recommended books for entrepreneurs	158	86	11	1	1	257
54	bias in college admissions	136	17	10	0	1	164
55	malcolm x biography	135	25	27	4	3	194
56	gaslighting	174	0	4	3	2	183
57	nefertiti	78	14	26	33	24	175
58	sam bush interview	192	25	0	0	0	217

Table 3.3: Testing topics and distribution of their segments from the qrels file

A similar topic distribution to training topics can be seen for the testing topics in table 3.3. A distribution of relevant segments (No Bad) for the testing topics can be seen in figure 3.4.

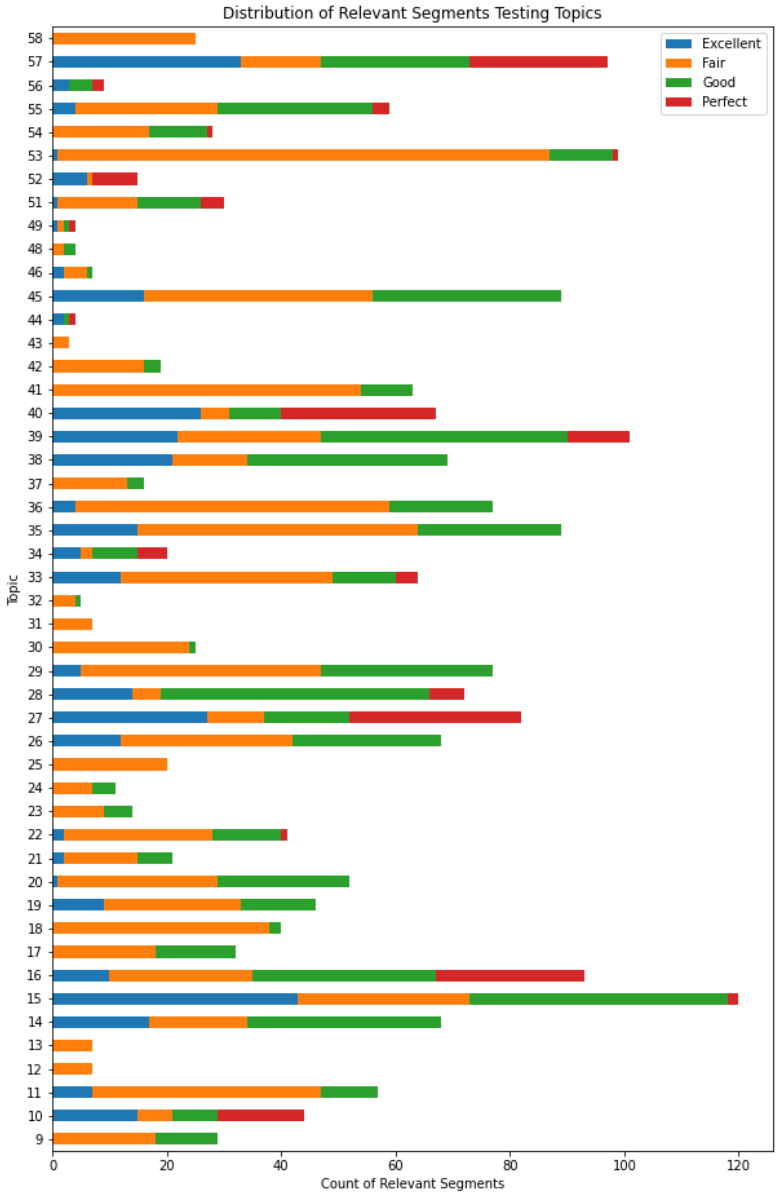


Figure 3.4: Distribution of Relevant Segments Per Testing Topic

3.2.4 Topics : Description

Along with the topics the dataset also provides descriptions of the provided topics. Considering the length of the descriptions, it would be difficult to present them in a table however we will look at some examples when we talk about our methods which involve using the descriptions for Named Entity Recognition and Wikipedia annotations in the next chapter.

3.2.5 Topics : Types

All the topics in the training and test sets have been provided with a type labelling split into three categories - 'topical', 'known-item'. and 're-finding'. Topical search pertains to search related to searching for a specific topic, a known item on the other hand would be a search task where the user already has an idea of what they are looking for [116]. in re-finding, a user trying to search for content they have already looked at and are trying to find again[110].

3.3 Models

In section 2.4 we established a conceptual framework for our discussion by looking at important IR concepts in detail. We also discussed the limitations of MLR and Neural IR evidenced in literature for noisy IR on the Spotify Podcast Dataset. Our choice of models, therefore, is informed by the success of past approaches mentioned in [61] along with the limitations of the hardware. Although we use the best available hardware in Google Colab Pro +, approaches used by the University of Maryland [43] require uninterrupted access to resources for more than 24 hours, that is, uninterrupted access to a high-performance cluster. The next few subsections will define our baselines which come from different approaches/architectures already defined in section 2.4.

3.3.1 Baseline 1: BM25

When we talked about probabilistic ranked retrieval in section 2.4, we emphasized how ranking functions developed much earlier than the current MLR or Neural IR approaches-albeit less prone to overfitting-are still relevant today with respect to their performance. Evidenced in [61], BM25 was not only a good baseline but was also a component of approaches that produced state-of-the-art results [81]. We further looked at a comparative study of BM25 [11] in which BM25 only slightly underperforms when the query is too short and another study by [71] highlighting a limitation when documents are too long.

The BM25 originally stands for *Best-Match* 25 and builds itself on the *eliteness 2-Poisson model*, a complete account of its development can be read from [94]. Mathematically BM25 can be expressed as [101](pg 233):

$$BM25(q,s) = \sum_{i \epsilon Q} \log\left(\frac{N}{df_t}\right) \times \frac{(k_1+1).tf_{td}}{K + tf_{td}} \times \frac{(k_2+1).tf_{tq}}{k_2 + tf_{tq}} \tag{3.1}$$

Where,

- N is number of documents

- df_t is the number of times t^{th} appears in the document
- $k_1, k_2, K$are parameters
- $K = k_1((1-b) + b.\frac{dl}{avdl}$
  Where,
  - dl is document length
  - avdl is the average document length
  - b has a value varying from 0 to 1000
- tf_{td}, tf_{tq} are term frequencies of term from the document and query respectively

There many variations of BM25 available such as BM25F[94] and BM25L [71], we will skip the granularity of both for our discussion

3.3.2 Baseline 2: DPH

The DPH model is essentially a variant of the DLH model based on the DFR framework. The model was introduced by [5] and is based on a frequentist statistical interpretation rather than a Bayesian interpretation with the ranking function based on a hypergeometric distribution. The D, L, and H in DLH stand for Divergence, Laplace Normalization, and Hypergeometric. In DPH, Poppers normalization is used in place of Laplace's Normalization. DLH has been evidenced in [5] with DPH being evidenced in work by the Dublin city university[81].

The weights given by the DLH model are given by [5] :

$$weight = \frac{lnf(tf||d)}{tf+1} \tag{3.2}$$

Expanded as

$$weight = \frac{tf.log_2.(\frac{\hat{p}}{P(t))}) + 0.5.log_2.(2\pi.tf.(1-\hat{p}))}{tf+1} \tag{3.3}$$

Where,

- $\hat{p}$ is the Maximum Likelihood Estimate of the term in the document
- tf is the term frequency of the term in the document
- P(t) relative term frequency of the term frequency in the document

It must be noted that the DLH model, and therefore the DPH model is parameter-free, implying that it doesnt require any training.

3.3.3 Baseline 3: PL2

The PL2 model was proposed by [51] as a normalization method on the Poisson Model to solve the *collection dependence problem*, that is, the need for re-calibration of the term-frequency normalization parameter for different document collections[27]. The model uses normalization 2 with PL2 based on the DFR framework[6].

The scoring function for given document d and segment S is[52] :

$$Score(q,S) = \sum_{t \epsilon Q} \frac{1}{1+tfn}(tfn \times log_2 \times \frac{tfn}{\lambda} + (\lambda + \frac{1}{12.tfn} - tfn).log_2 e + 0.5.log_2(2\pi \times tfn)) \quad (3.4)$$

Where,

- λ is mean and variance of a Poisson distribution
- tfn is term frequency normalization with normalization 2

The tfn is given by

$$tfn = tf.log_2(1 + c.\frac{avg_l}{l}, (c > 0) \quad (3.5)$$

Where,

- avg_l is the average length of the document
- c is the free parameter of the normalization method
- tf is term frequency

The model has been evidenced for early precision as a desirable outcome for our search engine and therefore is a choice of a baseline[51][52][107][109].

3.3.4 Baseline 4: Dirichlet LM

Dirichlet Language Modelling (LM) is a Language Modelling approach with Dirichlet Smoothing. Language modelling is intuitively built on Query Likelihood Modelling and aims at modelling the process of query generation. The ranking is done based on the probability that a query could be observed as a random sample from the document model in question. A basic LM for IR can be given by[101](pages 242-246):

$$P(d|q) \propto P(d) \prod_{t \epsilon Q}((1-\lambda)P(t|M_c) + \lambda P(t|M_d)) \quad (3.6)$$

Where,

- λ is a model parameter
- M_d is a language model of document d
- M_c is a language model of document c
- t is the term t in document d

Equation 3.7 essentially aims to capture that the document user is looking for is d[101](page 246).

The Need for smoothing can be intuitively understood from the fact the probability of finding a document for a given query can become zero if the term in the document does not appear in the query[46].

The Dirichlet Smoothing is given by :

$$P(q_i|D) = \frac{tf_{q_i,d} + \mu \frac{tf_{q_i,C}}{|C|}}{|D| + \mu} \tag{3.7}$$

We inform our choice using the comparative study by [11] in which Dirichlet Smoothing outperformed BM25 for a topic-finding task on web data.

3.3.5 Baseline 5: Learning to Rank XGBRanker

For our first LTR approach, we first look at LambdaMart[21] list-wise approach for inspiration. The LTR approach uses Multiple Additive Regression trees to train the model according to an evaluation measure, in this case, the NDCG. Our motivation to use the model comes from the success of [25] in the *Yahoo! Learning to Rank Challenge*.

In our ranker, we use the XGBoost (Extreme Gradient Boosting)[26] Ranker as the LTR approach albeit a comparative study[22] of the LTR approaches highlights the problem with LTR approaches along with caveats about how calculating NDCG differently impacts results and shows Coordinate Ascent having a slight advantage over other LTR approaches thus informing the choice for another LTR approach we define in the next subsection.

An exhaustive account of the algorithm of the model can be found in [26] but intuitively can be understood from the concept of Gradient Boosting in Trees where an ensemble of weak learning decision trees is used for predictive modelling[50].

Gradient boosting can be intuitively understood as an exercise to reduce bias by creating a class H, the hypothesis class of weak learners, with high training errors and bias. The ensemble class of weak learners is essentially decision trees with very little depth.

$$H_T(\overrightarrow{x}) = \sum_{t=1}^{T} \alpha_t h_t(\overrightarrow{x}) \tag{3.8}$$

Where, t is the iteration, and T is the total number of iterations and α is the step size. Equation 3.13 highlights one important aspect, that is, the ensemble is built in a step-wise iterative manner, which is similar to how gradient descent works but here the iterative procedure is not to change the model parameters but to add more functions which are low-depth decision trees.

The loss function can be visualized as.

$$l(H) = \frac{1}{n} \sum_{i=1}^{N} l(H(x_i), y_i) \tag{3.9}$$

After t iterations and already having a class of weak learners, at the iteration *t+1* we try to find weak learn which minimizes loss the maximum, such as:

$$h_{t+1} = argmin_{h \in H} l(H_t + \alpha h_t) \tag{3.10}$$

When the h_t is found it will be added to the ensemble

$$H_{t+1} := H_t + \alpha h \tag{3.11}$$

To find such an $h \in H$ we do a gradient descent in the function space.

A more detailed account can be studied from [50]

3.3.6 Baseline 6: Learning to Rank Coordinate Ascent (FastRank)

The Coordinate Ascent model also known as the FastRank is a list-wise learning to rank approach where the ranking function comprises of the optimally weighted linear combination of the features of the query document or in our query-segment pairs. The "coordinate ascent" derives its name from the optimization method of the model to optimize the weights, that is, coordinate ascent optimization. Since the weights are optimized for a provided metric, in our case, the NDCG, the approach is classified as a list-wise approach.

The hyperparameters to be tuned in the case of Coordinate Ascent are i) R, the number restarts from a random initial weigh, and ii) T the iterations performed after a restart[22].

The coordinate ascent method builds itself on previous linear approaches such as Logistic regression, linear discriminate model, albeit the Coordinate is novel in terms of how it looks at the parameter space[76].

The model choice was motivated by results from a comparative study of many other LTR approaches [22].

3.3.7 Baseline 7: Nueral IR Expando Mono Duo-T5 Re-ranker

Our final architecture in the discussion entailed using a neural re-ranker based on the T-5 architecture or the Text-to-Text Transfer Transformer [93] known as the "Expando Mono-Duo".

The "Expando" in the title refers to document expansion of the documents using keyword representations before indexing them. The "Mono" and "Duo" are stages in the multi-stage re-ranking pipeline of the model. The most interesting aspect of the overall architecture is the combination of point-wise and pair-wise approaches in the "Mono" and "Duo" stages of the re-ranking.

Though none of the neural approaches in [61] were proven to outperform the "conventional" models we have already chosen for our discussion, the Expando Mono-Duo T5 re-ranker was specifically chosen to understand gains after using pre-existing approaches in the ranking pipeline and also from the perspective of understanding if pre-training on a related large dataset albeit with a different kind of data profile such as the MS-MARCO dataset [84] would be beneficial to our ranking or not?

For *ad-hoc* document retrieval tasks a two-stage architecture of first using a ranking function and then re-ranking the documents has been evidenced as an effective approach with a wide consensus[7]. A similar approach using a sequence-to-sequence pre-trained Transformer such as BERT[35] has already been used and has proven to be effective[86]. The approach using the T5 architecture provides an improvement over the already suggested method using BERT[92].

It must be noted that the previous approach of *doc2query* by the authors is analogous to the "Expando" approach taken in the architecture since it is essentially the same approach of expanding the texts with keyword representations of the documents before indexing them.

The first stage retrieval in the approach is done using a simple keyword retrieval and is then followed by a two-stage re-ranking pipeline with a point-wise and pair-wise approach of "Mono"

and "Duo" both of which are based on pre-trained sequence-to-sequence Transformers architecture (in our case the T-5).

The pre-trained sequence-to-sequence Transformer is trained on document-query pairs and then is used to predict a query for a given document in question, for enrichment the predicted query is appended to the document. This is defined as the H_{-1} stage.

The H_O entails Keyword retrieval which is formally known as *candidate retrieval* in multi-stage ranking and a "conventional" ranking function such as a BM25 can be used here to get the candidate passages to be re-ranked later.

The H_1 comprises of the *Mono point-wise* re-ranking also known as a *relevance classification*[67]. A score s_i is estimated for a candidate document $d_i \in R_{n-1}$ for a query q :

$$P(\text{Relevant} = 1|d_i, q) \tag{3.12}$$

The last stage, that is, the H_2 stage involves *Duo pair-wise re-ranking*. A probability $p_{i,j}$ is calculated for candidate documents $d_{i,j}$ to estimate if the document d_i is more relevant to the query q than document d_j:

$$P(d_i > d_j|d_i, d_j, q) \tag{3.13}$$

3.4 Evaluation Metrics

Evaluating information retrieval is important to assess retrieval effectiveness. From the perspective of intuition, an important question to ask is, *how does a user look at a document after a search query?*. A rather knee-jerk reaction in terms of looking at one's past user experience in a retrospective way might reveal that a user rarely goes beyond the first 5-10 documents on the page of a search result or say the first page. Before we look into that 'knee-jerk' answer, it's important to understand the components needed to assess the effectiveness of a retrieval system i) a document collection ii) a set of information needs, formally referred to in our podcast dataset as *topics*, and iii) relevance judgments which could either have a binary of relevant/non-relevant for query-document pairs or have an ordinal grading say "Excellent", "Good", Bad", etcetera. Alternatively, there could also be pair-wise relevance or a total ordered assessment. But before we jump into defining evaluative measures it is important to note that evaluation for IR can be divided into two broad categories of *offline* and *online* evaluation. Since our discussion only involves *offline* evaluation, we will restrict ourselves to only *offline* evaluation methods.

3.4.1 The IR train-test split

To judge a test collection, an important question would be to ask, *what is the minimum number of information needs required to check evaluation on a certain test collection?* Since it would be reasonable to ask how is a certain approach to IR judged effective for a very large dataset? A minimum of 50 information needs along with the availability of their relevance judgments has been evidenced to be a reasonable test collection size by[101].

Another important is the assumption of testing and training collections. The *test* collection is only used to judge the performance of a retrieval collection and hence must not be used to tune

the model parameters or say even in MLR to learn the "ranking" of the system for it would only end-up overfitting the model to a certain information needs and will not be reflective of the actual performance of the system. To tune the parameters, in the case of a conventional approach that has such parameters, or training an MLR approach, it is important to use a *train* collection to tune/train the parameters and come up with an optimal set of parameters to later evaluate on the *test* collection. In our discussion too, we would use a *train* collection to train our approaches and a *test* collection to finally evaluate the models with other approaches presented in [61].

3.4.2 Precision and Recall

It must be noted that Precision and recall as metrics are only defined for an un-ranked scenario. that is, the ranking of documents would not matter when calculating these metrics. Although we would only be looking at Precision for our discussion with a cut-off at 10, which will imply that we are looking at Precision metrics for the first top 10 retrieved documents. By saying top-10 ranked documents we do not imply that if a more ranked document is ranked better within the first 10 documents, precision would be better or worse, rather the evaluation metric is agnostic of rank. If we had chosen a cut-off for recall, the agnosticism to rank would have been the same.

Precision is defined as the ratio of relevant documents retrieved with respect to retrieved documents. Mathematically it can be defined as:

$$Precision = \frac{\text{No. of relevant documents retrieved}}{\text{Total retrieved documents}} \tag{3.14}$$

Recall can be defined as the ratio of relevant items retrieved with respect to total relevant items in that particular set, mathematically expressed as:

$$Recall = \frac{\text{No. of relevant documents retrieved}}{\text{Total relevant documents}} \tag{3.15}$$

The two metrics however can be better understood from a contingency table

	Relevant	Non-Relevant
Retrieved	true-positives	false-positives
Non-retrieved	false-negatives	true-negatives

Table 3.4: Contingency Table

The equations 3.14 and 3.15 can then be rewritten as

$$Precision = \frac{\text{true positives}}{\text{true positives} + \text{false positives}} \tag{3.16}$$

And

$$Recall = \frac{\text{true positives}}{\text{true positives} + \text{false negatives}} \tag{3.17}$$

It can be seen from equations of both precision and recall that the evaluation metrics rather focus on true positives, that is only looking at relevant documents that the IR system has been

able to retrieve. In some scenarios, precision can be of high interest, for example, in a web each engine where the relevance of documents on the first page or in the first few results is of special interest to the user. In our use case scenario, maximizing recall for the first 10 results would thus be an important criterion.

3.4.3 Evaluating Ranked Retrieval : Normalized Discounted Cumulative Gain

An important question to ask concerning the evaluation metrics we have just defined in subsection 3.4.2 is, *how will we determine if the system has returned the most relevant documents at the best rank?* The evaluation metrics precision and recall only look at documents in the form of sets of documents, but what if you are looking at precision in the first 10 retrieved documents but the most relevant documents appear from the 6th position onwards to the 10th? Would that be a fair metric to have when compared to a system that retrieves documents from the 2nd position onwards till only the 6th? Though both the systems retrieve 5 relevant documents in the top 10, the second system performs better because the user will see the relevant documents further up in search results. Another important question to ask would be if all documents are equally relevant, or are some documents more relevant than others? Novel graded relevance judgments to account for the effectiveness of IR systems that rank more relevant documents better have solved the aforementioned conundrum. Mean average precision (mAP)[49], Normalized Discounted Cumulative Gain (NDCG)[57] are two such evaluation metrics. For our discussion, we will limit ourselves to NDCG.

Discounted Cumulative Gain till rank p was originally defined as[57] :

$$DCG_p = \sum_{i=1}^{p} \frac{rel_i}{log_2(1+i)} \tag{3.18}$$

It is important to note that a sufficient mathematical explanation was not provided when DCG was proposed and was rather chosen for the smoothing properties[33]. A mathematical justification can however be found in a theoretical analysis by Wang et al[115].

A further improvement to equation 3.18 was made by [20] in the paper proposing RankNet and can be defined as:

$$DCG_p = \sum_{i=1}^{p} \frac{2^{rel_i} - 1}{log_2(1+i)} \tag{3.19}$$

Equation 3.19 lays more focus than equation 3.18 on the importance of ranking more relevant documents first. Both the equations are based on the following assumptions [57]

- A highly relevant documents retrieval is more important than the retrieval of a document that is marginally relevant to the user
- A relevant document retrieved at a high rank is less valuable for a user since it might not ever be examined by the user

NDCG as a metric is normalized by the optimized NDCG scenario, that is the scenario in which the NDCG is maximum and all relevant documents are retrieved ideally as they should be

in the relevance judgment.

NDCG therefore can be defined as:

$$NDCG = \frac{DCG}{optDCG} \tag{3.20}$$

For our discussion, we will be looking at NDCG and NDCG at rank 30.

3.5 Framework : Python Terrier

For our choice of the framework, we had a few primary requirements. The first was to choose a framework that can make an efficient IR pipeline that can ensure easily reproducible results. The second requirement was to design or opt for a framework that has been well-optimized to process large amounts of data since our dataset with a 90GB size, sufficient conversion computation, and varied baselines would require a lot of computation and RAM usage. Our final and third requirement was to be able to look at a framework that could unify different types of approaches, that is, "conventional", LTR, Neural-IR, in a way that it would help other IR researchers without much learning curve easily adapt to the code and build further on it.

Multiple approaches with *SOLR*, *Lucene*, *elastic-stack*, were tried before finally settling with the Python Terrier[74] framework which had significant advantages over the others in terms of ease of setting up pipelines, reproducing results, and writing code which could unify all different architectures in a script which requires a very little learning curve for other IR practitioners.

All indexing, model implementations, and evaluations were done using the Python Terrier framework and a popular Natural-Language-Processing library SpaCy[54] was used for Parts-of-Speech tagging and Named-Entity Recognition.

3.6 Hardware

The most important and challenging aspect to conduct the experiments was the hardware requirements required for the IR system design, experiments, indexing, and even the initial file conversion to readily readable trec-XML format.

A table with hardware configuration has been shown in table 3.5. UCLs High-Performance Cluster, *Blaze* was also tried because of hard resets every night, most processes didnt complete and the performance was also affected by other performances running on the system therefore we skip the details of *Blaze* in our hardware listing.

	Hardware				
Device	**CPU**	**RAM**	**GPU**	**GPU RAM**	**Storage**
Laptop(cf 1)	Intel i7 8th Gen	16GB	Nvidia GeForce 1660 Ti	8 GB	600 GB Free Space
Colab Pro (cf 2)	Intel Xeon	32GB	Nvidia K80, Nvidia T4	16GB	Needed Google Drive Integration (500 GB)
Colab Pro + (cf 3)	Intel Xeon	52 GB	Nvidia P100, Nvidia V100	16GB	Extended Google Drive Integration (2 TB)

Table 3.5: List of Configurations Tried

Table 3.5 however does not show running times and result, for simplicity we present a summary of them in table 3.6.

	Usage Summary		
Device	**Training Time**	**Result**	**Comments**
Laptop(cf 1)	30+ Hours Before Crash	Insufficient Ram	Not Optimal
Colab Pro (cf 2)	Max Run time of 12 hours but crashed in 4+ Hours	Insufficient Ram	Not Optimal
Colab Pro + (cf 3)	Most task completed in 17-18 hours, Some Crashed after 24 hours	Sufficient for most architectures	Similar or Better Hardware Recommended

Table 3.6: Usage summary from the configurations used

As can be seen from tables 3.5 and 3.6, *cf1* and *cf2* proved to be insufficient in terms of hardware and did not meet the requirements of the experiment runs for our discussion. *cf1* was plagued by continual crashes and running of memory. *cf2* had sufficient CPU resources but most tasks required more RAM than what was offered by the configuration, furthermore, a maximum run-time of 12 hours proved to be a bottleneck if the tasks needed to be optimized with less memory usage and longer run times. Therefore, as a final option, the experiments were run on the hardware provided by Googles Colab Pro+.

It must be noted however that while section 3.3 provides a list of baselines and approaches used, it only lists the final approaches used and other neural architectures, pre-processing approaches were tried on *cf 3* which made the hardware and maximum run-time feel severely limited resulting in crashes and inconclusive results.

A list of failed approaches that were tried are

- Colbert end-to-end Dense Retrieval [63]. **Reason:** the session ran out of GPU resources despite training on a maxed out Tesla V100 with 24 hours of maximum run-time on *cf 3*. A batch-wise approach was tried and led to a failure despite in total of 40 hours of computation time.
- docT5Query query-prediction [85]. **Reason:** The GPU, and RAM resources proved to be insufficient despite a batch-wise approach similar to Colbert.
- Other pre-processing approaches were tried which led to very large run times because of optimization for memory. We tried a summarization approach of documents inspired by the work of [105], a text normalization approach by [97] among others.
- We do not aim to extend further on the discussion of the failed approaches since they are not relevant from the perspective of the discussion that follows and also because of the hardware requirements.

3.7 Experiments

In Chapter 1 we talked about our hypothesis of improving noisy IR using concept-based modelling, a background of which we have provided in section 2.4.5. Through our experiments we aim to accomplish i) understanding the difference between various approaches based on different architectures, and if they can be further optimized ii) reproducing state-of-the-art results in [61] by the Dublin City University[81] and iii) improving the state-of-the-art results by proposing a novel approach using proactive information retrieval.

Each of our experiments can be structured in the following manner. First, we run the baselines or the bare minimums of a particular approach we take, if the results prove to be better or

comparable to those listed in [61], we improve on the approach by taking more creative approaches such as combining it with other well-performing models, expanding topics or queries using terms from the top document and re-retrieving the top results and if the second set of experiments provides promising results, we dig further into the best performing approach.

Our first experiment only looks to replicate the simple 'runs' of top-performing baselines in [61] using models we have defined in section 3.3. Our second and third experiments expand on the first experiment by using various approaches like combining scores, intersection and union of top-ranking documents, and query expansion.

In our fourth and fifth experiments, we explore LTR approaches to evaluate their success and investigate if it is worthy to expand on them for our discussion. In the sixth experiment, we do the same albeit we look at Neural re-ranking using the Expando-Mono-Duo approach and explore if it is a useful approach for noisy IR.

For our seventh experiment, we look at a combination of approaches taken by [81] and reproduce state-of-the-art results. The eighth experiment for proactive information retrieval builds further on the seventh experiment in trying to not only inquisitively solve for noisy IR by using both enriched topics and segments but also by using approaches successful in the seventh experiment to produce state-of-the-art results and provide insights for further discussion.

Chapter 4

Implementations, Results and Discussion

In this chapter, we provide implementations of our experiments, diving deeper into the questions we asked when we listed our experiments in section 3.7. All the implementations are followed by results and a small discussion.

4.1 Indexing

Indexing the collection seemed a rather herculean task. All the configurations in table 3.5 were tried. And this was after trying multiple frameworks for the experiments mentioned in section 3.5. After the choice of *Python Terrier* was made, *cf 1* had already been ruled out by using approaches such as *Solr* and *elastic-stack*, the initial Python Terrier experiments were run on *cf 2* and were prone to memory crashes and were split into-batches, after much struggle, *cf 2* was upgraded to *cf 3* and Python-terriers in-built indexing was used to index the collection of 105,362 XML files generated using the segmentation scripts to convert JSON files into the *trec-xml* format.

With *cf 3* the high-ram version of 52GB was able to index the collection with close to 12 hours of run-time. Python terriers in-built *TREC Collection Indexer* was used, statistics of the indexing have been shown below.

Field	**Collection Statistics**
No. of documents	3593829
No. of terms	209849
No. of postings	324936225
No. of tokens	500325872

Table 4.1: Index Statistics From Python Terrier

In python terrier the default indexing pipeline removes stopwords and uses *Porter-Stemmer* as a pre-processing step.

4.2 Experiment 1: "Conventional" Baseline Runs

Our first experiment was to tune/run the baselines on the training topics and then evaluate them for the best performance. The idea was to evaluate the performance of the baselines later on in the test set of the collections. All the approaches listed in section 3.3 were tried and the models were trained on the training topics and then tested on the testing topics provided.

4.2.1 Simple Runs

In this section, a simple approach was taken to run the baselines. For BM25, PL2 model tuning was performed by performing a grid search on their respective parameter spaces. NDCG at rank 30 was kept at the metric to be optimized. Baselines of DPH and DirichletLM were also run. The results on Train Set were compiled and are presented differently for both train and test sets in the next section.

4.2.2 Results

The training results for the "vanilla" conventional runs were as follows. For tuning, BM25, the optimal parameters were found to be a value of 0.6 for b and a value of 0.5 for k_1. For the Pl2 model, the value of c was found to be 20.

Train Set Results			
Model	**NDCG**	**NDCG at 30**	**Precision at 10**
BM25	0.41	0.27	0.32
DPH	0.42	0.27	0.32
PL2	**0.42**	**0.28**	**0.36**
QL	0.4	0.25	0.35

Table 4.2: Train Set Runs after tuning "Conventional Models"

The results on the test set were as follows and have been compared to results from TREC submission in [61].

Test Set Results						
Results	**NDCG**		**NDCG at 30**		**Precision at 10**	
Model	**Trec Results**	**Reproduced Results**	**Trec Results**	**Reproduced Results**	**Trec Results**	**Reproduced Results**
BM25	0.52	0.53	0.4	0.38	0.49	0.47
DPH	**NA**	0.53	**0.4**	0.37	NA	0.46
PL2	NA	**0.54**	NA	**0.38**	NA	**0.48**
QL	0.52	0.52	0.4	0.36	0.48	0.45

Table 4.3: Test Set Runs after tuning "Conventional Models" compared with Trec Results published in [61]. Our results are listed under "Reproduced Results"

4.2.3 Discussion of Results

The PL2 model after tuning from a simple grid-search outperformed every other "conventional" model on the training set as can be seen from table 4.2. We achieved an overall NDCG of 0.42 and

a precision of 0.36 with a cut-off at rank 10. For the first 10 results, this seemed rather effective when juxtaposed to other approaches that were run. The gain in precision was significant and all other scores matched or were slightly better than the other two well-performing baselines-BM25 and DPH.

In the testing set, we compare the results to what was presented in [61]. Most of our approaches had comparable results to what was presented in [61], with our PL2 approach outperforming the baseline performance presented in the paper. The Performance of our baseline had close to 3.8% gains on the TREC baselines in terms of the overall NDCG. The other metrics were similar to top performing baselines in [61]. When compared to neural approaches using BERT, the NDCG gain overall was close to 20%. Verily, this provided enough motivation to not move forward with computationally much more expensive neural approaches for which the hardware requirement was much more than the conventional approaches and required more training time on a very high-end GPU with more GPU ram and even more training time.

4.3 Experiment 2: Combining Scores and Query Expansion

In this section, we applied some other basic approaches to the "conventional" models we defined in section 3.3. This exercise was to understand if combining different models in ways like taking a linear combination of scores, taking a union of the top-ranked documents, or taking an intersection of the top-ranked documents helped improve the efficiency of the IR system. We also tried the recommended way of Query Expansion by [5] to use the DLH-inspired DPH model.

4.3.1 Combining Scores

Our first approach was to combine the best performing models from table 4.2. We took a linear combination of the scores of BM25 and DPH, and then ranked the documents again, finally evaluating them on the relevance judgement files from the training set. The second approach was to run the DPH model with query expansion, this was a basic Query Expansion approach tried to test the effectiveness of Query Expansion on noisy retrieval. We used 10 expansion terms from the top 3 documents to see if the effectiveness was increased. Our other approach was to try out a union and intersection of top-ranking documents to see if it improved the performance. The idea here was to learn about features that could be used for training LTR approaches for the rather "unconventional" baselines we defined in section 3.3. The last two models in table 4.2 were also linear combinations of the conventional models to see if the effectiveness improved.

4.3.2 Results

The results of tuning and then evaluating the IR models on the train set are presented in table 4.4 and the results of the same approaches and their corresponding test set scores are given in table 4.5.

Train Set Results			
Model	**NDCG**	**NDCG at 30**	**Precision at 10**
BM25 + DPH	**0.42**	**0.26**	**0.32**
DPH with Query Expansion	0.38	0.26	0.32
BM25 DPH union	0.24	0.02	0.025
BM25 DPH intersection	0.24	0.02	0.025
PL2 + DPH	**0.43**	**0.28**	**0.36**
BM25 + PL2	0.42	0.28	0.34

Table 4.4: Train Set Results with Combination, Query Expansion, and Union/Intersection approaches

Test Set Results			
Model	**NDCG**	**NDCG at 30**	**Precision at 10**
BM25 + DPH	**0.54**	0.38	0.46
DPH QE	**0.54**	**0.4**	**0.47**
BM25 DPH union	0.3	0.04	0.04
BM25 DPH intersection	31	0.04	0.05
PL2 + DPH	**0.54**	0.38	0.46
BM25 + PL2	**0.54**	0.38	**0.47**

Table 4.5: Test Set Results with Combination, Query Expansion, and Union/Intersection approaches

4.3.3 Discussion

The approaches produced very minute gains in performance. Taking linear scores of models is effective but still, produces results comparable to the best performing baseline. that is, the PL2 model. Tables 4.2 and 4.4 have very similar results save for PL2+DPH showing slight gains and DPH with QE not performing as well on the query test. But from table 4.5 it can be seen that our experimental QE approach outperformed all the models for the metric NDCG at 30 confirming the findings by [5]. While the performance remained largely comparable to best performing models in table 4.3, the scores for NDCG at 30 remained the best for any other approach taken on the best performing models.

4.4 Experiment 3: Query Expansion Experiments

Expanding in the literal sense on the query expansion approach we try to expand the queries in this section and try to increase the number of terms in the Query Expansion approach from more top-ranking documents to see if the gains can be improved.

4.4.1 Increasing Terms and test results

We followed a similar pipeline of models first, initially ranking documents with DPH, then using Python terriers default model for query expansion(Bo1), and then re-ranking the documents. We increased the number of terms to 15. We then introduced a new pipeline of using the BM25 for initial ranking, then expanding the documents with 10 and 15 terms, and finally re-ranking them with a DPH and saw appreciable gains on the test set.

Train Set Results Docs = 3				
Model	**Terms**	**NDCG**	**NDCG at 30**	**Precision at 10**
DPH QE	10	0.38	0.26	0.32
DPH QE	15	0.38	0.26	0.31
DPH–>QE–>BM25	10	**0.45**	**0.31**	**0.39**
DPH–>QE–>BM25	15	**0.45**	**0.31**	**0.39**

Table 4.6: Train set results on experiments with increasing terms on Query Expansion

4.4.2 Increasing Docs and Test Results

In this exercise, we checked the sensitivity of increasing terms with increasing the number of documents to be considered as well and arrived at new models with the best scores in our discussion so far. One notable addition here was to use our best performing "conventional" model-PL2- as the final re-ranker, the initial ranking is done by a BM25 and then the queries are expanded by BO1 ranker.

Train Set Results Docs = 6				
Model	**Terms**	**NDCG**	**NDCG at 30**	**Precision at 10**
DPH–>QE–>BM25	10	0.45	0.32	0.39
DPH–>QE–>BM25	15	0.45	0.32	0.4
DPH–>QE–>BM25	20	**0.45**	**0.31**	**0.41**
PL2–>QE—>BM25	15	**0.45**	**0.32**	**0.4**

Table 4.7: Train set results on experiments with increasing documents on Query Expansion

4.4.3 Results: Train Set and Final Query Expansion Models

For our final set we ran the experiments on the test set and corroborated if their success translated well on the test set.

Test Set Results Docs = 6					
Model	**Terms**	**Docs**	**NDCG**	**NDCG at 30**	**Precision at 10**
DPH–>QE–>BM25	10	3	0.54	0.4	**0.49**
DPH–>QE–>BM25	15	6	0.55	0.41	0.48
DPH–>QE–>BM25	20	6	0.55	0.41	0.48
PL2–>QE—>BM25	15	6	**0.56**	**0.41**	**0.48**

Table 4.8: Test-Set results on final Query Expansion Models

4.4.4 Discussion

Through our experiments, we found Query Expansion to be quite effective and already achieved gains of close to 7.7% on the baselines mentioned in Jones[61]. We will extend this approach further later in this chapter when we look at Named-Entity Recognition much like [81]s submission to [61] and reproduce the results and suggest further improvements through Proactive IR.

4.5 Experiment 4: Learning to Rank - XGB Ranker

For our learning to rank approach, we extend upon the LambdaMart approach to validate if it is an effective way to look at noisy information retrieval. However, the results despite hyper-parameter tuning remained underwhelming and did not show any promise despite trying various other features.

We took BM25, PL2, DPH, and Term Frequency and Features trying to find optimal configurations with optimal hyper-parameter tuning by trying gammas of 0.4,0.6,0.8 and minimum child weights of 0.20, 0.40,0.8 and with different learning rates of 0.1, 0.01, 0.001, and maximum depths of 3,6, and 8. We report the best performing models however in terms of performance they performed rather poorly when juxtaposed to even conventional baselines and Query-Expansion Approaches.

4.5.1 Approach and Results

The first 5 topics were picked as topics of training and the last 3 topics were picked as topics of validation to fit the model and the approach was optimized on the overall ndcg. The final evaluation has been on the test set of topics with results being rather unimpressive.

The values of gamma and minimum child weight were kept constant at 6, and 0.20 respectively.

XGBranker				
Learning rate	**Max depth**	**NDCG**	**NDCG at 30**	**Precision at 10**
0.1	6	0.32	0.08	0.09
0.01	6	0.31	0.06	0.09
0.001	6	0.31	0.06	0.09
0.001	3	0.3	0.05	0.07
0.001	**8**	**0.32**	**0.08**	**0.1**

Table 4.9: XGB Ranker Final Model Configurations

4.5.2 Ablation Study

We tried a further ablation and see if some features were more important than the others. The evaluation has been done on the testing topics with the final configuration taken from table 4.9 highlighted in blue.

XGBranker Ablation			
Features	**NDCG**	**NDCG at 30**	**Precision at 10**
BM25, TF, PL2, DPH	0.32	0.08	0.1
BM25, PL2, DPH	0.3	0.06	0.08
BM25, TF, PL2	0.29	0.06	0.09
BM25, TF, DPH	0.32	0.06	0.09
BM25, TF	0.31	0.05	0.05

Table 4.10: Ablation on the previous best performing configuration of LTR approach

4.5.3 Discussion

As can be seen from both the tables 4.9, 4.10 the results were rather demotivating to proceed further in extending upon this approach and rather proved insightful in terms of understanding any LTR approaches were useful in developing an IR system for noisy retrieval. The ablation proved that using all the features was only minutely more useful than using BM25, TF, and DPH.

4.6 Experiment 5: Learning to Rank-FastRank (Coordinate Ascent)

For our coordinate ascent approach, we look at the implementation available in Python through the package called *FastRank*. We pick the coordinate ascent method to corroborate if [22]s findings apply to our noisy dataset in the context of noisy IR.

We tried a small ablation study with the same features in table 4.10 to see if the linear weighting combinations had any different insights to offer. Albeit much like XGBRanker, the results did not match the performance of the conventional models we looked at in the previous sections.

4.6.1 Approach

The implementation provided by *FastRank* is rather straightforward and for parameters, we used the recommended settings on setting the fields to random values with an arbitrary seed value of *1234567* for reproducibility.

For training, we use the entire test set here and use the fitted model on the entire test set and evaluate which feature combination is more/less effective.

4.6.2 Results : Ablation Study on Test Set

The results in table 4.11 when juxtaposed to 4.10 look much better but are still largely underwhelming when compared to the "conventional" models in our previous sections.

Coordinate Ascent Ablation			
Features	**NDCG**	**NDCG at 30**	**Precision at 10**
BM25, TF, PL2, DPH	**0.41**	**0.17**	**0.2**
BM25, PL2, DPH	0.4	0.15	0.18
BM25, TF, PL2	**0.41**	**0.17**	**0.2**
BM25, TF, DPH	0.39	0.13	0.13
BM25, TF	0.32	0.06	0.08

Table 4.11: Coordinate Ascent ablation study carried out using \emph{FastRank}

4.6.3 Discussion

The models with all the features, that is BM25, TF, PL2, and DPH performed similarly to the one with only BM25, TF, and PL2. Quite understandably we echo the same conclusion as from the previous section that the LTR approach has somewhat proved to be ineffective for noisy IR on our dataset with the features we have used. However, we were able to corroborate [22]s findings about coordinate ascent to be more effective even in our use case. It would be interesting to see if LTR approaches behave differently on large datasets like ours but with no noise and study also if adding different features other than scores, such as *word embeddings* would make them more useful on a dataset like ours. Verily, such a study would require much more computational power and is beyond the scope of our discussion.

4.7 Experiment 6: Neural Re-ranking

For our Neural-Ranking approach, we use a pre-trained sequence-to-sequence Transformer approach using a T5, called the *Expando Mono-Duo T5* approach. We explained the details about this approach in section 3.3. Since pre-training the dataset, is more computationally expensive, we use a T5 pre-trained on the MS MARCO Dataset[84].

With more computational resources that could run a GPU comparable to a V100 for more than 12 hours and more ram we could have tried a pre-training approach but did not do so only because of computational limitations.

4.7.1 Approach

The default approach here was to first rank the documents using a BM25 and then use the mono pipeline to rank the documents. The mono-pipeline had a cut-off of top 50 documents to be re-ranked by the duo-pipeline. We tried two different cut-offs for the top documents in the mono-pipeline, 50 and 100. We also tried different approaches with the first ranker by using different best-performing models from the previous sections such as the PL2 and the Query Expansion pipeline involving BM25 as the first ranker, followed by the Bo1 expansion, and re-ranking by a PL2 in the end.

4.7.2 Train results

The results of neural re-ranking on the training set have been shown below in table 4.12.

Baseline	**Top Docs Mono Cut-Off**	**NDCG**	**NDCG at 30**	**Precision at 10**
BM25	50	0.22	0.18	0.19
BM25	100	0.22	0.12	0.08
PL2	50	0.19	0.16	0.16
QE	50	**0.24**	**0.2**	**0.19**

Table 4.12: Expando Mono-Duo approaches on the training set

4.7.3 Test results

For the test set, we only chose the best performing models from table 4.12 and performed re-ranking with the same cut-offs and compared the results.

Expando Mono Duo Test Set				
Baseline	**Top Docs**	**NDCG**	**NDCG at 30**	**Precision at 10**
BM25	50	0.32	0.27	0.3
QE	50	**0.32**	**0.28**	**0.33**

Table 4.13: Expando Mono-Duo on the test set

4.7.4 Discussion

The Neural re-ranking approach is computationally more expensive than other approaches albeit still less expensive than a full-scale approach where the sequence-to-sequence transformer is pre-trained on the dataset. Although the process was computationally more expensive it still led to underwhelming results despite the requirement for more computational power and especially dedicated use of GPU resources. Much like LTR approaches using XGBRanker and Coordinate Ascent, Neural re-ranking did not prove to be as effective as "conventional" models. But given our choice of the dataset, with a large diversity of topics and noise, it is not surprising that the results lead to better performance in conventional models.

4.8 Experiment 7: Named Entity Recognition and Part-of-Speech Tagging

Named Entity Recognition (NER) refers to tasks in Natural Language Processing (NLP) where important entities such as important people, places, objects, years, etcetera are identified from the document or text. NER further falls under the umbrella of Sequence Tagging in NLP. Part-of-speech tagging (POS) is another task that belongs to sequence tagging and is simply put identifying the parts of speech of a particular document or text[98].

Extending further on our approaches of best-performing models such as the PL2, Linear Combination of scores, and Query Expansion, while also using the same model combinations such as [81], our aim with this experiment was to produce state-of-the-art results published in [61]. Therefore, the motivation for NER and POS-tagging-related experiments can be found in the aim to re-produce results from approaches that were possible to replicate with our current available hardware.

4.8.1 Approach

We start our first experiment by using POS tagging on the description of the training topics, we extract nouns from the corresponding description of a particular topic, then expand the topics using the nouns extracted and finally run IR using the modified training topics. We later evaluate our model on the relevance judgments for the training set.

For our second experiment we replace POS-tagging with entities while keeping the rest of the architecture the same, that is, we expand the topics using entities from the corresponding descriptions of the topics of the training set.

In our final experiment, we extract both nouns and entities and expand the queries using both and run the IR task on the training set and evaluate it.

Later, we evaluate all three approaches on the test set. Our modelling choices remain the best-performing models we have explored throughout our discussion in this chapter. We compare all our modelling choices to [81]s to a linear combination of the scores from BM25 and DPH.

For both POS-tagging and NER, we use SpaCy[54], convert text into tokens with SpaCys in-built tokenize and then extract nouns or entities or both as per the requirement of our experiment.

4.8.2 Nouns: Results

The choice of our models is evident from table 4.14 and comprises of only our best-performing approaches. The train-set results seemed largely close with [81]s approach performing the best on just the extraction of nouns

Nouns Train Set			
Features	**NDCG**	**NDCG at 30**	**Precision at 10**
PL2	0.42	0.26	0.28
DCU	**0.44**	**0.27**	**0.34**
QE BM25–>QE–>PL2	0.42	0.27	0.3
BM25 + PL2	0.43	0.27	0.31
PL2 + DPH	0.42	0.26	0.28

Table 4.14: POS-Tagging and extraction of Nouns on the train-set

The test-set however, flipped the scales in favour of our best performing Query-Expansion approach explained in section 4.4. The model outperformed [81]s approach for all the three evaluation metrics as can be seen from table 4.14.

Nouns Test Set			
Features	**NDCG**	**NDCG at 30**	**Precision at 10**
PL2	0.49	0.32	0.39
DCU	0.53	0.36	0.44
QE BM25–>QE–>PL2	**0.55**	**0.38**	**0.44**
BM25 + PL2	0.53	0.35	0.43
PL2 + DPH	0.5	0.32	0.39

Table 4.15: POS-Tagging and extraction of Nouns on the test-set

4.8.3 Entities: Results

With our entities approach we were able to reproduce the best performing results from [61] while also improving upon them through own Query-Expansion approach using BM25–¿QE–¿PL2. However, there were better performing approaches in [61] but they required access to significantly more hardware and thus are beyond the scope of our discussion.

What must be noted however is that [81]s approach seemed comparable to one of our approaches in the training set, which also combined a BM25 and a PL2 model. Our Query Expansion however fared slightly better in early precision with a score of 0.29 as juxtaposed to 0.26 from the best performing approaches on the overall NDCG. It can be argued that with the rank-cut off at 10 included, the Query-Expansion approach fared slightly better than the other two approaches.

Entities Train Set			
Features	**NDCG**	**NDCG at 30**	**Precision at 10**
PL2	0.38	0.24	0.25
QE BM25–>QE–>PL2	0.39	0.25	0.29
DCU	**0.42**	**0.25**	**0.26**
BM25 + PL2	**0.42**	**0.25**	**0.26**
PL2 + DPH	0.4	0.23	0.25

Table 4.16: NER and extraction of Entities on the train-set

The training set however, had a clear winner in our Query Expansion approach, having the best scores in all the three evaluation metrics.

Entities Test Set			
Features	**NDCG**	**NDCG at 30**	**Precision at 10**
PL2	0.57	0.41	0.49
DCU	0.57	0.4	0.48
QE BM25–>QE–>PL2	**0.58**	**0.42**	**0.51**
BM25 + PL2	0.57	0.41	0.5
PL2 + DPH	0.57	0.4	0.48

Table 4.17: NER and extraction of Entities on the test-set

4.8.4 Nouns + Entities: Results

The nouns+entities approach continued the pattern of Dublin City Universitys approach proving to be slightly better on the training set but was second to our approach of Query-Expansion approach on the training set.

Nouns + Entities Train Set			
Features	**NDCG**	**NDCG at 30**	**Precision at 10**
PL2	0.43	0.26	0.28
QE BM25–>QE–>PL2	0.43	0.28	0.35
DCU	**0.45**	**0.29**	**0.36**
BM25 + PL2	0.43	0.28	0.32
PL2 + DPH	0.43	0.26	0.29

Table 4.18: NER + POS tagging and extraction of Entities+ Nouns both on the train-set

Overall after comparing tables 4.14, 4.16, and 4.18, it is evident that the entity recognition approach with our QE pipeline is the most effective.

Nouns + Entities Test Set			
Features	**NDCG**	**NDCG at 30**	**Precision at 10**
PL2	0.49	0.32	0.39
DCU	0.54	0.36	0.45
QE BM25–>QE–>PL2	**0.55**	**0.38**	**0.45**
BM25 + PL2	0.53	0.36	0.44
PL2 + DPH	0.5	0.33	0.4

Table 4.19: NER + POS tagging and extraction of Entities+ Nouns both on the test-set

4.8.5 Discussion

Overall, NER and POS Tagging led to interesting insights about retrieval effectiveness improvement by expanding queries. Verily, adding terms from top-ranking documents, by using a combination of a first-effective ranker and then, another ranker such as a PL2 on the expanded queries proved to be even more effective.

Without NER the same Query Expansion proved to be very effective (table 4.8 but extraction and addition of Queries certainly proved to be more effective and led to a gain of 3.5% in the overall ndcg and 6% in early precision (table 4.17 with NDCG at rank 30 being the same for both the approaches.

4.9 Experiment 8 : Proactive Information Retrieval

Our final set of experiments involved looking at building a proactive information retrieval based on the ideas we have explored in section 2.4.5. The motivation to pursue concept-based modelling comes from one simple observation from the last section, the noun, entities, and the addition of both noun and entities works differently in the training and testing set. We need an approach that works similar to how modern proactive information retrieval systems work without looking at the large availability of pre-existing user history. The topic descriptions verily can be treated like a brief user history, but what is the reliability that one approach of looking at nouns, entities, or the addition of both will work better in the real world? Of course, the scenario invites a discussion of its own, but we use the inquisitive motivation here to propose a new kind of search engine for noisy segment retrieval through concept-based modelling.

We hypothesize that such a library of "concepts" that could capture relevant signals in both the segments and topics and hence increase the overlap could be found in Wikipedia. The increase of "overlap" of meaning, or capturing the "relevant signal" we hypothesize would happen through information need disambiguation which would add more semantic meaning without using a very computationally expensive resource which would require a lot of hardware.

A similar approach has already been explored by [19] and we would first define and test our hypothesis before proceeding with the final experiment of evaluating our hypothesis on an IR system. Our alternative hypothesis is :

$\mathbf{H_{alt}}$: Concept based modelling using Wikipedia concepts will lead to a greater relevance overlap between the segments and the information needs.

With our null hypothesis being:

H_O: Concept-based modelling using Wikipedia concepts will not lead to a greater relevance overlap between the segments and the information needs.

Our approach to testing the above hypothesis is explained in the next section.

4.9.1 Approach: Pre-processing the train Set

To annotate both the segments and queries with Wikipedia concepts, we use the Wikifier API [13] but also come across a challenge, the time taken by the application programming interface or the API, averaged around 6 seconds per response and the dataset contains around 3.6 million segments. Given the sheer size of the dataset, it would be computationally impossible to test our hypothesis on the entire dataset, therefore we down-sample the Spotify podcast dataset into a small dataset consisting of around 15,000 negative segments along with the most relevant segments present for each topic, we label this dataset as the *podcast small* dataset.

To overcome the barrier of time, we decide to split the segments into batches of 1,000 and hit the Wikifier API for annotated responses. In all, it took around **28 hours** of computation time to get all the segments annotated. For the topics, we add both queries+descriptions and then annotate them using the Wikifier Service API.

4.9.2 Approach: Structuring output files

The Wikifier API service not only annotates a given text with Wikipedia concepts, but it also provides a relevance score to each of the concepts it gives back as a response, namely Cosine Similarity and Page-rank scores. For future experiments and keeping in mind an exhaustive analysis by ranking concepts in terms of the relevance of these scores, we record both of these scores in the response dictionary for both, the topics and the segments.

A snippet of annotated concepts for topic one has been shown below in figure 4.1.

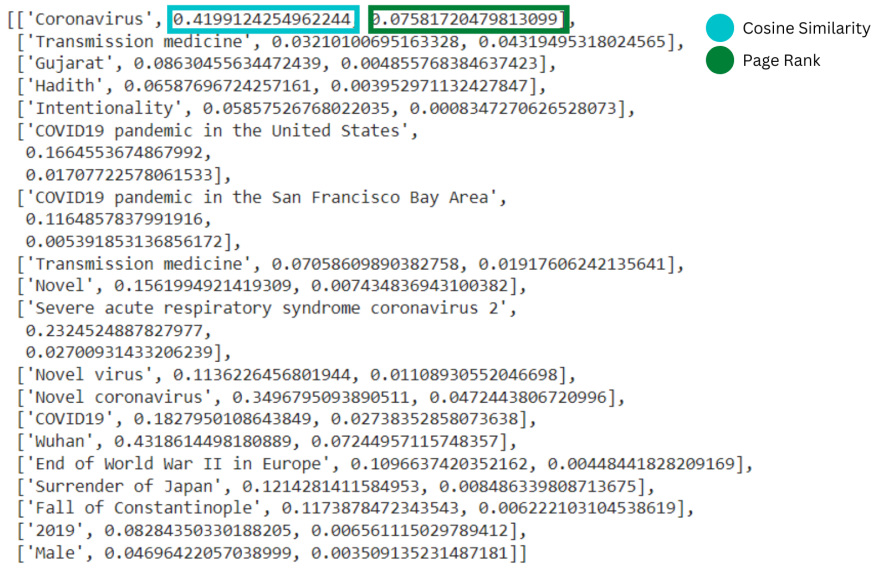


Figure 4.1: A snippet of Topic Responses, the key corresponds to Topic Number or ID, with each concept being a list of its own, with concept at 0 index, Cosine Similarity score at the 1st index, and Page-rank score at the 2nd index

A similar structure was followed for the output files of segments the only difference being topic id replaced by document id.

4.9.3 Approach: Hypothesis Testing

To answer our hypothesis, we calculate the Jaccard similarity coefficient ρ between all the segments and the 8 training topics. The Jaccard similarity coefficient is given by :

$$\rho(t, S) = \frac{t \cap S}{t \cup S} \tag{4.1}$$

Where,

- t denotes the topic, and
- S S denotes the segment

We propose the two sets of $\rho_{relevant}$ and $\rho_{non-relevant}$ will have different medians such that $\rho_{relevant} > \rho_{non-relevant}$ which we test using the one-tailed Mann-Whitney U test. We also plot the results visually through box plots in the next sub-section.

It must be noted that multiple approaches to calculating the Jaccard Similarity coefficient were tried to see if the output scores of Cosine Similarity and Page-rank were more significant in emphasizing whether a concept carried a better signal or not. The first approach involved treating all the annotated concepts as important, the second involved choosing cut-offs with top 1, 2, 3, 5, 10, and 20 concepts according to cosine-similarity scores and page-rank scores. The overall approach here remained the same with making sets of $\rho_{relevant}$ and $\rho_{non-relevant}$. Further

analysis of the relevance of the response scores in annotated topics/segments invites a discussion of its own and it would be interesting to see how different datasets would behave differently in this regard but this is beyond the scope of our discussion.

Therefore, we choose to present only the most significant findings agnostic of the Cosine similarity score and page-rank score, that is, the overall set of concepts. We find statistically significant differences in $\rho_{relevant}$ and $\rho_{non-relevant}$ which we report in the next subsection.

4.9.4 Results: Hypothesis Testing

The results of the one tailed Mann-Whitney U-test are presented below in table 4.20. The topic-wise results for the training set reveal that we reject the null hypothesis, $\mathbf{H_O}$, for each topic.

Topic	Relevant docs	Non Relevant docs	Median relevant	Median non elevant	Mann Whitney U pvalue	Mann Whitney U Statistic
1	70	14109	0.027	0.01	1.03E-49	25680
2	63	14116	0.023	0.01	8.84E-46	47466
3	78	14101	0.043	0.02	1.22E-16	264688
4	78	14101	0.034	0.03	1.41E-06	393032.5
5	80	14099	0.039	0.03	1.42E-03	455014.5
6	37	14142	0.027	0.01	8.37E-48	35747.5
7	77	14102	0.016	0.01	2.80E-44	56321.5
8	80	14099	0.032	0.02	6.31E-16	271960.5

Table 4.20: Results of one-tailed Mann-Whitney U test

We present a box-plot of the results below comparing ρ for relevant and non-relevant sets below in figure :

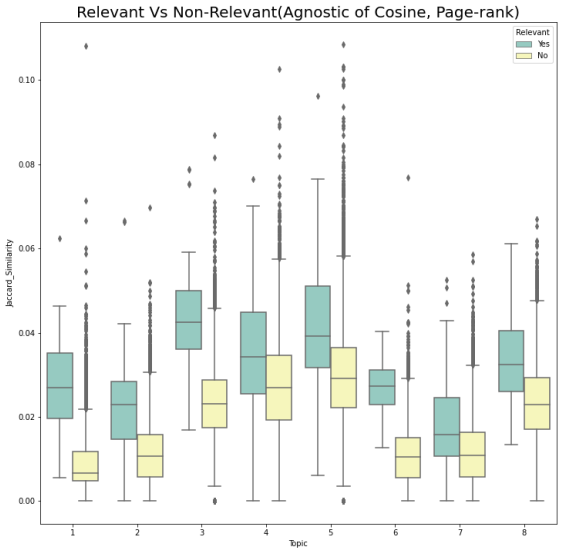


Figure 4.2: Topic wise comparison of ρ between relevant and non-relevant sets of topics in the training set

More plots related to distribution with respect to ranking with Cosine similarity scores and

Page rank scores can be found in appendix section (B. We further compare our approach to the best performing approaches from the previous sections on the *Podcast Small Dataset*.

4.9.5 Approach: Information Retrieval on *Podcast Small*

To design IR on the podcast small dataset we appended the entire set of concepts to each corresponding topic and the segment albeit adding specified formatting to it so that it doesnt coincide with the pre-existing contents of the document.

For topics we do this in two ways i) we simply appended the concepts to the topics ii) we appended the concepts to topics+entities in description + concepts. The reasoning for ii) being, the success of NER approach from the previous set of experiments.

In the case of the segments, we simply appended the concepts in the specified formatting to each segment. The specified formatting can be understood from the example shown below for training topics in table 4.21.

Topic	**Wikified Topics (Shortened)**
coronavirus spread	coronavirus spread wiki_COVID19_pandemic_in_the_United_States wiki_Coronavirus wiki_Novel ...
greta thunberg cross atlantic	greta thunberg cross +... atlantic wiki_Climate_change wiki_Greta_Thunberg....

Table 4.21: Example of Wikification format for topics 1 and 2

The specified format to add Wikipedia concepts for wikification was to convert concepts into concept chunks as shown in table 4.21, such as, "Covid19 pandemic in the United States" would have the pre-fix "wiki" before it with every blank space in the concept replaced with an underscore "_", therefore transforming the concept to **"wiki_COVID19_pandemic_in_the_United_States"**. Every concept corresponding to a certain topic or a segment was appended to the topic/segment followed by a single space.

After the creation of documents, the *Podcast Small* dataset was then formatted in the Trec-XML format and indexed using Python-Terriers indexer.

4.9.6 Results: Information Retrieval on *Podcast Small*

We tried several approaches and the results from this section were rather surprising, even surpassing our best-performing models from the previous sections. Preliminary findings from our approach to Proactive Information Retrieval using Wikipedia Concepts led us to submit our results for the PASIR Workshop 2022 (Proactive and Agent-Supported Information Retrieval). A copy of our submission can be found in the appendix refPASIR. However, we managed to improve on the results presented in the paper the results of which can be found below in table 4.22.

Model	Query Type	Index Collection	NDCG	NDCG at 30	Precision at 10
BM25+DPH	Original Topics	Segments (No change)	0.48	0.3	0.31
BM25+DPH	Wikified Query	Segments (No change)	0.42	0.18	0.15
BM25 +DPH	Original Topics	Wikified Segments	0.51	**0.34**	**0.41**
BM25+DPH	Wikified Query	Wikfied Segments	0.5	0.3	0.31
BM25+DPH	NER ENT QE	Wikfied Segments	**0.55**	**0.39**	**0.4**
BM25+DPH	Wikified Query +ENT	Wikified Segments	**0.51**	**0.3**	**0.36**
BM25+DPH	NER ENT QE	Wikfied Segments freq 2	**0.57**	**0.4**	**0.39**
BM25+DPH	NER+ POS Tagging	Wikified Segments	**0.56**	**0.42**	**0.42**
BM25–>QE–>PL2	NER ENT QE	Segments (No change)	0.47	0.27	0.26
BM25–>QE–>PL2	Wikified Query	Segments (No change)	0.34	0.13	0.1
BM25–>QE–>PL2	Original Topics	Segments (No change)	0.47	0.27	0.31
BM25–>QE–>PL2	NER ENT QE	Wikfied Segments	0.53	0.36	0.38
BM25–>QE–>PL2	Original Topics	Wikfied Segments	0.5	0.31	0.38
BM25–>QE–>PL2	POS Tagging(Nouns)	Wikfied Segments	**0.56**	**0.39**	0.37
BM25–>QE–>PL2	NER+ POS Tagging	Wikified Segments	**0.57**	**0.41**	**0.4**
DCU (BM25 + DPH + NER)		Segments (No change)	0.51	0.32	0.3
PL2 (tuned)	NER ENT QE	Segments (No change)	0.51	0.32	0.29
PL2 (tuned)	NER ENT QE	Wikfied Segments	**0.55**	0.38	**0.4**
PL2 (tuned)	Wikified Query+ENT	Wikfied Segments	0.4	0.22	0.19
PL2 (tuned)	Original Topics	Wikfied Segments	0.48	0.3	0.3

Table 4.22: Proactive Information Retrieval Experiments. Our initially proposed model for Proactive Information Retrieval can be seen highlighted in yellow, while our new approaches can be seen highlighted with blue. The approach by DCU is highlighted with light green

4.9.7 Discussion: IR results on *Podcast Small*

Our results proved to be rather promising. We report 21 Models here, but trying different combinations revealed two important insights i) Our "conventional" architectures did not tolerate the wikified queries well perhaps this was due to the increased query length ii) Our wikified documents with concept enrichment significantly benefited from the process and led to state-of-the-art results.

Overall, from the model proposed by DCU[81], we achieved around 12% gains in overall NDCG, with much more significant gains in early-retrieval which is of special interest to us given our use-case scenario is to optimize the results for early retrieval for a search engine. Our NER + POS tagging approach using a BM25+DPH on the wikified documents achieved 28% improvement on NDCG at 30 over the DCU model, and 30% improvement on precision at rank 10 over the DCU model.

Even using non-enriched queries on the linear combination of BM25+DPH showed gains of 32% in Precision at rank 10 when enriched documents were used as juxtaposed to the same approach on non-enriched documents in row 1 of the table. Even our best approach on the train set using Query Expansion from section 4.4 was outperformed by the NER+POS Tagging approach for topics on the wikified documents by 32% on precision at 10.

The gains when using the Wikified segment collection remained consistent with all modelling approaches. The success prompted us to enrich the dataset further and try a larger experiment.

4.9.8 From *Podcast Small* to *Podcast Small Extended (SE)*

The promising results on *Podcast Small* using wikified topics and segments prompted us to create an extended dataset using the Wikifier API service. We increased the sub-sample close to 10 times by increasing the dataset size to 150,000 total segments while sampling segments relevant to the test topics and negative segments picked at random.

Since the response time per hit for every segment averaged at around 6 seconds, the task seemed rather onerous and required around **250 hours of computing time**. Since we had limitations for ram but had the maximum process time of 24 hours on Googles Colab pro+ we split all the segments into batches of 1,000 and then saved them discretely as 150 batches to create the *Podcast Small Extended* or *Podcast SE* dataset.

The overall idea was to evaluate the retrieval performance on the test topics and provide motivation for further discussion on Proactive-Information Retrieval. A topic-wise summary of test topics for the ***Podcast SE*** Dataset has been shown in table 4.23 below.

Topic	**Relevant_docs**	**Non_Relevant_docs**
9	118	149684
10	135	149667
11	185	149617
12	153	149649
13	191	149611
14	144	149658
15	294	149508
16	166	149636
17	253	149549
18	171	149631
19	174	149628
20	202	149600
21	144	149658
22	177	149625
23	192	149610
24	147	149655
25	166	149636
26	136	149666
27	181	149621
28	160	149642
29	151	149651
30	194	149608
31	227	149575
32	240	149562
33	218	149584
34	249	149553
35	173	149629

Table 4.23 continued from previous page

Topic	Relevant_docs	Non_Relevant_docs
36	208	149594
37	200	149602
38	139	149663
39	185	149617
40	195	149607
41	161	149641
42	197	149605
43	140	149662
44	199	149603
45	178	149624
46	123	149679
47	0	149802
48	160	149642
49	172	149630
50	0	149802
51	183	149619
52	280	149522
53	248	149554
54	149	149653
55	168	149634
56	173	149629
57	167	149802
58	204	149598

Table 4.23: Topic-wise relevant/non-relevant summary of *Podcast SE* dataset

This missing values for topics 47 and 50 arise due to inconsistencies in a long **250 Hour** compute time for API responses from Wikifier API service since it doesnt support bulk API calls. However, since our idea is to provide a foundation for an extended discussion in the future, we choose to exclude those in our discussion and evaluate our dataset on all the other topics which still have relevant segments available in our dataset.

We proceed to evaluate IR using the same models as *Podcast Small* on test topics in the next sub-section.

4.9.9 Results : IR Results on Podcast SE

The IR results on *Podcast SE* were interesting and led us to interesting questions which can be asked in a further full scale study of Wikified segments.

Model	Query Type	Index Collection	NDCG	NDCG at 30	Precision at 10
BM25+DPH	Original Topics	Segments (No change)	0.57	0.39	0.47
BM25+DPH	Wikified Query	Segments (No change)	0.49	0.25	0.27
BM25 +DPH	Original Topics	Wikified Segments	0.57	0.38	**0.48**
BM25+DPH	Wikified Query	Wikfied Segments	0.56	0.37	0.43
BM25+DPH	NER ENT QE	Wikfied Segments	**0.59**	**0.41**	**0.5**
BM25+DPH	Wikified Query +ENT	Wikified Segments	0.56	0.38	0.44
BM25+DPH	NER ENT QE	Wikfied Segments freq 2	0.57	0.39	0.47
BM25+DPH	NER+ POS Tagging	Wikified Segments	0.55	0.35	0.4
BM25–>QE–>PL2	NER ENT QE	Segments (No change)	**0.6**	**0.43**	**0.52**
BM25–>QE–>PL2	Wikified Query	Segments (No change)	0.49	0.25	0.29
BM25–>QE–>PL2	Original Topics	Segments (No change)	**0.58**	**0.4**	**0.48**
BM25–>QE–>PL2	NER ENT QE	Wikfied Segments	**0.59**	**0.42**	**0.53**
BM25–>QE–>PL2	Original Topics	Wikfied Segments	**0.58**	0.4	0.47
BM25–>QE–>PL2	POS Tagging(Nouns)	Wikfied Segments	0.55	0.36	0.42
BM25–>QE–>PL2	NER+ POS Tagging	Wikified Segments	0.55	0.34	0.41
DCU (BM25 + DPH + NER)		Segments (No change)	**0.59**	**0.42**	**0.5**
PL2 (tuned)	NER ENT QE	Segments (No change)	**0.59**	**0.42**	**0.5**
PL2 (tuned)	NER ENT QE	Wikfied Segments	**0.6**	**0.42**	**0.5**
PL2 (tuned)	Wikified Query+ENT	Wikfied Segments	0.49	0.25	0.23
PL2 (tuned)	Original Topics	Wikfied Segments	**0.58**	**0.4**	**0.48**
BM25–>QE–>PL2 + PL2	NER ENT QE	Segments (No Change) + Wikified	**0.61**	**0.44**	**0.51**

Table 4.24: IR on Podcast SE on the test-set of topics. The highlighted models from Podcast Small Retain their highlight colours for easy readability. Well-performing approaches on the test-set are highlighted with peach, and the best-performing approach is highlighted with dark green in the last row.

4.9.10 Discussion : Results *Podcast SE*

The results from the train set topics on *Podcast Small* showed early precision gains to the tune of 32% by just using enriched segments in the collection (table 4.22). Similar approaches, however, performed very differently on the test-set of topics. While topics 47 and 50 were excluded from the IR task, we evaluated on rest of the topics and found that on *Podcast SE*, the approach by DCU still proved to be one of the best performing approaches. The indexed collection with enriched segments containing Wikipedia concepts did not offer gains to the tune of what they offered on the training set on *Podcast Small*. However, the best early precision of 0.53 was still achieved by our Query expansion approach on Wikified segments with the NER approach on the queries. The approach still offered 6% of gains in precision at 10 on the approach by DCU[81] with other metrics being similar. The same approach on the raw segments also showed gains over the DCU approach, prompting us to ask an interesting question, *would a linear combination of scores of the same documents with enrichment and no-enrichment as indexed collections, lead to even more gains?*

To test this hypothesis further, we combined our query expansion approach which ran on the segment collection with no-enrichment linearly with our PL2 model on wikified segment collection and saw the best NDCG at rank 30 score, the best NDCG score among all the models. The approach has been highlighted in dark green in the last row of table 4.24.

Overall, our Wikipedia enrichment of documents still ran neck-to-neck and in some cases provided gains over all the other best-performing approaches we have seen throughout our discussion, encouraging us to ask important questions for further study.

Chapter 5

Analysis

Throughout our experiments in the last chapter, our overall approach involved navigating around all types of IR approaches to seek a direction that would allow us to improve on state-of-the-art results. We started our discussion by running "conventional" baselines, then proceeded in the direction of a linear combination of scores. We then proceeded to experiment with Query Expansion and tried various approaches with Query Expansion before proceeding with LTR approaches, Neural Re-ranking, and producing state-of-the-art results with NLP approaches. Our final section looked at Proactive-information retrieval and suggested a novel approach to solve noisy information retrieval albeit through two sufficiently down-sampled datasets which we created by using the Wikifier[13] API service-*Podcast Small*(28 hours of computing time) and *Podcast SE*(250 hours of computing time).

But one thing that remained consistent throughout our discussion was the robust performance of "conventional" approaches as juxtaposed to LTR or neural re-ranking approaches. Our best performing models from Experiments 7 and 8 provide appreciable gains on all neural approaches in [61] save for the Combine and re-rank approach by [43] which again uses a combination of neural re-ranking approaches, word-embeddings, and sequential-dependence model. Albeit one important point to note about [43] approaches is that it directly appends the topic with its description and then uses a weighted combination of the new topic as the final topic. Whereas, throughout our discussion, we tried to restrict ourselves to query-enrichment methods which were closer to real-world scenarios.

By real-world scenarios, we imply the foundations we looked at for Proactive Information retrieval in Chapter 3. We treat descriptions as user history, especially in the use case of concept-based modelling. For NLP-related information extraction from descriptions, we seek inspiration from [81], with the underlying assumption the same as proactive information retrieval, that is, we treat the descriptions as user history. But an important question despite should be why are "conventional" approaches more successful than any other approach?

A key explanation of this would be how Bayesian/frequentist approaches for IR work in general [5]. The idea is to not look at the meaning of the terms, or look at lexical similarity, but rather only look at IR as a problem of probability. An intuitively logical answer to why neural approaches didnt produce results as good as the probabilistic ones is tied to the characteristics of our dataset in terms of being noisy and a high word-error rate of around 18% along with other issues like some

of the podcasts didnt have the English language in general (see section 1.4), this could particularly be an impediment to understanding context using embedding-based approach using Transformers. When treating IR as a problem of probability, the problems related to word-error rate, or context do not apply in a strict linguistic sense in terms of context or meaning. The emphasis, in a clearer way, is to find the probability of terms appearing, with different levels of sophistication and the underlying approach of the probabilistic model.

The LTR approaches too, fail to learn anything from the features despite hyperparameter tuning and that is why they do not find a mention in any of the better-performing approaches in [61]. An explanation of this could be that the training data in our case proved to be insufficient to learn anything meaningful, or more features such as embeddings, could have led to better results, a question that needs to be explored and has partly been explored (albeit not through LTR) through an embedding-based approach taken by [43] which produces promising results although uses the combination of the topic + entire description to achieve so.

Our novel approach to capturing relevant signals using Wikipedia concepts by enriching both the segments and topics with them, showed immense promise on the *Podcast Small* dataset while still being part of our best-performing dataset on the *Podcast SE* dataset. The reason for this we argue is how concept-based modelling solves for disambiguation in both the topics and segments. While we showed the overlap and failed to reject our null hypothesis in section 4.9.4 using the Jaccard similarity coefficient ρ, the results of IR proved that using topics with NER and POS Tagging approaches was overall a more effective approach. An argument for this could be the sensitivity of our proposed models to different topic lengths, a discussion that invites a more granular study in understanding how to associate Wikipedia concepts more efficiently to the topics or even in terms of enriching the documents. Both the datasets annotated by us are created from an intention for a further study of sensitivity to appending top 1,2,3,5, and 10 concepts to both segments and the topics. While explored Jaccardian similarity with different sensitivities in both segments and topics, that is, between top 1 concepts of topics and top 1 concepts of segments, between top 3 concepts of topics and top 3 concepts of segments, according to the Cosine similarity and page-rank scores, we did not explore the sensitivity of top 1,2,3,5,10 concepts in topics to all the concepts in the segments or top 1,2,3,4,5,10 concepts in the segments. A sensitivity analysis in this regard we believe could lead to more efficient gains along with the combination of NLP approaches such as NER or POS-tagging for the extraction of information.

The success of using NER or POS-tagging approaches on the wikified segments does highlight an important fact that appending Wikipedia concepts to the segments, strengthens the "context" of the documents which is again further strengthened in better matches by using NER/POS-tagging approaches on the topics, which lead to topic disambiguation by addition of entities/nouns. Verily, this is description-dependent, but a proactive information retrieval system could have sufficient user history to extract such entities from a user.

For a final comparison of all our approaches to the ones present in [61], we present all the best performing approaches in table 5.1 below.

Model	Dataset	Topics	Trec Paper	Our Approach	Trec Paper	Our Approach	Trec Paper	Our Approach
DCU (BM25 +DPH + NER)	Spotify Podcast Dataset	Test	**0.57**	0.57	0.42	0.4	0.5	0.5
BM25–>QE–>PL2	Spotify Podcast Dataset	Test	-	**0.58**	-	**0.42**	-	**0.51**
Wikfied Docs (BM25–>QE–>PL2 + NER +POS tagging)	Podcast Small	Train	-	**0.57**	-	**0.41**	-	**0.4**
DCU (BM25 +DPH + NER)	Podcast Small	Train	-	0.54	-	0.32	-	0.3
DCU (BM25 +DPH + NER)	Podcast SE	Test	-	**0.59**	-	**0.42**	-	**0.5**
BM25–>QE–>PL2 (Docs)+ PL2(Wikified) + NER	Podcast SE	Test	-	**0.61**	-	**0.44**	-	**0.51**

Table 5.1: Final comparison of results, all best performing approaches highlighted in green, with the column in Blue providing results available in [61]

5.0.1 Effectiveness of NLP approaches on Topics

A larger highlight of all the results from the last chapter has been the substantial difference in the enrichment of topics made to the effectiveness of IR on the entire dataset. While enrichment of segments provided enormous gains up to 32% in early precision on *Podcast Small*, it must be noted that the gains were also due to enrichment of topics with entities, nouns, and a combination of both nouns + entities. Our hypothesis from section 4.9.4, was that adding relevant signals using concept modelling increases the overlap between topics and their relevant segments, which we calculated using the Jaccard similarity coefficient, ρ.

Regarding why NLP approaches and concept-based modelling approaches work so effectively, we argue that the enrichment through any of these techniques overall leads to the disambiguation of meaning, through the addition of relevant concepts. Quite remarkably so the approach is the most efficient with nouns and entities both on the train set, and with entities on the test set. We have already discussed how we suspect topic length to be an issue concerning topics appended with Wikipedia concepts and Wikipedia concepts + entities. We present examples of all the approaches on topic 1 from the train in table 5.2.

Topic Type	Topical
Original Topic	coronavirus spread
Description	What were people saying about the spread of the novel coronavirus NCOV-19 in Wuhan at the end of 2019?
Enrichment Examples	
Approach	Modified topic
Nouns	coronavirus wuhan spread people end
Entities	2019 coronavirus spread wuhan
Nouns + Entities	coronavirus wuhan spread people 2019 end
Wikified Topic	original topic + wiki_covid19_pandemic_in_the_united_states wiki_surrender_of_japan wiki_coronavirus wiki_novel +... wiki_severe_acute_respiratory_syndrome_coronavirus_2
Wikified topic + ent	Entities + Wikfied Topic

Table 5.2: NLP and Wikipedia enrichment approaches on Topic 1 from the train set

Chapter 6

Conclusion

When we started our discussion we emphasized how solving noisy IR is pivotal in bridging the 'Gap' between the developed and developing world in the vein of AIEd. Motivated by previous state-of-the-art approaches, we designed our experiments to look at what architectures/approaches can solve noisy IR the best, eventually confirming what was observable in [61]. We further reproduced the best results and improved them by providing a novel approach of using proactive information retrieval on sophisticated probabilistic models with Query Expansion as the best solution capable of providing 30% gains on early precision on the training topics on *Podcast Small* and around 6% gains on the testing topics on *Podcast SE*. We also generated two datasets, *Podcast Small* and Podcast SE using the Wikifier API service to enrich the segments. Both datasets provide options for studying concept-based modelling with different cut-offs on Cosine Similarity and Page-rank scores provided by the Wikifier API service.

We suggest a future discussion on the following themes, looking at :

- Sensitivity to cut-offs of page-rank and cosine similarity scores on topic concepts and if certain cut-offs can provide performance comparable to or better than NER or POS-tagging approaches.
- Exploring a similar sensitivity for segments and understanding what approach works better, that is NER, POS Tagging, or other NLP approaches that can be used for information extraction.
- Novel approaches of using Wikifiedia annotation by using Neural approaches trained on Wikipedia datasets.
- A full-scale study of annotation on the entire Spotify Podcast Dataset
- A comparison of Information retrieval on other noisy datasets along with a linguistic analysis of the characteristics to understand what kind of IR approach works well on datasets with certain characteristics.
- Appending the concepts as plain text and using full-scale end-to-end Neural retrieval using recent approaches such as ColBERT[63].

6.1 Final Proposed Model

Though we have looked at multiple approaches throughout our discussion which can provide gains on state-of-the-art performance. Our final proposed model can defined as follows :

$$Score(q_i, s_j) = QE(q_{text} + q_{entities}, s_{text}) + PL2(q_{text} + q_{entities}, s_{text} + s_{wikified}) \tag{6.1}$$

Where q represents queries or more formally in our task *topics* and s represents segments. Here by equation 6.1 we mean that a model with our Query Expansion which comprises of using BM25 as the first ranker, followed by query expansion using a B01 model (implemented through Python Terrier), and then re-ranking by PL2 run on a non-enriched collection linearly coupled with a baseline such as the PL2 model run on the Wikipedia enriched collection will provide appreciable gains in early information retrieval and a higher overall NDCG.

Bibliography

[1] Relevance weighting of search terms. ourna ote merican ociet or normation cience, 27 (3): 129 {146, author=Robertson, Stephen E and Jones, Sparck, year=1976, publisher=May-June.

[2] Transcribe videos nbsp;—nbsp; cloud speech-to-text documentation nbsp;—nbsp; google cloud.

[3] Tomoyosi Akiba, Hiromitsu Hishizaki, Hiroaki Nanjo, and Gareth JF Jones. Overview of the ntcir-11 spokenquery&doc task. 2014.

[4] Tomoyosi Akiba, Hiromitsu Nishizaki, Kiyoaki Aikawa, Xinhui Hu, Yoshiaki Itoh, Tatsuya Kawahara, Seiichi Nakagawa, Hiroaki Nanjo, and Yoichi Yamashita. Overview of the ntcir-10 spokendoc-2 task. In *NTCIR*. Citeseer, 2013.

[5] Giambattista Amati. Frequentist and bayesian approach to information retrieval. In *European Conference on Information Retrieval*, pages 13–24. Springer, 2006.

[6] Gianni Amati and Cornelis Joost Van Rijsbergen. Probabilistic models of information retrieval based on measuring the divergence from randomness. *ACM Transactions on Information Systems (TOIS)*, 20(4):357–389, 2002.

[7] Nima Asadi and Jimmy Lin. Effectiveness/efficiency tradeoffs for candidate generation in multi-stage retrieval architectures. In *Proceedings of the 36th international ACM SIGIR conference on Research and development in information retrieval*, pages 997–1000, 2013.

[8] Daniel Ewell Atkins, John Seely Brown, and Allen L Hammond. *A review of the open educational resources (OER) movement: Achievements, challenges, and new opportunities*, volume 164. Creative common Mountain View, 2007.

[9] Hiteshwar Kumar Azad and Akshay Deepak. A new approach for query expansion using wikipedia and wordnet. *Information sciences*, 492:147–163, 2019.

[10] Lalit R Bahl, Frederick Jelinek, and Robert L Mercer. A maximum likelihood approach to continuous speech recognition. *IEEE transactions on pattern analysis and machine intelligence*, (2):179–190, 1983.

[11] Graham Bennett, Falk Scholer, and Alexandra Uitdenbogerd. A comparative study of probabilistic and language models for information retrieval. In *Database Technologies 2008: Proceedings of the Nineteenth Australasian Database Conference (ADC 2008)*, pages 65–74. RMIT University, 2008.

[12] Adam Berger and John Lafferty. Information retrieval as statistical translation. In *ACM SIGIR Forum*, volume 51, pages 219–226. ACM New York, NY, USA, 2017.

[13] Janez Brank, Gregor Leban, and Marko Grobelnik. Annotating documents with relevant wikipedia concepts. *Proceedings of SiKDD*, 472, 2017.

[14] Peter F Brown, John Cocke, Stephen A Della Pietra, Vincent J Della Pietra, Frederick Jelinek, John Lafferty, Robert L Mercer, and Paul S Roossin. A statistical approach to machine translation. *Computational linguistics*, 16(2):79–85, 1990.

[15] Helen L Brownson. Research on handling scientific information: Improvements in communication and information handling contribute to scientific progress. *Science*, 132(3444):1922–1931, 1960.

[16] Michael K Buckland. Emanuel goldberg, electronic document retrieval, and vannevar bush's memex. *Journal of the American Society for Information Science*, 43(4):284–294, 1992.

[17] Sahan Bulathwela, María Pérez-Ortiz, Catherine Holloway, and John Shawe-Taylor. Could ai democratise education? socio-technical imaginaries of an edtech revolution. *arXiv preprint arXiv:2112.02034*, 2021.

[18] Sahan Bulathwela, María Pérez-Ortiz, Erik Novak, Emine Yilmaz, and John Shawe-Taylor. Peek: A large dataset of learner engagement with educational videos. *arXiv preprint arXiv:2109.03154*, 2021.

[19] Sahan Bulathwela, Maria Perez-Ortiz, Emine Yilmaz, and John Shawe-Taylor. Towards an integrative educational recommender for lifelong learners (student abstract). In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 13759–13760, 2020.

[20] Chris Burges, Tal Shaked, Erin Renshaw, Ari Lazier, Matt Deeds, Nicole Hamilton, and Greg Hullender. Learning to rank using gradient descent. In *Proceedings of the 22nd international conference on Machine learning*, pages 89–96, 2005.

[21] Christopher JC Burges. From ranknet to lambdarank to lambdamart: An overview. *Learning*, 11(23-581):81, 2010.

[22] Róbert Busa-Fekete, György Szarvas, Tamás Elteto, and Balázs Kégl. An apple-to-apple comparison of learning-to-rank algorithms in terms of normalized discounted cumulative gain. In *ECAI 2012-20th European Conference on Artificial Intelligence: Preference Learning: Problems and Applications in AI Workshop*, volume 242. Ios Press, 2012.

[23] Vannevar Bush et al. As we may think. *The atlantic monthly*, 176(1):101–108, 1945.

[24] Stephen Carson et al. 2004 mit ocw program evaluation findings report. *Massachusetts Institute of Technology*, 2005.

[25] O Chapelle, Y Chang, and TY Liu. The yahoo! learning to rank challenge, 2010.

[26] Tianqi Chen and Carlos Guestrin. XGBoost. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, aug 2016.

[27] Abdur Chowdhury, M Catherine McCabe, David Grossman, and Ophir Frieder. Document normalization revisited. In *Proceedings of the 25th annual international ACM SIGIR conference on Research and development in information retrieval*, pages 381–382, 2002.

[28] Cyril Cleverdon. The cranfield tests on index language devices. In *Aslib proceedings*. MCB UP Ltd, 1967.

[29] Cyril W Cleverdon. The evaluation of systems used in information retrieval. In *Proceedings of the international conference on scientific information*, volume 1, pages 687–698. National Academy of Sciences Washington, DC,, 1959.

[30] Ann Clifton, Aasish Pappu, Sravana Reddy, Yongze Yu, Jussi Karlgren, Ben Carterette, and Rosie Jones. The spotify podcast dataset. *arXiv preprint arXiv:2004.04270*, 2020.

[31] Daniel Cohen, Qingyao Ai, and W Bruce Croft. Adaptability of neural networks on varying granularity ir tasks. *arXiv preprint arXiv:1606.07565*, 2016.

[32] Albert T Corbett and John R Anderson. Knowledge tracing: Modeling the acquisition of procedural knowledge. *User modeling and user-adapted interaction*, 4(4):253–278, 1994.

[33] W Bruce Croft, Donald Metzler, and Trevor Strohman. *Search engines: Information retrieval in practice*, volume 520. Addison-Wesley Reading, 2010.

[34] Scott Deerwester, Susan T Dumais, George W Furnas, Thomas K Landauer, and Richard Harshman. Indexing by latent semantic analysis. *Journal of the American society for information science*, 41(6):391–407, 1990.

[35] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.

[36] Stephen Downes. Models for sustainable open educational resources. *Interdisciplinary Journal of E-Learning and Learning Objects*, 3(1):29–44, 2007.

[37] Maria Eskevich, Robin Aly, Roeland Ordelman, David Nicolas Racca, Shu Chen, and Gareth JF Jones. Sava at mediaeval 2015: Search and anchoring in video archives. In *MediaEval*. Citeseer, 2015.

[38] Maria Eskevich, Robin Aly, David Racca, Roeland Ordelman, Shu Chen, and Gareth JF Jones. The search and hyperlinking task at mediaeval 2014. 2014.

[39] Maria Eskevich, Gareth JF Jones, Robin Aly, Roeland JF Ordelman, Shu Chen, Danish Nadeem, Camille Guinaudeau, Guillaume Gravier, Pascale Sébillot, Tom De Nies, et al. Multimedia information seeking through search and hyperlinking. In *Proceedings of the 3rd ACM conference on International conference on multimedia retrieval*, pages 287–294, 2013.

[40] Charles Fadel, Wayne Holmes, and Maya Bialik. Artificial intelligence in education: Promises and implications for teaching and learning. *The Center for Curriculum Redesign, Boston, MA*, 2019.

[41] Norbert Fuhr. Optimum polynomial retrieval functions based on the probability ranking principle. *ACM Transactions on Information Systems (TOIS)*, 7(3):183–204, 1989.

[42] Norbert Fuhr. Probabilistic models in information retrieval. *The computer journal*, 35(3):243–255, 1992.

[43] Petra Galuščáková, Suraj Nair, and Douglas W Oard. Combine and re-rank: The university of maryland at the trec 2020 podcasts track. 2020.

[44] Jianfeng Gao, Patrick Pantel, Michael Gamon, Xiaodong He, and Li Deng. Modeling interestingness with deep neural networks. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 2–13, 2014.

[45] John S Garofolo, Cedric GP Auzanne, Ellen M Voorhees, et al. The trec spoken document retrieval track: A success story. *NIST SPECIAL PUBLICATION SP*, 500(246):107–130, 2000.

[46] Nazli Goharian. Language model. 2012.

[47] Emanuel Goldberg. A new process of micro-photography. *British Journal of Photography*, 73(3458):462–465, 1926.

[48] V Grech. Publishing on the www. part 5-a brief history of the internet and the world wide web. *Images in Paediatric Cardiology*, 3(3):15, 2001.

[49] Donna Harman. Evaluation issues in information retrieval. *Information Processing and Management*, 28(4):439 – –440, 1992.

[50] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. Boosting and additive trees. In *The elements of statistical learning*, pages 337–387. Springer, 2009.

[51] Ben He and Iadh Ounis. A study of parameter tuning for term frequency normalization. In *Proceedings of the twelfth international conference on Information and knowledge management*, pages 10–16, 2003.

[52] Ben He and Iadh Ounis. Term frequency normalisation tuning for bm25 and dfr models. In *European Conference on Information Retrieval*, pages 200–214. Springer, 2005.

[53] JE Holmstrom. Section iii. opening plenary session. In *The Royal Society Scientific Information Conference*, volume 21, 1948.

[54] Matthew Honnibal, Ines Montani, Sofie Van Landeghem, and Adriane Boyd. spacy: Industrial-strength natural language processing in python. 2020.

[55] Po-Sen Huang, Xiaodong He, Jianfeng Gao, Li Deng, Alex Acero, and Larry Heck. Learning deep structured semantic models for web search using clickthrough data. In *Proceedings of the 22nd ACM international conference on Information & Knowledge Management*, pages 2333–2338, 2013.

[56] Jan Hylén. Open educational resources: Opportunities and challenges. 2021.

[57] Kalervo J"arvelin and Jaana Kek"al"ainen. Cumulated gain-based evaluation of ir techniques. *ACM Trans. Inf. Syst.*, 20(4):422–446, 2002.

[58] Frederick Jelinek. Interpolated estimation of markov source parameters from sparse data. In *Proc. Workshop on Pattern Recognition in Practice, 1980*, 1980.

[59] Gareth JF Jones. About sound and vision: Clef beyond text retrieval tasks. *Information Retrieval Evaluation in a Changing World*, pages 307–329, 2019.

[60] Karen Sparck Jones. Automatic language and information processing: rethinking evaluation. *Natural Language Engineering*, 7(1):29–46, 2001.

[61] Rosie Jones, Ben Carterette, Ann Clifton, Maria Eskevich, Gareth JF Jones, Jussi Karlgren, Aasish Pappu, Sravana Reddy, and Yongze Yu. Trec 2020 podcasts track overview. *arXiv preprint arXiv:2103.15953*, 2021.

[62] Michael Keen. *Measures and averaging methods used in performance testing of indexing systems*. Aslib Cranfield Research Project, 1966.

[63] Omar Khattab and Matei Zaharia. Colbert: Efficient and effective passage search via contextualized late interaction over bert. In *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval*, pages 39–48, 2020.

[64] John Lafferty and Chengxiang Zhai. Document language models, query models, and risk minimization for information retrieval. In *Proceedings of the 24th annual international ACM SIGIR conference on Research and development in information retrieval*, pages 111–119, 2001.

[65] Martha Larson, Maria Eskevich, Roeland Ordelman, Christoph Kofler, Sebastian Schmiedeke, and Gareth JF Jones. Overview of mediaeval 2011 rich speech retrieval task and genre tagging task. CEUR Workshop Proceedings, 2011.

[66] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *nature*, 521(7553):436–444, 2015.

[67] Jimmy Lin, Rodrigo Nogueira, and Andrew Yates. Pretrained transformers for text ranking: Bert and beyond. *Synthesis Lectures on Human Language Technologies*, 14(4):1–325, 2021.

[68] Tie-Yan Liu et al. Learning to rank for information retrieval. *Foundations and Trends® in Information Retrieval*, 3(3):225–331, 2009.

[69] Hans Peter Luhn. A statistical approach to mechanized encoding and searching of literary information. *IBM Journal of research and development*, 1(4):309–317, 1957.

[70] Hans Peter Luhn. The automatic creation of literature abstracts. *IBM Journal of research and development*, 2(2):159–165, 1958.

[71] Yuanhua Lv and ChengXiang Zhai. When documents are very long, bm25 fails! In *Proceedings of the 34th international ACM SIGIR conference on Research and development in Information Retrieval*, pages 1103–1104, 2011.

[72] Lin Ma, Zhengdong Lu, and Hang Li. Learning to answer questions from image using convolutional neural network. In *Thirtieth AAAI Conference on Artificial Intelligence*, 2016.

[73] Lin Ma, Zhengdong Lu, Lifeng Shang, and Hang Li. Multimodal convolutional neural networks for matching image and sentence. In *Proceedings of the IEEE international conference on computer vision*, pages 2623–2631, 2015.

[74] Craig Macdonald and Nicola Tonellotto. Declarative experimentation in information retrieval using pyterrier. In *Proceedings of the 2020 ACM SIGIR on International Conference on Theory of Information Retrieval*, pages 161–168, 2020.

[75] ME Maron, JL Kuhns, and LC Ray. Probabilistic indexing. a statistical technique for document identification and retrieval. Technical report, THOMPSON RAMO WOOLDRIDGE INC LOS ANGELES CA, 1959.

[76] Donald Metzler and W Bruce Croft. Linear feature-based models for information retrieval. *Information Retrieval*, 10(3):257–274, 2007.

[77] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*, 2013.

[78] Bhaskar Mitra. Exploring session context using distributed representations of queries and reformulations. In *Proceedings of the 38th international ACM SIGIR conference on research and development in information retrieval*, pages 3–12, 2015.

[79] Bhaskar Mitra and Nick Craswell. Query auto-completion for rare prefixes. In *Proceedings of the 24th ACM international on conference on information and knowledge management*, pages 1755–1758, 2015.

[80] Bhaskar Mitra, Nick Craswell, et al. An introduction to neural information retrieval. *Foundations and Trends® in Information Retrieval*, 13(1):1–126, 2018.

[81] Yasufumi Moriya and Gareth JF Jones. Dcu-adapt at the trec 2020 podcasts track. In *TREC*, 2020.

[82] Burt Nanus. The use of electronic computers for information retrieval. *Bulletin of the Medical Library Association*, 48(3):278, 1960.

[83] Jamal Abdul Nasir, Iraklis Varlamis, and Samreen Ishfaq. A knowledge-based semantic framework for query expansion. *Information processing & management*, 56(5):1605–1617, 2019.

[84] Tri Nguyen, Mir Rosenberg, Xia Song, Jianfeng Gao, Saurabh Tiwary, Rangan Majumder, and Li Deng. Ms marco: A human generated machine reading comprehension dataset. In *CoCo@ NIPs*, 2016.

[85] Rodrigo Nogueira, Jimmy Lin, and AI Epistemic. From doc2query to doctttttquery. *Online preprint*, 6, 2019.

[86] Rodrigo Nogueira, Wei Yang, Kyunghyun Cho, and Jimmy Lin. Multi-stage document ranking with bert. *arXiv preprint arXiv:1910.14424*, 2019.

[87] Kezban Dilek Onal, Ye Zhang, Ismail Sengor Altingovde, Md Mustafizur Rahman, Pinar Karagoz, Alex Braylan, Brandon Dang, Heng-Lu Chang, Henna Kim, Quinten McNamara, et al. Neural information retrieval: At the end of the early years. *Information Retrieval Journal*, 21(2):111–182, 2018.

[88] MIT OpenCourseWare. Program evaluation findings summary. *Massachusetts Institute of Technology OpenCourseWare, Massachusetts*, 2011.

[89] Pavel Pecina, Petra Hoffmannova, Gareth JF Jones, Ying Zhang, and Douglas W Oard. Overview of the clef-2007 cross-language speech retrieval track. In *Workshop of the Cross-Language Evaluation Forum for European Languages*, pages 674–686. Springer, 2007.

[90] Guangyuan Piao and John G Breslin. Analyzing mooc entries of professionals on linkedin for user modeling and personalized mooc recommendations. In *Proceedings of the 2016 conference on user modeling adaptation and personalization*, pages 291–292, 2016.

[91] Jay M Ponte and W Bruce Croft. A language modeling approach to information retrieval. In *ACM SIGIR Forum*, volume 51, pages 202–208. ACM New York, NY, USA, 2017.

[92] Ronak Pradeep, Rodrigo Nogueira, and Jimmy Lin. The expando-mono-duo design pattern for text ranking with pretrained sequence-to-sequence models. *arXiv preprint arXiv:2101.05667*, 2021.

[93] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, Peter J Liu, et al. Exploring the limits of transfer learning with a unified text-to-text transformer. *J. Mach. Learn. Res.*, 21(140):1–67, 2020.

[94] Stephen Robertson, Hugo Zaragoza, et al. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends® in Information Retrieval*, 3(4):333–389, 2009.

[95] Stephen E Robertson. The probability ranking principle in ir. *Journal of documentation*, 1977.

[96] Stephen E. Robertson and Micheline M Hancock-Beaulieu. On the evaluation of ir systems. *Information Processing & Management*, 28(4):457–466, 1992.

[97] Anurag Roy, Shalmoli Ghosh, Kripabandhu Ghosh, and Saptarshi Ghosh. An unsupervised normalization algorithm for noisy text: A case study for information retrieval and stance detection. *Journal of Data and Information Quality (JDIQ)*, 13(3):1–25, 2021.

[98] Arya Roy. Recent trends in named entity recognition (ner). *arXiv preprint arXiv:2101.11420*, 2021.

[99] Gerard Salton and Christopher Buckley. Term-weighting approaches in automatic text retrieval. *Information processing & management*, 24(5):513–523, 1988.

[100] Mark Sanderson and W Bruce Croft. The history of information retrieval research. *Proceedings of the IEEE*, 100(Special Centennial Issue):1444–1451, 2012.

[101] Hinrich Schütze, Christopher D Manning, and Prabhakar Raghavan. *Introduction to information retrieval*, volume 39. Cambridge University Press Cambridge, 2008.

[102] Amartya Sen. Development as freedom (1999). *The globalization and development reader: Perspectives on development and global change*, 525, 2014.

[103] Yelong Shen, Xiaodong He, Jianfeng Gao, Li Deng, and Grégoire Mesnil. Learning semantic representations using convolutional neural networks for web search. In *Proceedings of the 23rd international conference on world wide web*, pages 373–374, 2014.

[104] Yang Song, Ali Mamdouh Elkahky, and Xiaodong He. Multi-rate deep learning for temporal recommendation. In *Proceedings of the 39th International ACM SIGIR conference on Research and Development in Information Retrieval*, pages 909–912, 2016.

[105] Damiano Spina, Johanne R Trippas, Lawrence Cavedon, and Mark Sanderson. Extracting audio summaries to support effective spoken document search. *Journal of the Association for Information Science and Technology*, 68(9):2101–2115, 2017.

[106] Rohail Syed and Kevyn Collins-Thompson. Retrieval algorithms optimized for human learning. In *Proceedings of the 40th international ACM SIGIR conference on research and development in information retrieval*, pages 555–564, 2017.

[107] Diana Tanase. Using the divergence framework for randomness: Chic 2013 lab report. In *CLEF (Working Notes)*, 2013.

[108] Mortimer Taube, Cloyd D Gull, and Irma S Wachtel. Unit terms in coordinate indexing. *Journal of the American Society for Information Science*, 3(4):213, 1952.

[109] Terrier. Divergence from randomness (dfr) framework.

[110] Shari Thurow. Optimizing for re-finding search behavior, Mar 2022.

[111] Ghazaleh H Torbati, Andrew Yates, and Gerhard Weikum. Personalized entity search by sparse and scrutable user profiles. In *Proceedings of the 2020 Conference on Human Information Interaction and Retrieval*, pages 427–431, 2020.

[112] Jana Varlejs. Ralph shaw and the rapid selector. In *Proceedings of the 1998 Conference on the History and Heritage of Science Information Systems*, pages 148–155. Citeseer, 1999.

[113] Ellen M Voorhees. The evolution of cranfield. In *Information retrieval evaluation in a changing world*, pages 45–69. Springer, 2019.

[114] Ellen M Voorhees, Donna K Harman, et al. *TREC: Experiment and evaluation in information retrieval*, volume 63. Citeseer, 2005.

[115] Ji Wan, Dayong Wang, Steven Chu Hong Hoi, Pengcheng Wu, Jianke Zhu, Yongdong Zhang, and Jintao Li. Deep learning for content-based image retrieval: A comprehensive study. In *Proceedings of the 22nd ACM international conference on Multimedia*, pages 157–166, 2014.

[116] Barbara M Wildemuth and Ann L O'Neill. The" known" in known-item searches: Empirical support for user-centered design (research note). *College & Research Libraries*, 56(3):265–281, 1995.

[117] Liu Yang, Qi Guo, Yang Song, Sha Meng, Milad Shokouhi, Kieran McDonald, and W Bruce Croft. Modeling user interests for zero-query ranking. In *European Conference on Information Retrieval*, pages 171–184. Springer, 2016.

[118] Li Yuan, Sheila MacNeill, and Willem G Kraan. Open educational resources-opportunities and challenges for higher education. 2008.

[119] Yisong Yue, Thomas Finley, Filip Radlinski, and Thorsten Joachims. A support vector method for optimizing average precision. In *Proceedings of the 30th annual international ACM SIGIR conference on Research and development in information retrieval*, pages 271–278, 2007.

[120] Fattane Zarrinkalam, Hossein Fani, Ebrahim Bagheri, and Mohsen Kahani. Predicting users' future interests on twitter. In *European Conference on Information Retrieval*, pages 464–476. Springer, 2017.

[121] Fattane Zarrinkalam, Stefano Faralli, Guangyuan Piao, Ebrahim Bagheri, et al. Extracting, mining and predicting users' interests from social media. *Foundations and Trends® in Information Retrieval*, 14(5):445–617, 2020.

[122] Ye Zhang, Md Mustafizur Rahman, Alex Braylan, Brandon Dang, Heng-Lu Chang, Henna Kim, Quinten McNamara, Aaron Angert, Edward Banner, Vivek Khetan, et al. Neural information retrieval: A literature review. *arXiv preprint arXiv:1611.06792*, 2016.

Appendix A

Submission to PASIR 2022

Towards Proactive Information Retrieval with Wikipedia Concepts

Anonymous*,†

[1]Anonymous Institute

Abstract

Extracting useful information from the user history to clearly understand informational needs is a key feature of a proactive information retrieval system. When it comes to understanding information and relevance, Wikipedia can provide the background knowledge an intelligent system needs. This work explores how exploiting the context of a query using Wikipedia concepts can improve proactive information retrieval. We formulate two models that use entity linking to associate Wikipedia topics to the relevance model. Our experiments around a podcast segment retrieval task demonstrate that the there is a clear signal of relevance in Wikipedia concepts while a ranking model can improve precision by incorporating them. We also find Wikifying the background context of a query can be help disambiguate the meaning of the query further helping proactive information retrieval.

1. Introduction

The informational needs of people are highly contextual and can depend on many different factors such as their current knowledge state, interests and goals [1, 2]. However, one of the foundational components of creating an effective information companion is to minimise the human effort required in ii) expressing a human information need and ii) digesting a informational result set. With the information overload, both of the above tasks have become challenging posing struggles in creating AI systems that can cooperate with human users to support their informational journey. With the emergence of the semantic web, the majority of digital information in today's world is enriched with metadata that is easily manageable with machines. Developing Artificial Intelligence (AI) that can mildly, at least understand how information and knowledge is structured can help address this struggle. In this paper, we argue that Wikipedia, an openly available encyclopedia, can be a humanly intuitive knowledge base that has the potential to feed the world view many information retrieval systems need. With the vast understanding of the universal knowledge it can feed into an intelligent information retrieval system, we point out that it can both help refine user informational needs by making clearer sense of user queries (both explicit and implicit) while compiling large result sets into structured narratives that human users can navigate with optimal effort and higher engagement.

Woodstock'22: Symposium on the irreproducible science, June 07–11, 2022, Woodstock, NY
anon@anonymous.org (Anonymous)
https://anonymous.anon.io/ (Anonymous)
0000-0002-0877-7063 (Anonymous)
© 2022 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).
CEUR Workshop Proceedings (CEUR-WS.org)

2. Related Work

Retrieving relevant documents to a user information need, involves mining features from the textual documents that can be used to match them to user needs.

2.1. Retrieving Relevant Documents

Many different approaches in information retrieval rely on textual features of the user queries and the documents to compute relevance. Probabilistic relevance models that rely on the bag-of-words representation of documents and queries such as BM25 [3] are foundational to information retrieval. Prior evidence has demonstrated the success of BM25 with the exception of failure on long documents [4]. However, more sophisticated probabilistic models such as DPH [5], motivated by the Divergence from Randomness (DFR) model based on a hypergeometric distribution and Popper normalisation instead of Laplace normalization, are outperforming simpler models like BM25 [6].

Amid the information overload, information retrieval applications have become critical to improving human information seeking scenarios. This need has led to competitions such as the podcast segment retrieval challenge [7] that has paved way to improving the state-of-the-art. While more sophisticated machine learning based and deep learning-based ranking models [8, 9] have emerged over time, it is still evident that simple count-based ranking algorithms still dominate solution tables [6]. This is due to multiple reasons, The probabilistic bag-of-words methods are word token based, making them significantly more explainable and interpretable in comparison to their complex counterparts. The computation cost of the simpler probabilistic models is also substantially lower, usually only having to conduct hyperparameter tuning and avoid training altogether. Such computationally efficient models are also much simpler to operate in real world scenarios attracting the majority of the community to still use them. Even within the competition, while the best performing model uses neural embeddings, they use neural embedding of tokens rather than a neural representation of the documents [10]. The model also uses a re-ranking model trained on a different dataset adding to computational costs significantly. The Dublin City Univerity (DCU) model uses a variety of features ranging from text, WordNet based synonyms and entities for expanding the query [11]. As the run 2 described in their model uses automatically extracted entities to expand the query, we consider this work as the most relevant model to our automatic enrichment of query (and segments) with Wikipedia concepts/entities. User queries, inherently, are short averaging two to three key words in general [12]. Exploiting auxiliary information available to recover the context of the search is essential in such scenarios. In recent competitions such as the podcast segment retrieval challenge, the queries used were accompanied by a small description that provided the contextual information about the query itself. It was evidenced that the leading models in this challenge had to use the description to extract features and sometimes use the identified keywords to make queries to external search engines to further expand the queries [6]. However, in a real world system, such information is not available to the proactive retrieval system and hence it has to rely on the user interactions from the past [13].

Proactive information retrieval is a field in information retrieval that gained a lot of attention recently. Supporting user information queries with minimal/zero effort will enable users to use

Figure A.2: Submission to PASIR 2022 page 2

information retrieval systems more effectively. However, a key part of facilitating information retrieval tasks proactively is building systems that can capture the relevant signal from the user that indicate the informational needs of them. Prior works in this domain have identified several ways to harvest user intent. The simplest of forms in this direction is to explicitly ask questions [14] or use the demographic information of the user [15] to profile them. Rather than hindering user experience by explicitly disrupting user experience, many works harvest implicit user actions such as clicks, impressions and previous searches [16, 15]. Especially in the social media domain, the interactions of users are used to create concept-based models [17] that can be used to infer user preferences and create zero effort queries for new content. The extracted preference features are usually used to expand information queries [18, 19], synthesise zero-effort queries [17] or personalise search results [20]. In this work, we argue in support of using concept-based user models as a form of zero-effort query for informational recommendation and learning scenarios.

2.2. Concept-based User Modelling and Wikipedia

While concept-based user modelling can be utilised to compose zero-effort queries using user past interactions, there are many definitions for a *concept* in this application domain. In the field of document retrieval and personalisation, keyword extraction is a heavily used form of identifying concepts from textual documents. When working with short text such as social media posts, the common method is to use the words in the posts as words and build a user profile over these words [21]. While this approach provide fine grained features, the feature space can be too vast and pose challenges in increasing recall. To address this issue, other systems use and topic detection using unsupervised learning approaches such as LDA [22]. Such unsupervised techniques are complex to tune and are not guaranteed to give humanly intuitive topics. While expert annotation (e.g. in education [23]) has been one of the approaches to obtain humanly intuitive, representative topics from textual content, this approach is not scalable. **Wikification**, a more recent approach, looks promising towards automatically extracting explainable concepts from text. Wikification identifies Wikipedia concepts present in a document by connecting natural text to Wikipedia articles via entity linking [24]. In this work, we argue for the usefulness of Wikification in creating precise, humanly intuitive concept representations at scale. Our experiments also demonstrate how Wikipedia concepts can be used to i) disambiguate query keyword and ii) at the absence of them, create entity features that can find relevant documents for users proactively.

3. Wikipedia to Support Proactive Search-based Systems

Through this work, we aim to explore how the knowledge contained in Wikipedia can be utilised to better a proactive search based system. In order to tread towards this direction, we aim to answer multiple research questions in this work as a promising step. We hypothesise that associating Wikipedia concepts to a natural language based IR system can improve a proactive IR system by i) bettering disambiguation of the meaning of user queries, ii) by exploiting the semantic meaning of words to identify relevant information and iii) by creating opportunities for the system to understand the learner interactions more meaningfully.

Figure A.3: Submission to PASIR 2022 page 3

RQ1 Does Wikipedia concepts carry a signal that indicates relevance of documents to queries?

RQ2 Can Wikipedia annotations improve information retrieval?

Through this work, we conduct a few preliminary experiments to validate these research questions. We choose the segment retrieval task from the Podcast Track at TREC 2020 [7] for these experiments as search is a key component of proactive information retrieval. We use this task specifically as the query description is provided as part of the query, providing the context of the query. In many proactive IR systems, the query context is mined using the user interaction history. In this case, the description can be treated as a context representation mined from the users' prior interactions.

3.1. Information Retrieval Models

We restrict our model choices to a probabilistic retrieval method BM25 and a hypergeometric weighing model DPH. The reasons for restricting to probabilistic models are i) its superior performance [6], ii) the simplicity to introducing Wikipedia concept features, iii) data efficiency and iv) practical feasibility to real world systems. The models we utilise in this these experiments use the query q, query description d and the segment s.

3.1.1. Baseline Models

We use BM25 and DPH models as probabilistic baseline models for this work. These two models use the textual tokens of the query q and the segment s to compute the relevance score rel as per equation 1 where `txt` identifies the text token features..

$$rel(q, s) = f(q_{\texttt{txt}}, s_{\texttt{txt}}) \tag{1}$$

As described in section 2, we implement and test DCU run 2 model [11] that expands the query with entities extracted from the description as per equation 2 that expands equation 1. The entities are extracted using Named Entity Recognition(NER).

$$rel(q, d, s) = f(q_{\texttt{txt}} + d_{\texttt{ent}}, s_{\texttt{txt}}) \tag{2}$$

where `ent` represents the entities extracted from the query description d.

3.2. Proposed Models

The proposed models in this work uses Wikipedia concepts to enrich the representations used in the IR model. We hypothesise that adding Wikipedia concepts can improve precision of the model as exact entities can be matched between the query + context and the segments. We propose two models that use Wikipedia topics both in query and segment expansion.

Figure A.4: Submission to PASIR 2022 page 2

Wiki_rel , This model expands the query and the document by adding the Wikipedia concepts extracted using both the query q and the query description d as per equation 4. This model replaces the entities in the DCU model by Wikipedia concepts.

$$rel(q, d, s) = f(q_{\text{txt}} + q_{\texttt{wiki}} + d_{\texttt{wiki}}, s_{\text{txt}} + s_{\texttt{wiki}}) \tag{3}$$

where `wiki` represents the concepts extracted from the query description d.

Ent_Wiki_rel , This model expands the DCU model by enriching the query, description and the segment with Wikipedia concepts. This formulation is presented in equation 4.

$$rel(q, d, s) = f(q_{\text{txt}} + d_{\text{ent}} + q_{\texttt{wiki}} + d_{\texttt{wiki}}, s_{\text{txt}} + s_{\texttt{wiki}}) \tag{4}$$

The two proposed models help us validate RQ2 by i) Wiki_rel model validating if *replacing* named-entities with Wikipedia features improves the ranking models and ii) Wiki_Ent_rel model validating if *adding* Wikipedia features improves the model.

3.3. Data

We per section 3, we use the Podcast segment retrieval task to demonstrate usefulness of Wikipedia features in proactive information retrieval. The Spotify podcast Dataset contains approx 100,000 transcripts from around 60,000 hours of audio data culminating in the largest corpus of transcribed speech data. The 100k episodes in the dataset were randomly sampled from 105,360 English podcast episodes published between January 1, 2019 to March 1, 2020 on Spotify with 10% of the podcasts from professional creators with high production values and the other 90% coming from amateurs. The data is also provided with training and testing topics (queries) along with human relevance judgements. There are 8 topics in the training dataset and another 50 topics in the test set.

To keep the computational costs low, we run the experiments outlined in section 3.4 exclusively using the training data in the podcast dataset. We first transform the dataset into two minute overlapping segments as described in [6]. Then, we take all the relevant segments as positive documents for the different queries. As there is a substantial number of segments that are irrelevant (as the whole dataset contains 3.5 Mn segments), we down sample this data and get a negative segment set of $\approx$ 14000 segments that are not relevant to any of the labelled queries. This set of positive and negative segments make up the smaller scale dataset that we use in our empirical experiments. We refer to this dataset as `Podcast Small` dataset. We use the Wikifier [24] to associate Wikipedia concepts to both the queries (query + description) and the podcast segments as the models in section 3.1 require.

3.4. Experiments

The empirical experiments we run in this work aim to answer RQ 1 and 2. We use the Podcast Small dataset for these experiments. In a proactive information retrieval setting, the user queries are inferred from users' historical interactions. We treat the query description provided in the dataset in the place of the features extracted from the user history. We hypothesise that the

Figure A.5: Submission to PASIR 2022 page 5

query description, can be used in two ways, i) as the user context that can be used in a zero effort query to make up the context and rank relevant documents from the corpus and/or ii) as a mean to disambiguate the true meaning of a short query provided by the user without prompting the user to provide further clarifications.

To answer RQ1, we aim to see if there is a significant alignment between the Wikipedia concepts present in the query and a relevant document in contrast to a non-relevant document. To measure this quantitatively, we measure the Jaccard similarity coefficient ρ between the query and the document. Then, for each query, we take the sets of values ρ_{rel} and $\rho_{\text{non rel}}$ and compare the difference of median using a one-tailed Mann–Whitney U test ($H_{alt.} : \rho_{\text{rel}} > \rho_{\text{non rel}}$).

To answer RQ2, we take the annotated Wikipedia concepts to the query and the podcast segments as additional features. Then we develop a new set of information retrieval algorithms outlined in section 3.1 to account for the alignment of Wikipedia concepts present. Then we use the same Podcast Small dataset to evaluate if the ranking of relevant documents is different between the baselines and the proposed models. To validate how Wikifying the context can be used to disambiguate a query, we ran Wikification on each query using i) the query words exclusively and ii) the query words + the description words, to observe if more precise identification of Wikipedia concepts is carried out.

In order to run the evaluation experiments needed for RQ2, we utilise `Python Terrier` library [25]. The text processing and NER enrichment needed for the DCU model is done using the `Spacy` library [26]. The Wikipedia annotations are sourced using the `Wikifier` service [24].

3.4.1. Evaluation

To evaluate if the Jaccard similarity coefficients in relation to RQ1 are significantly different, we use the test statistic of the hypothesis test. We present the performance of the ranking models related to validating RQ2 using NDCG, NDCG@30 and Precision@10 as these are the same evaluation metrics used in prior work [6]. NDCG uses graded relevance instead of binary relevance/non-relevance to focus on highly relevant documents, implying that retrieving a highly relevant document is much more important than retrieving a less important document. Whereas precision measures the number of relevant documents retrieved over the total relevant documents. Similar to the prior work, we calculate NDCG for the entire set of relevant segments available and up-till rank 30. Precision is computed by using a cut-off of top-10 ranked segments.

4. Experimental Results

The results of the experiments are reported in this section. Figure 1 show the difference of medians of Jaccard similarity coefficient while Table 1 summarises the results obtained from the hypothesis test conducted to verify RQ1. Table 2 outlines the ranking performance obtained in the experiments run in order to verify RQ2.

4.1. Query Keyword Disambiguation using the Context

We hyptothesise that the context gathered by the user can also be used to disambiguate the meaning of keywords in a search query. To test this, we take 7 queries from the training dataset

Figure A.6: Submission to PASIR 2022 page 6

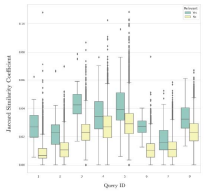


Figure 1: Jaccard similarity distribution of relevant vs. non-relevant segments for each query.

Table 1
The difference of medians test for each query.

	Number of Documents		Median Jaccard Similarity			MannWhitney U test	
Query ID	Relevant	Non-Relevant	Relevant	Non-Relevant	Difference	Test Statistic	p value
1	70	14109	0.027	0.007	0.020	25680	1.03E-49
2	63	14116	0.023	0.011	0.012	47466	8.84E-46
3	78	14101	0.043	0.023	0.019	264688	1.22E-16
4	78	14101	0.034	0.027	0.007	393033	1.41E-06
5	80	14099	0.039	0.029	0.010	455015	1.42E-03
6	37	14142	0.027	0.010	0.017	35748	8.37E-48
7	77	14102	0.016	0.011	0.005	56322	2.80E-44
8	80	14099	0.032	0.023	0.010	271961	6.31E-16

Table 2
Ranking performance of models. The best performance in **bold** and the second best in *italic*.

	Features			Metrics		
Model	Query Text	NER Entities	Wiki Concepts	NDCG	NDCG at 30	Precision at 10
Baselines						
DPH	×	o	o	0.48	0.30	*0.32*
BM25	×	o	o	0.48	0.28	0.31
DCU	×	×	o	**0.51**	**0.32**	0.30
New Proposals						
Wiki_rel	×	o	×	*0.49*	0.29	0.31
Ent_Wiki_rel	×	×	×	**0.51**	*0.30*	**0.36**

that belongs to categories *topical* and *known item* ("story about riding a bird" is ignored as it is a query that belongs to the *refinding* category). We enrich these queries using Wikification in two settings, i) query only vs. ii) query + description, to investigate if the identified Wiki concepts are different. Table 3 reports how the salient entities in the query keywords are detected differently in the two settings.

Figure A.7: Submission to PASIR 2022 page 7

Table 3
Query Word Disambiguation when using the query exclusively vs. using the description.

Query	Query Only	Query + Description
coronavirus spread	wiki/Coronavirus	wiki/Novel_coronavirus
greta thunberg cross atlantic	-	wiki/Greta_Thunberg
black hole image	wiki/Black_hole	wiki/Black_hole
daniel ek interview	-	wiki/Daniel_Ek
michelle obama becoming	-	wiki/Michelle_Obama
	wiki/Becoming_(philosophy)	wiki/Becoming_(book)
anna delvey	wiki/Indian_anna	wiki/Anna_Sorokin
facebook stock prediction	wiki/Facebook	wiki/Facebook
	wiki/Stock	wiki/Stock

5. Discussion

The results obtained in the preliminary experiments conducted are very promising. Majority of the results suggest that Wikipedia concepts can help the IR system.

5.1. The Usefulness of Wikipedia Features (RQ1)

Figure 1 clearly shows that the Jaccard similarity coefficient between the relevant segments ρ_{rel} and the non-relevant documents $\rho_{\text{non rel}}$ are different from each other. It is even evident that the confidence intervals don't even overlap in some of the queries in the training data. The test statistics presented in table 1 further confirms this observation. In a one-tailed hypothesis test, all the queries demonstrate statistically significant differences of medians where the median Jaccard similarity of the Wikipedia concepts of the relevant segments and the query is greater than that of the non-relevant segments. These results enable us to conclude that the the overlap of Wikipedia concepts between queries and relevant documents is a influential signal that can be incorporated in relevance computation. These results indicate that there is promise in using Wikipedia concepts as a feature in IR systems.

5.2. Utility of Wikification in Information Retrieval (RQ2)

The usefulness of Wikipedia concepts in relevance prediction is validated in the results presented in table 2. The results in the table shows that replacing that replacing the NER-based entities with Wikipedia concepts is not as fruitful as augmenting them while the entities are retained. It is observed that both the proposed models outperform the entity based DCU model in precision at 10. This aligns with the our understanding of the effect of Wikipedia concepts. NER, while detects specific persons, organisations and other entities in the text, only adds the textual representations of these entities to the query to expand it. Therefore, the newly added queries are not capable of entity disambiguation (when the same term occurs in a segment meaning something else. e.g. Apple, the fruit vs. the company) and coreference resolution (when a different textual form is used to mean the same entity. e.g. United Nations vs. UN). On the contrary, enriching both the query and the document with Wikipedia entities enable accounting for both of these effects making the relevance calculation more precise. The *Ent_Wiki_rel* model

Figure A.8: Submission to PASIR 2022 page 8

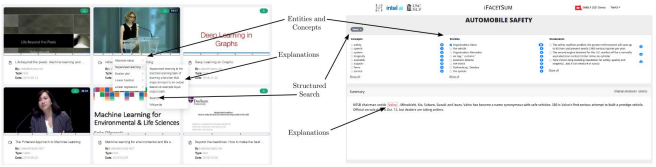


Figure 2: Test

that contains both the NER-based entities and Wikipedia concepts manage to improve the precision at 10 significantly while retaining the NDCG performance. This shows that keeping the entities in the query is important to retain the NDCG gains found in the DCU model.

In this scenario, we use the Wikipedia concepts and entities extracted from the description to improve the search ranking. While we treat the description in this scenario as a representation extracted from the user history, many concept-based user models described in section 2 support extracting features such as keywords [20] and Wikipedia concepts [1] from content.

Query Keyword Disambiguation The results in table 3 also give us insights into the potential of using query context for keyword disambiguation. It is seen that in multiple queries, certain concepts were not even ditected when the query was used exclusively for Wikificaiton (e.g. Greta Thunberg, Michelle Obama etc.). However, with the presence of context (that can be extracted from historic encounter of concepts of a user [27]), the algorithm managed to identify these concepts in the query increasing recall. Table 3 also shows instances where the precision of identification is improved in the presence of the context. The more precise *Novel Coronavirus* was detected in the first query while the system understood that the query was about Michelle Obama's book, *Becoming* in the fifth query, not the philosophical concept. These results show how many scenarios where short query keywords can be clarified automatically by referring to the context rather than needlessly prompting the human user.

5.3. Wikipedia Concepts and the User Experience

The previous section discussed one opportunity of Wikification, automatic query word disambiguation. Such features will substantially decrease the number of user prompts in a system leading to a smoother experience. Wikipedia concepts also bring the advantage of being grounded in a taxanomy that is intuitive to humans. The human intuitive nature of Wikipedia concepts allow they system to provide more transparent result sets that the user can navigate more efficiently. Figure 2 shows two recent systems demonstrating the potential using Wikipedia based representations can to improve human-in-the-loop information retrieval.

The left system, X5Learn [28] shows how different Wikipedia concepts are related to different segments of videos. The Jaccard-based relevance can be used to visualise the degree of concept overlap between the segments and the query indicating relevance (connecting to the hypothesis of RQ1). Also, using Wikipedia concepts rather than free-form keywords or NER-based entities

Figure A.9: Submission to PASIR 2022 page 9

allow the system to provide explanations of the concepts users encounter. This can be done using the knowledge base behind the taxonomy (in this case, Wikipedia as per figure 2 (left)) Many users seeking information do not fully understand the knowledge landscape they navigate and can use learning opportunities to get more familiar with the topics.

IFacetSum, a multi-faceted mix-initiative interface shown in figure 2(right) is an example of how the result sets can be summarised using the entities and concepts in the document. As shown in the demonstration, the detected entities and concepts allow the user to understand the contents of the relevant documents quickly. Their user study also found that the users didn't utilise the statements and summaries as much as they used the concepts and entities [29]. However, this system lacks the capabilities of explaining the topics as it is not backed by a knowledge base pointing to the advantages of using a taxonomy that has auxiliary information that can be used. In a nutshell, many recent visualisation approaches uses concepts to represent documents. Having Wikipedia as a topic taxonomy enables these systems to make use of the advancements of Wikification that is advancing rapidly at present. It also, allows taking use of other information in Wikipedia (e.g. semantic relatedness between topics, hierarchy of concepts etc.) to model the concepts in a much more sophisticated manner. This in turn, helps bridging the system seamlessly to the end user, creating cleaner interaction footprints that transform into the input for proactive search.

6. Conclusion

Through this work, we explore the potential of using Wikipedia concepts to drive the proactive search experience of an IR system. We explored the usefulness of Wiki concepts focusing on 3 research questions. We treated the query description in a podcast segment retrieval task dataset as the user context a proactive IR system will use and ran a series of experiments. Our first experiment showed that the overlap of Wikipedia concepts between the relevant segments and the query are significantly larger than with the non-relevant segments. A subsequent experiment that added Wikipedia concept features to the search query and the document showed that the relevance model that uses Wikipedia concepts can improve precision of the ranking model while retaining the NDCG score in comparison to a counterpart that uses entities instead of Wikipedia concepts. A deeper analysis on the Wikification of the queries also showed that the meaning of the query keywords can be enhanced by using the query context. We also demonstrated using two recent works on how Wikipedia taxonomy can help an IR system to better connect with the end user and enable them to carry out structure IR tasks by proactively helping them navigate the information in the result sets. It is seen that having Wikipedia symbols allow presenting the result set in a humanly intuitive form that encourages the user to efficiently refine their query in the subsequent steps. All the observations from the observations and the analysed systems indicate that Wikipedia concepts have a key role to play in creating proactive information retrieval systems that can facilitate a higher degree human-in-the-loop operation. In the future work, we aim to use more sophisticated representations of Wikipedia concepts and advanced relevance models to run large scale experiments on the full podcast dataset with 3.5 Mn segments. Understanding how to use Wikipedia concepts effectively (e.g. by ranking them) is a top priority.

Figure A.10: Submission to PASIR 2022 page 10

References

[1] S. Bulathwela, M. Pérez-Ortiz, E. Yilmaz, J. Shawe-Taylor, Towards an integrative educational recommender for lifelong learners, in: AAAI Conference on Artificial Intelligence, AAAI 20, 2020.

[2] W. Jiang, Z. A. Pardos, Q. Wei, Goal-based course recommendation, in: Proceedings of International Conference on Learning Analytics & Knowledge, 2019.

[3] S. Robertson, H. Zaragoza, et al., The probabilistic relevance framework: Bm25 and beyond, Foundations and Trends in Information Retrieval 3 (2009) 333–389.

[4] Y. Lv, C. Zhai, When documents are very long, bm25 fails!, in: Proceedings of the 34th international ACM SIGIR conference on Research and development in Information Retrieval, 2011, pp. 1103–1104.

[5] G. Amati, Frequentist and bayesian approach to information retrieval, in: European Conference on Information Retrieval, Springer, 2006, pp. 13–24.

[6] R. Jones, B. Carterette, A. Clifton, M. Eskevich, G. J. Jones, J. Karlgren, A. Pappu, S. Reddy, Y. Yu, Trec 2020 podcasts track overview, arXiv preprint arXiv:2103.15953 (2021).

[7] Y. Yu, J. Karlgren, H. Bonab, A. Clifton, M. I. Tanveer, R. Jones, Spotify at the trec 2020 podcasts track: Segment retrieval, in: Proceedings of the Twenty-Ninth Text REtrieval Conference (TREC 2020), 2020.

[8] T. Joachims, Optimizing search engines using clickthrough data, in: Proc. of Int. Conf. on Knowledge Discovery and Data Mining, 2002.

[9] C. Burges, R. Ragno, Q. Le, C. J. Burges, Learning to rank with non-smooth cost functions, in: Advances in Neural Information Processing Systems, 2007.

[10] P. Galušcáková, S. Nair, D. W. Oard, Combine and re-rank: The university of maryland at the trec 2020 podcasts track (2020).

[11] Y. Moriya, G. J. Jones, Dcu-adapt at the trec 2020 podcasts track., in: TREC, 2020.

[12] A. Arampatzis, J. Kamps, A study of query length, in: Proceedings of the 31st annual international ACM SIGIR conference on Research and development in information retrieval, 2008, pp. 811–812.

[13] P. Sen, Proactive information retrieval, Ph.D. thesis, Dublin City University, 2021.

[14] G. H. Torbati, A. Yates, G. Weikum, Personalized entity search by sparse and scrutable user profiles, in: Proceedings of the 2020 Conference on Human Information Interaction and Retrieval, 2020, pp. 427–431.

[15] L. Yang, Q. Guo, Y. Song, S. Meng, M. Shokouhi, K. McDonald, W. B. Croft, Modeling user interests for zero-query ranking, in: European Conference on Information Retrieval, Springer, 2016, pp. 171–184.

[16] S. Bulathwela, M. Perez-Ortiz, E. Novak, E. Yilmaz, J. Shawe-Taylor, Peek: A large dataset of learner engagement with educational videos, in: Proc. of RecSys Workshop on Online Recommender Systems and User Modeling (ORSUM'21), 2021. URL: https://arxiv.org/abs/2109.03154.

[17] F. Zarrinkalam, S. Faralli, G. Piao, E. Bagheri, Extracting, mining and predicting users' interests from social media (2020).

[18] J. A. Nasir, I. Varlamis, S. Ishfaq, A knowledge-based semantic framework for query expansion, Information processing & management 56 (2019) 1605–1617.

Figure A.11: Submission to PASIR 2022 page 11

[19] H. K. Azad, A. Deepak, A new approach for query expansion using wikipedia and wordnet, Information sciences 492 (2019) 147–163.
[20] R. Syed, K. Collins-Thompson, Retrieval algorithms optimized for human learning, in: Proc. of Int. Conf. on Research and Development in Information Retrieval (SIGIR), 2017.
[21] G. Piao, J. G. Breslin, Analyzing mooc entries of professionals on linkedin for user modeling and personalized mooc recommendations, in: Proceedings of the 2016 Conference on User Modeling Adaptation and Personalization, UMAP '16, 2016.
[22] F. Zarrinkalam, H. Fani, E. Bagheri, M. Kahani, Predicting users' future interests on twitter, in: European Conference on Information Retrieval, Springer, 2017, pp. 464–476.
[23] A. T. Corbett, J. R. Anderson, Knowledge tracing: Modeling the acquisition of procedural knowledge, User Modeling and User-Adapted Interaction 4 (1994).
[24] J. Brank, G. Leban, M. Grobelnik, Annotating documents with relevant wikipedia concepts, in: Proc. of Slovenian KDD Conf. on Data Mining and Data Warehouses (SiKDD), 2017.
[25] C. Macdonald, N. Tonellotto, Declarative experimentation in information retrieval using PyTerrier, in: Proceedings of the 2020 ACM SIGIR on International Conference on Theory of Information Retrieval, ACM, 2020. doi:10.1145/3409256.3409829.
[26] S. V. L. Matthew Honnibal, Ines Montani, A. Boyd, spacy: Industrial-strength natural language processing in python., 2020. URL: https://zenodo.org/record/6907665#.YwnNPXbMK4Q.
[27] S. Bulathwela, M. Pérez-Ortiz, E. Yilmaz, J. Shawe-Taylor, Truelearn: A family of bayesian algorithms to match lifelong learners to open educational resources, in: AAAI Conference on Artificial Intelligence, AAAI 20, 2020.
[28] S. Bulathwela, S. Kreitmayer, M. Pérez-Ortiz, What's in it for me? augmenting recommended learning resources with navigable annotations, in: Proceedings of the 25th International Conference on Intelligent User Interfaces Companion, IUI 20, 2020.
[29] E. Hirsch, A. Eirew, O. Shapira, A. Caciularu, A. Cattan, O. Ernst, R. Pasunuru, H. Ronen, M. Bansal, I. Dagan, iFacetSum: Coreference-based interactive faceted summarization for multi-document exploration, in: Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, Association for Computational Linguistics, Online and Punta Cana, Dominican Republic, 2021, pp. 283–297. URL: https://aclanthology.org/2021.emnlp-demo.33. doi:10.18653/v1/2021.emnlp-demo.33.

Figure A.12: Submission to PASIR 2022 page 12

Appendix B

Other Charts *Podcast Small*

B.1 Topic-wise Relevant vs Non-relevant Based on Cosine Similarity Score cut off at rank 1,2,3,5,10 and 20

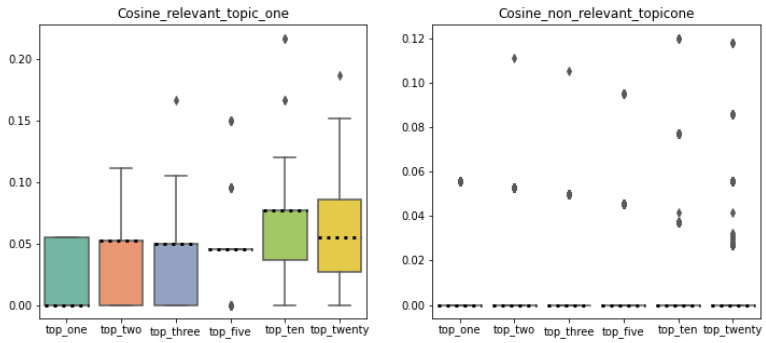


Figure B.1: Topic 1 relevant vs non-relevant segments (Jaccard Similarity Score on Cosine Similarity Score Rank Cut-offs

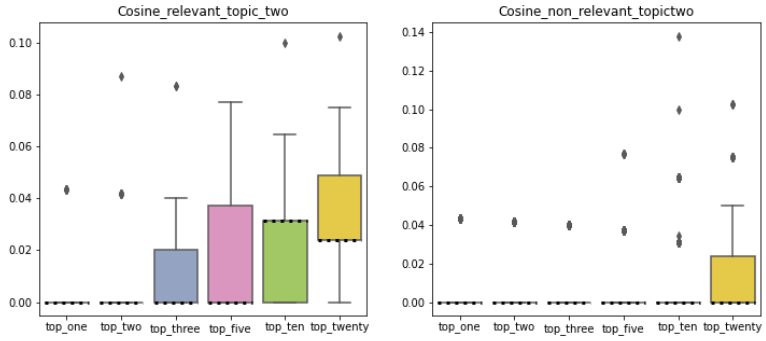


Figure B.2: Topic 2 relevant vs non-relevant segments (Jaccard Similarity Score on Cosine Similarity Score Rank Cut-offs

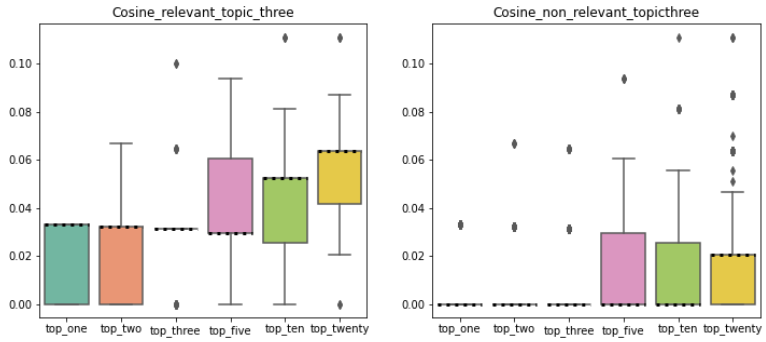


Figure B.3: Topic 3 relevant vs non-relevant segments (Jaccard Similarity Score on Cosine Similarity Score Rank Cut-offs

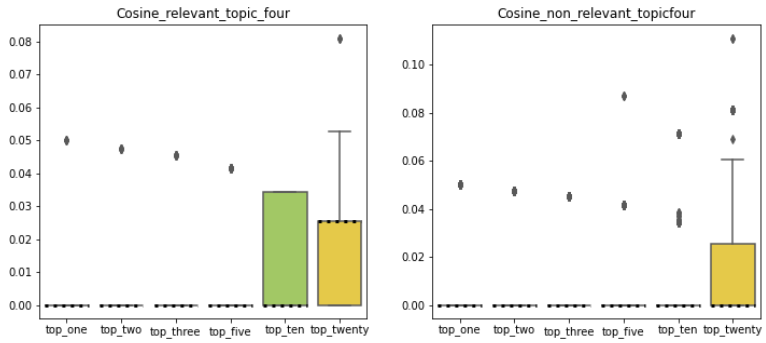


Figure B.4: Topic 4 relevant vs non-relevant segments (Jaccard Similarity Score on Cosine Similarity Score Rank Cut-offs

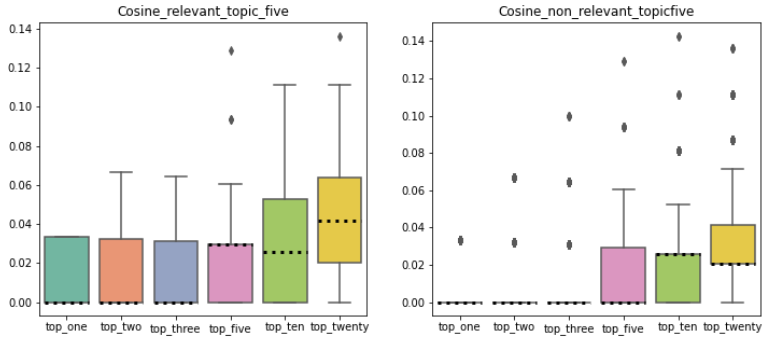


Figure B.5: Topic 5 relevant vs non-relevant segments (Jaccard Similarity Score on Cosine Similarity Score Rank Cut-offs

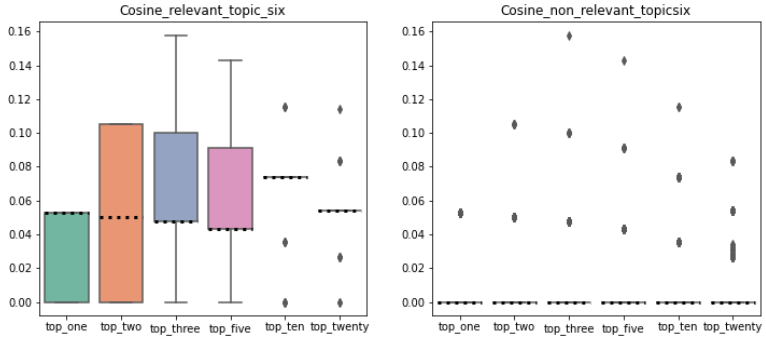


Figure B.6: Topic 6 relevant vs non-relevant segments (Jaccard Similarity Score on Cosine Similarity Score Rank Cut-offs

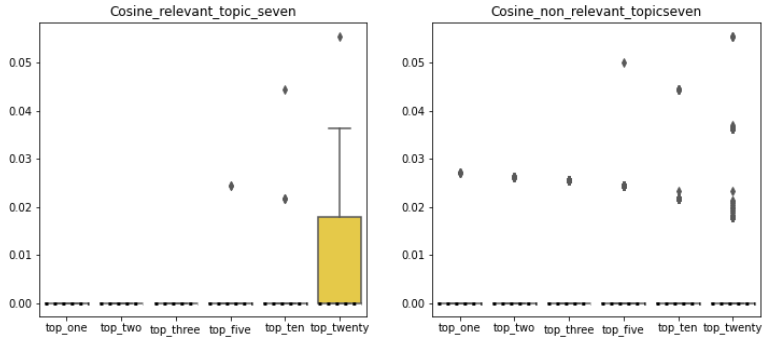


Figure B.7: Topic 7 relevant vs non-relevant segments (Jaccard Similarity Score on Cosine Similarity Score Rank Cut-offs

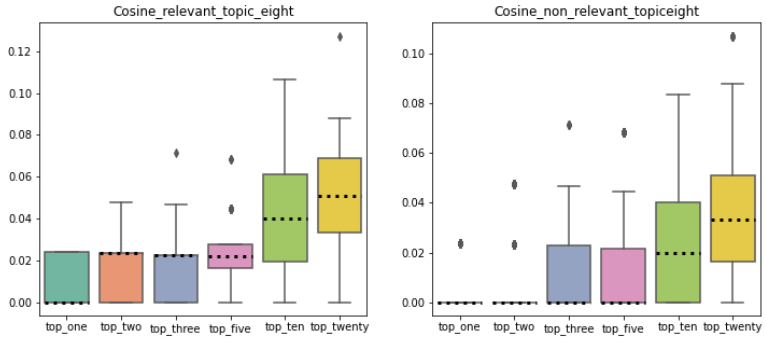


Figure B.8: Topic 8 relevant vs non-relevant segments (Jaccard Similarity Score on Cosine Similarity Score Rank Cut-offs

B.2 Topic-wise Relevant vs Non-relevant Based on Page-rank Score cut off at rank 1,2,3,5,10 and 20

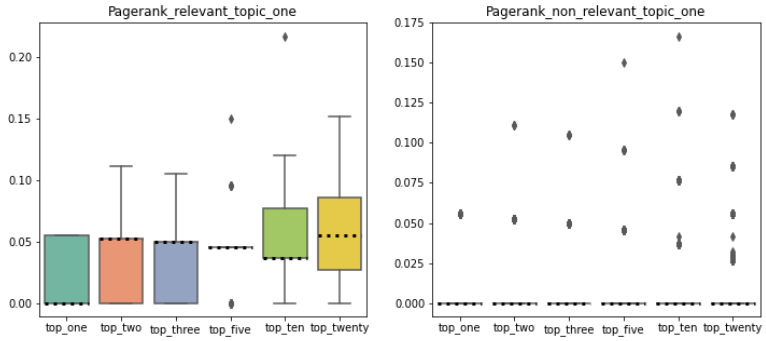


Figure B.9: Topic 1 relevant vs non-relevant segments (Jaccard Similarity Score on Page-rank Score rank cut-offs

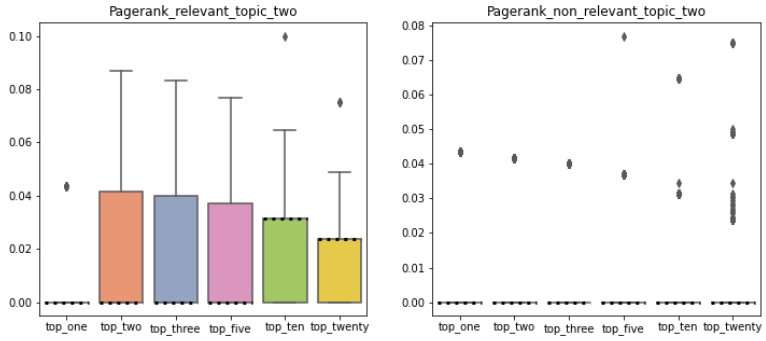


Figure B.10: Topic 2 relevant vs non-relevant segments (Jaccard Similarity Score on Page-rank Score rank cut-offs

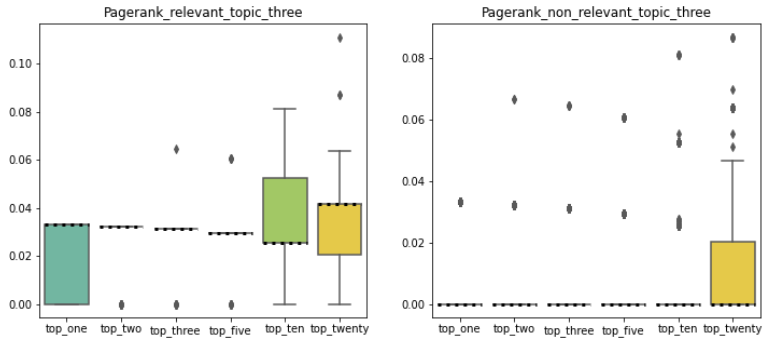


Figure B.11: Topic 3 relevant vs non-relevant segments (Jaccard Similarity Score on Page-rank Score rank cut-offs

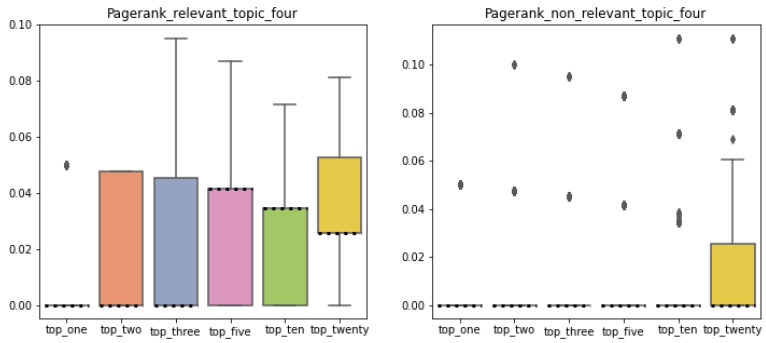


Figure B.12: Topic 4 relevant vs non-relevant segments (Jaccard Similarity Score on Page-rank Score rank cut-offs

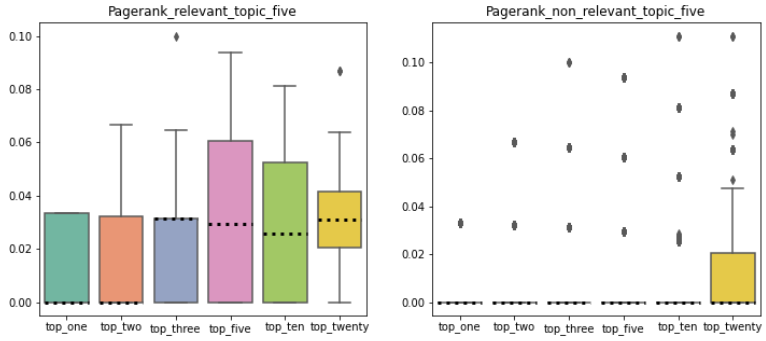


Figure B.13: Topic 5 relevant vs non-relevant segments (Jaccard Similarity Score on Page-rank Score rank cut-offs

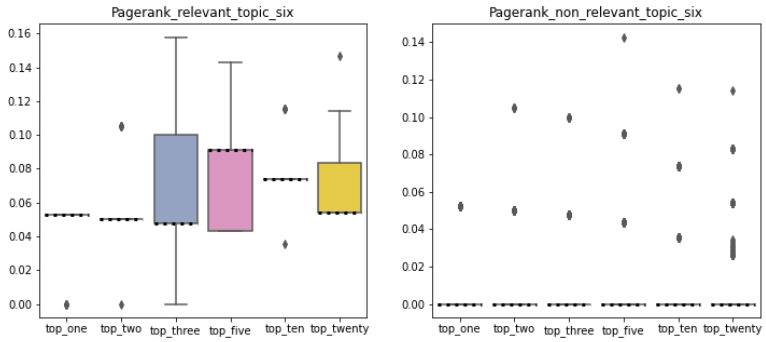


Figure B.14: Topic 6 relevant vs non-relevant segments (Jaccard Similarity Score on Page-rank Score rank cut-offs

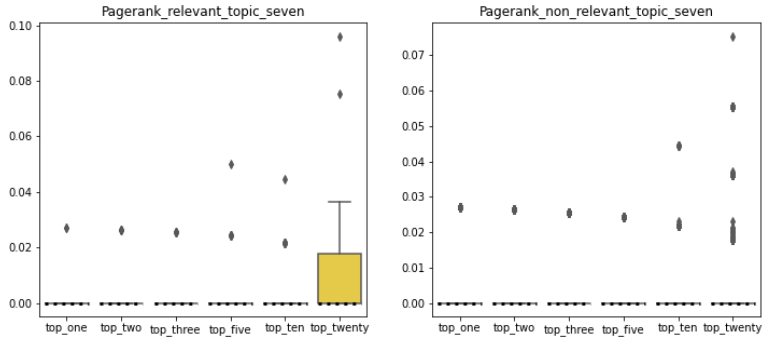


Figure B.15: Topic 7 relevant vs non-relevant segments (Jaccard Similarity Score on Page-rank Score rank cut-offs

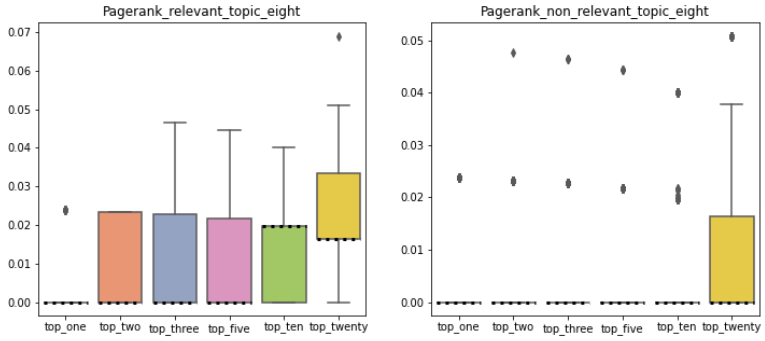


Figure B.16: Topic 8 relevant vs non-relevant segments (Jaccard Similarity Score on Page-rank Score rank cut-offs

Appendix C

Code

All code implementations can be found at https://github.com/tab-mus-33/Noisy-IR

Since the project involves around 400+ GB of additional data dependency, the files can be found on https://drive.google.com/drive/folders/1SdID4kNAE5y3MR2PuDXuCC8GfGQkihuW?usp=sharing
The Baselines and LTR approaches can be run through https://github.com/tab-mus-33/Noisy-IR/blob/main/Basleline_Runs_%2B_QE_%2B_LTR.ipynb

The file assumes conversion to TREC XML format from JSON files and can be done using https://github.com/tab-mus-33/Noisy-IR/blob/main/Convert_to_Trec_XML.ipynb which in turn uses https://github.com/tab-mus-33/Noisy-IR/blob/main/converter.py

Neural Approaches : https://github.com/tab-mus-33/Noisy-IR/blob/main/Nueral_Re_ranking.ipynb

NER Approaches with DCU Replication : https://github.com/tab-mus-33/Noisy-IR/blob/main/Named_Entity_Recognition.ipynb

Proactive IR Proposed on Podcast Small : https://github.com/tab-mus-33/Noisy-IR/blob/main/Proactive_IR_Experiments_train(Podcast_Small).ipynb

Proactive IR Proposed on Podcast SE : https://github.com/tab-mus-33/Noisy-IR/blob/main/Proactive_IR_Experiments_train(Podcast_Small).ipynb